

SoundSpace

AI기반 3단계 음악 검증 시스템

이성훈, 정혜원, 조우성, 최두석, 김기열

2026.02.13.



Now Playing
Presentation Intro.mp3



Contents

[별첨]

테이블 정의서

업무 흐름도 상세

단위 테스트

통합 테스트

1 프로젝트 개요

- 1.1 추진 배경 및 목적
- 1.2 팀원 및 역할분담
- 1.3 개발환경
- 1.4 프로젝트 일정 계획
- 1.5 서비스 용어 설명

2 서비스 기획

- 2.1 메뉴 구조도 (사이트맵)
- 2.2 기능 정의서
- 2.3 정책 정의서

3 서비스 설계

- 3.1 업무 흐름도 전체
- 3.2 시스템 아키텍처
- 3.3 ERD

4 서비스 주요 화면

5 모델 구조 상세

- 5.1 학습 데이터 소개
- 5.2 모델 구현
- 5.3 LLM 구현

6 결과 및 정리

- 6.1 프로젝트 성과
- 6.2 개선점 및 향후 발전 방향
- 6.3 소감

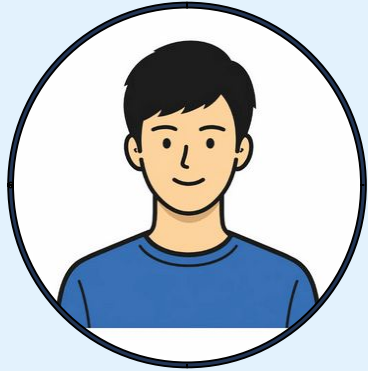


1. 프로젝트 개요

1.1 추진 배경 및 목적

서비스 과제		AI기반 3단계 음악 검증 시스템
추진 배경 및 목적	추진 배경	• 음악 검색의 한계: 수동 검색으로 인한 일관성 부족 및 취향 예측(이질감/오감점) 저하 발생 • 취향 파악의 어려움: 개인 취향 기호는 데이터가 공개에만 의존되어 전산적인 추천 이력 파악 불가능 • 높은 도입 장벽: 중소중견/일반이 개별적으로 AI 솔루션을 구축하기에는 과도한 비용과 인프라 담당 요구
	목적	• 표준화된 AI 검증 시스템: 딥러닝 기반 3단계 필터링(EMS→GMS→PMS)으로 대량 음악 자동 분석 및 점수화 • 실시간 학습 체계: 사용자 피드백 중 검증 데이터를 즉시 수집하여 AI 모델 정확도 지속 향상 • 개인화 큐레이션 자동화: LLM 기반 추천 이유 생성으로 사용자별 맞춤 음악 발견 경험 제공
차별점 및 벤치마킹 분석		• 3단계 필터링 아키텍처: 업계 유일의 구조화된 검증 프로세스로 1000곡 → 150곡 → 개인 라이브러리 자동 정제 • 딥러닝+LLM 하이브리드: 오디오 특징 학습과 메타데이터 분석을 결합하여 85% 이상 정확도 달성 • 지속 학습(Continual Learning): 사용자 승인/거부 데이터를 활용해 시간이 갈수록 취향 예측 정확도 향상 및 의외성 주입으로 필터 버블 방지

1.2 팀원 및 역할분담



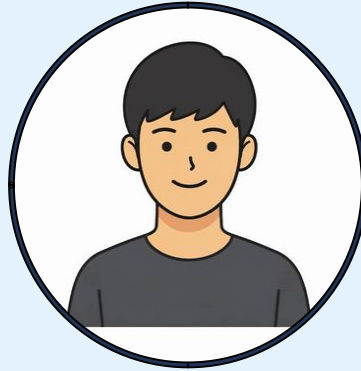
이성훈 (PM)

- Spring boot - FastAPI 간 RESTful API 통신 및 데이터 연동
- Spring 백엔드 Docker, Jenkins 기반의 CI/CD 자동화 구축
- Spring Security + JWT + Redis 활용 무상태 인증 및 토큰 보안 강화
- CatBoostRegressor 기반 사용자 음악 취향 분석 모델 구현
- AI 피드백 루프가 적용된 플레이리스트 CRUD 구현



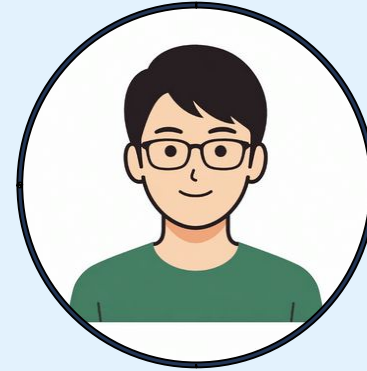
정혜원 (PL)

- 프로젝트 일정 관리 보조
- Ridge Regression 기반 오디오 특성 역추적 ML 모델 구현
- 코사인 유사도 기반 하이브리드 추천 엔진 구현
- 의외성 주입 및 피드백 반영 시스템 구현



조우성 (팀원)

- 프론트 엔드
- 각 프로그래밍 파트 통합
- 스트리밍 연동 (Tidal, Spotify, YouTube Music)
- 플레이리스트 불러오기 (URL 입력, 파일 업로드)



최두석 (팀원)

- 데이터베이스 스키마 설계
- GBRegressor 기반 오디오 특성 역추적 머신러닝 모델 구현
- SVM 기반 취향 추천 머신러닝 모델 구현
- LLM 통한 추천 결과 설명 기능 구현
- RAG+LLM 통한 음악 검색 기능 구현



김기열 (팀원)

- Sentence-BERT 텍스트 임베딩 + KNN 오디오 스코어 기반 멀티모달 앙상블 추천 모델 구현
- 3개 임베딩 모델, 3가지 텍스트 구성, 11개 α 포인트 그리드서치 등 20개 모델 비교 실험 설계
- FAISS 벡터 검색 + Gemini 2.5 Flash 기반 RAG 추천 설명 파이프라인 구현
- MMR 기반 정확도-다양성 균형 조절
- 및 시나리오별 에러 분석

1.3 개발환경

구분	항목	설명	버전
프론트엔드	HTML/CSS	모든 화면의 기본 구조와 디자인	-
	TypeScript	React 코드의 타입 안전성 확보, 버그 방지	-
	React	대시보드, 검사 페이지, 이력 등 모든 화면 UI 구현	-
	Vite	빠른 개발 서버 및 번들링 도구	6.0.5
백엔드	Spring Boot	웹 애플리케이션 프레임워크	3.5.9
	Java	비즈니스 로직 구현	JDK17
	Gradle	Groovy 기반 빌드 도구	8.14.3
	Hibernate	객체 중심 모델링을 통한 데이터 무결성 확보 및 ORM 최적화	6.6.39

1.3 개발환경

구분	항목	설명	버전
백엔드	Python	머신러닝 및 딥러닝 예측 모델 구현	3.10.6
	MariaDB	데이터 저장소(RDBMS)	-
	Redis	세션 관리 및 토큰 캐싱을 담당	-
	REST API	FAST API를 활용한 프론트엔드 - 백엔드 간 데이터 통신	-
AI	머신러닝/ 딥러닝 프레임워크	CatBoost Regressor, Ridge Regression, SVM, GB Regressor, KNN+RandomForest	-
	LLM 개발 프레임워크	FAISS + RAG + 프롬프트 엔지니어링	-
	LLM 모델	Gemini 2.0 / 2.5 Flash	-
	AI 플랫폼	Hugging Face	-
	벡터DB	ChromaDB	

1.4 프로젝트 일정 계획

주요 내용			01/15	01/16	01/19	01/20	01/21	01/22	01/23	01/26	01/27	01/28	01/29	01/30	02/02	02/03	02/04	02/05	02/06	02/09	02/10	02/11	02/12
요구분석	기획	기획안작성																					
		개발환경설정																					
		역할분담																					
	분석	요구사항수집/정리																					
		시스템아키텍처정의																					
설계	DB설계	ERD																					
	화면설계	스토리보드																					
개발	프론트엔드	주요페이지개발																					
		사용자인터페이스구현																					
	백엔드	주요알고리즘 및 모델구현																					
		API개발																					
		실시간데이터연동																					
	데이터정제	데이터수집/전처리																					
	머신러닝/딥러닝	모델학습 및 성능개선																					
	LLM	LLM튜닝및활용																					
테스트		단위테스트																					
		통합테스트																					
		최종보고서작성 및 수정보완																					
		프로젝트수행작성/줄가/최종보고																					

1.5 서비스 용어 설명 (EMS, PMS, GMS)

용어	설명
EMS (External Music Space)	Spotify, YouTube Music 등 외부 API 연동을 통해 수집된 원천 데이터셋 추천의 대상이 되는 전체 Candidate Pool 역할을 수행
PMS (Personal Music Space)	연동된 서비스의 플레이리스트와 최종 확정된 GMS 가 축적되는 공간 사용자의 ' 취향 DNA(Vector) '를 추출하는 핵심 피처로 활용
GMS (Gateway Music Space)	AI 모델이 추출한 DNA 를 바탕으로 EMS 에서 최적의 곡을 선별한 임시 추천 공간 사용자의 실시간 삭제/수정 피드백이 일어나는 Sandbox

2. 서비스 기획

2.1 메뉴 구조도(사이트 맵)



2.2 기능 정의서

번호	Depth1 (시스템)	Depth2 (대기능)	Depth3 (상세기능)	기능설명
1	회원 관리	인증	회원가입	이메일·비밀번호·닉네임 입력, 이메일 중복 검사
2			로그인	JWT Access Token + Refresh Token (Redis 저장)
3			로그아웃	Refresh Token 삭제, 클라이언트 토큰 파기
4			자동 로그인	Access Token 만료 시 Refresh Token으로 자동 갱신
5			역할	USER / ADMIN / MASTER
6	음악 공간	서비스 대시보드	이번 주 히트	Spotify Special Hot Tracks 표시
7			PMS 추천	내 플레이리스트 중 최근 트랙 표시
8			장르별 탐색	장르 클릭 → EMS 검색 연결
9			빠른 재생	트랙 클릭 시 즉시 재생

2.2 기능 정의서

번호	Depth1 (시스템)	Depth2 (대기능)	Depth3 (상세기능)	기능설명
10	음악 공간	PMS (Personal Music Space) _My Lounge	플레이리스트 목록	사용자가 확정한 플레이리스트 전체 표시
11			플레이리스트 생성	제목·설명·공개여부 입력
12			트랙 추가/삭제	드래그 순서 변경, 트랙 삭제
13			My Favorites	하트 버튼으로 좋아요 한 트랙 모음 (가로 스크롤 카드)
14			전체 재생	플레이리스트 전체 큐에 등록 후 재생
15			플랫폼 Import	Spotify·YouTube·Tidal 플레이리스트 자동 가져오기
16		EMS (External Music Space) _The Cargo	외부 플레이리스트 탐색	Spotify 공개 플레이리스트·Hot Tracks 표시
17			트랙 장바구니 담기	분석할 트랙 선택 후 장바구니 추가
18			AI 분석 요청	장바구니 → M1/M2/M3 모델 선택 → 분석 요청

2.2 기능 정의서

번호	Depth1 (시스템)	Depth2 (대기능)	Depth3 (상세기능)	기능설명
19	음악 공간	EMS (External Music Space) _The Cargo	분석 진행 프로그레스바	분석 준비(15%) → 플레이리스트 생성(35%) → AI 요청(55%) → 대기(75%) → 완료(100%)
20			Duration Lazy Loading	Duration 없는 트랙을 iTunes API로 자동 보완
21		GMS (Gateway Music Space) _The Lab	AI 추천 플레이리스트	FastAPI가 생성한 추천 결과 목록
22			트랙 미리 듣기	각 트랙 개별 재생
23			트랙 삭제	마음에 안 드는 트랙 제거
24			플레이리스트 승인	GMS → PMS로 확정 이동 (AI 학습 데이터로 활용)
25			플레이리스트 거절	GMS 플레이리스트 전체 삭제
26	라이브러리	Kuka House (L1 AI 추천 모달)	아티스트/곡명 검색	89,740곡 데이터 기반 유사 추천

2.2 기능 정의서

번호	Depth1 (시스템)	Depth2 (대기능)	Depth3 (상세기능)	기능설명
27	라이브러리	Kuka House (L1 AI 추천 모달)	모델 선택	앙상블 / 오디오 KNN / 텍스트 / 하이브리드
28			AI 추천 이유	Gemini RAG 기반 추천 이유 텍스트 생성
29			결과 재생	추천곡 클릭 → 플레이어 연동
30			Favorites 추가	추천 결과에서 바로 하트 버튼
31			드래그 앤 드롭	모달 위치 자유롭게 이동 가능
32		Deep Dive (L2 AI 추천)	쿼리 분석	한국어 → 영어 변환 + 감성/오디오 필터 자동 추출
33			벡터 검색	ChromaDB 임베딩 기반 유사도 검색 + 오디오 필터링
34			RAG 설명 생성	Gemini 기반 곡별 맞춤 추천 이유 텍스트 생성
35			신규 곡 동기화	MariaDB → 오디오 피쳐 예측 → ChromaDB 자동 저장
36		뮤직 플레이어 (전역)	재생/일시정지	오디오 스트림 제어
37			이전/다음 곡	큐 기반 이동

2.2 기능 정의서

번호	Depth1 (시스템)	Depth2 (대기능)	Depth3 (상세기능)	기능설명
38	라이브러리	뮤직 플레이어 (전역)	반복 재생	없음 / 전체 반복 / 한 곡 반복
39			셔플	큐 랜덤 섞기
40			진행 바	시간 표시, 클릭 탐색
41			볼륨	슬라이더 조절
42			Tidal Smart Match	제목·아티스트로 Tidal 트랙 자동 매칭 재생
43			최근 재생 기록	localStorage에 최대 50곡 자동 저장
44	설정	Settings_AI 모델 설정	AI 모델 선택	M1 / M2 / M3 중 선택
45			모델 즉시 재학습	선택 시 해당 모델로 재학습 + GMS 추천 갱신
46			테마 변경	Default / Jazz / Soul
47		connection	Spotify-브라우저 로그인	수동 — 플레이리스트별 버튼 클릭
48			YouTube Music-OAuth 팝업	자동 — 연결 즉시 전체 Import
49			Tidal-모달 로그인	자동 — 연결 즉시 전체 Import

2.3 정책 정의서

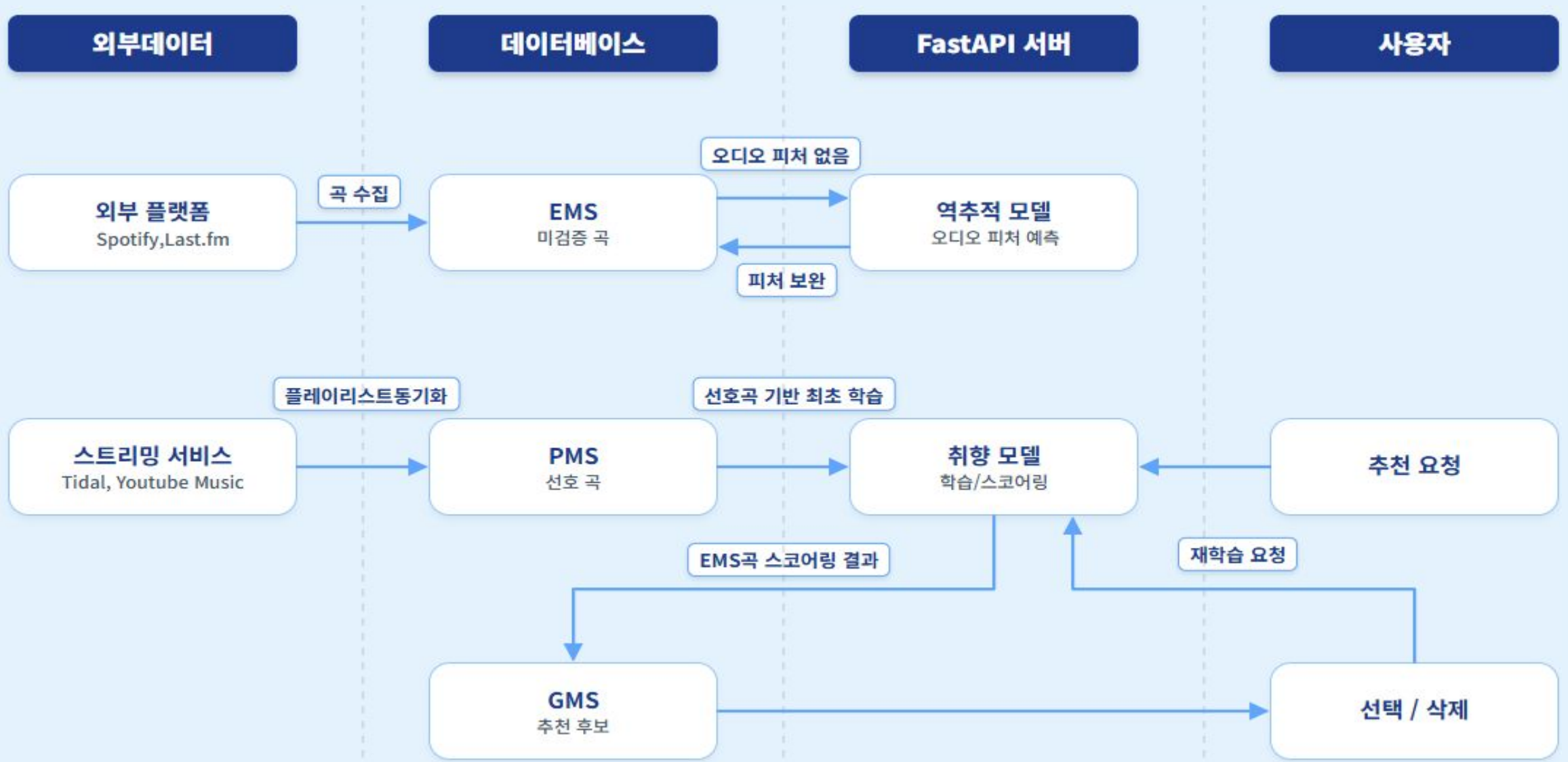
번호	대분류	정책명	세부 항목	소개	정책 정의
1	인증 정책	회원가입 정책	약관동의	개인정보 처리 방침 동의 여부	서비스 이용을 위한필수 동의항목
2		인증 정책	토큰 관리	JWT 인증 방식	Access Token과Refresh Token을 병행하며, 세션 관리를 위해Redis활용
3		역할 정책	권한 구분	사용자 역할 분류	접근 범위에 따라USER,ADMIN,MASTER3단계로 권한 부여
4	Space 정책	PMS 정책	개인 공간	Personal Music Space 정의	사용자가 최종적으로확정하고 소유한 나만의 플레이리스트 공간
5		EMS 정책	외부 공간	External Music Space 정의	외부 플랫폼(Spotify 등)에서 가져온가공 전/미검증플레이리스트 공간
6		GMS 정책	AI 공간	Gateway Music Space 정의	AI 분석을 통해 생성되어 사용자의검토(승인/거절)를 기다리는중간 단계 공간
7	플레이리스트 정책	상태 정책	status_flag	플레이리스트 상태 구분	PTP(준비) /PRP(처리중) /APV(승인) /REJ(거절) 상태로 세분화
8		출처 정책	source_type	플레이리스트 출처 구분	System(AI 생성) /Platform(외부 가져오기) /User(사용자 직접 생성)
9		이동 정책	공간 이동	플레이리스트 흐름	EMS(외부) →GMS(AI 분석/대기) →PMS(최종 승인/개인 저장소) 순으로 이동

2.3 정책 정의서

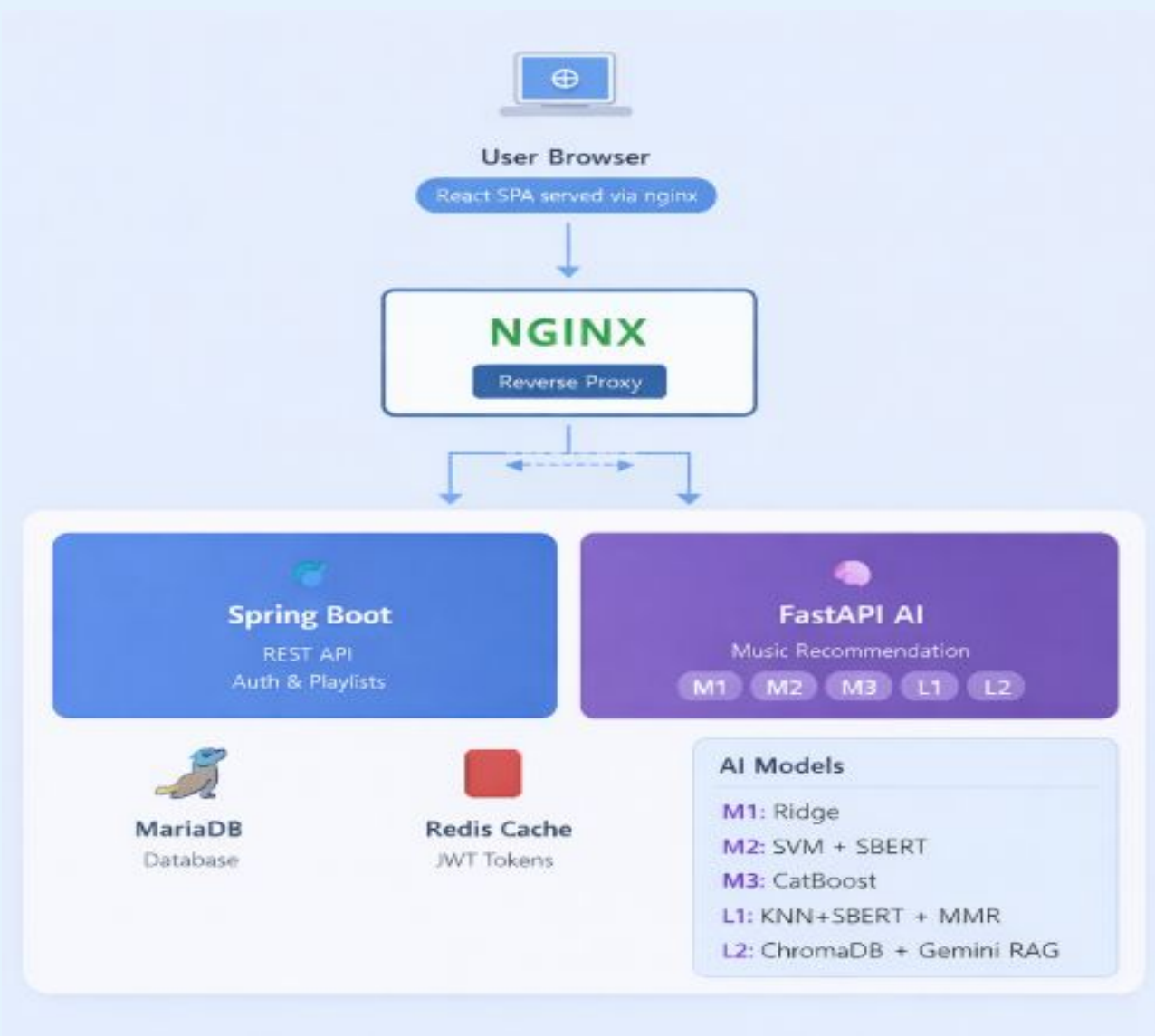
번호	대분류	정책명	세부 항목	소개	정책 정의
10	AI 분석 정책	모델 정책	모델 선택	AI 추천 모델 선택	사용자가 설정(Settings)에서 M1 / M2 / M3 모델 중 하나를 직접 선택 가능
11		학습 정책	학습 시점	AI 학습 데이터 수집	사용자의 취향을 정확히 반영하기 위해 GMS 승인(APV)시에만 데이터를 학습에 활용
12		분석 정책	타임아웃	AI 분석 시간 제한	LLM 또는 API 응답 시간이 60초를 초과할 경우 시스템 실패로 처리하고 사용자에게 알림
13	자연어 검색 정책	검색 정책	쿼리 처리	한국어 검색어 처리	사용자가 입력한 한국어 검색어를 LLM(Gemini)이 영어로 번역한 후, 영어 전용 임베딩 모델을 통해 벡터 검색 수행
14			결과 제한	검색 결과 제공 수	사용자에게 최적의 선택지를 제공하기 위해 기본 5곡을 노출하며, 추가 요청 시 최대 15곡까지 제한
15	데이터 정책	저장 정책	재생 기록	최근 재생 저장	브라우저 localStorage 활용, 최대 50곡까지 보관 (서버 미전송)
16			장바구니	분석 대기 트랙	서버 DB 저장, AI 분석 및 GMS 생성 완료 후 자동 비움
17		삭제 정책	회원 탈퇴	탈퇴 시 데이터 처리	회원 탈퇴 시 해당 계정과 관련된 모든 개인 데이터 영구 삭제
18	외부 플랫폼 연동 정책	Spotify	Import	플레이리스트 가져오기	수동방식 - 사용자가 원하는 플레이리스트별로 버튼 클릭 시 가져옴
19		YouTube/Tidal	Import	플레이리스트 가져오기	자동방식 - 계정 연동 성공 즉시 전체 플레이리스트 자동 Import

3. 서비스 설계

3.1 업무 흐름도 전체

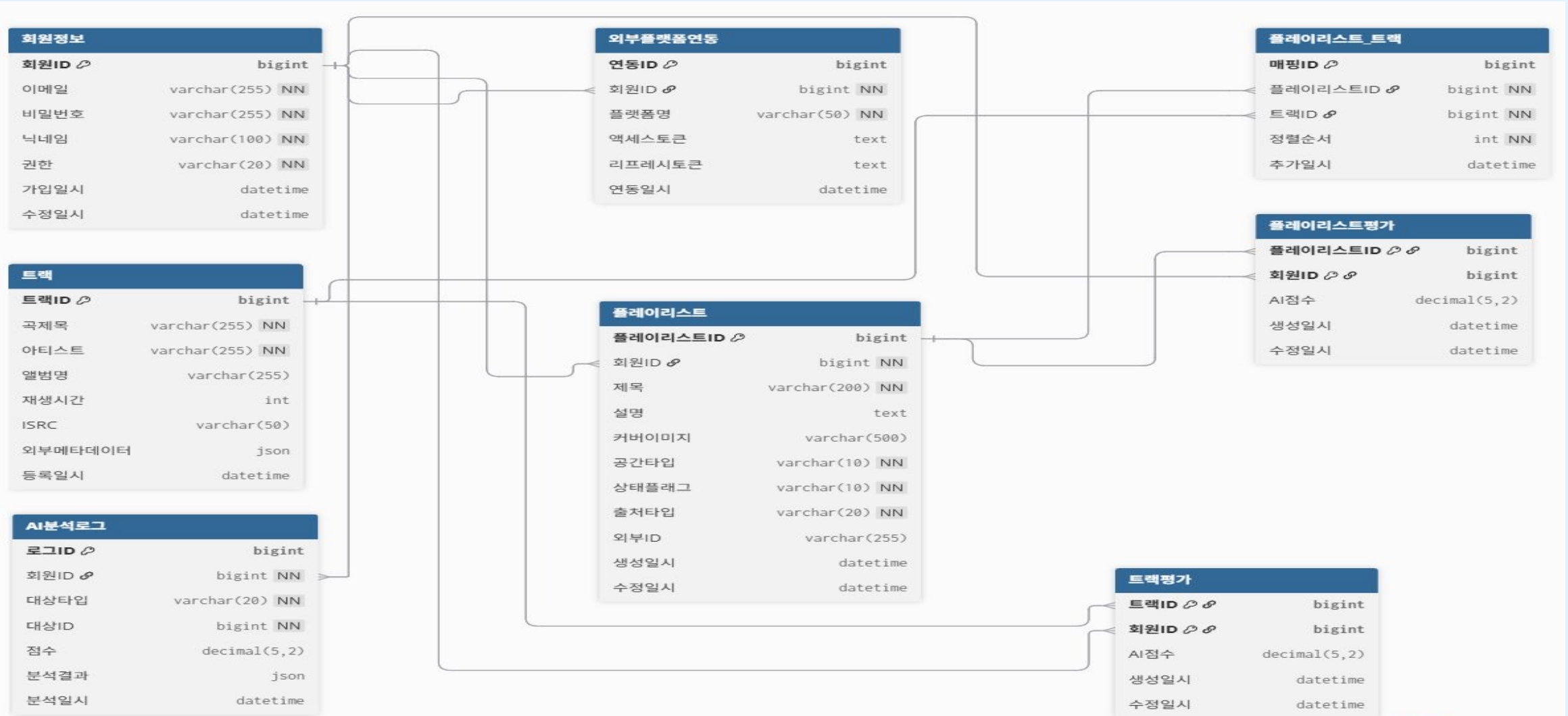


3.2 시스템 아키텍처

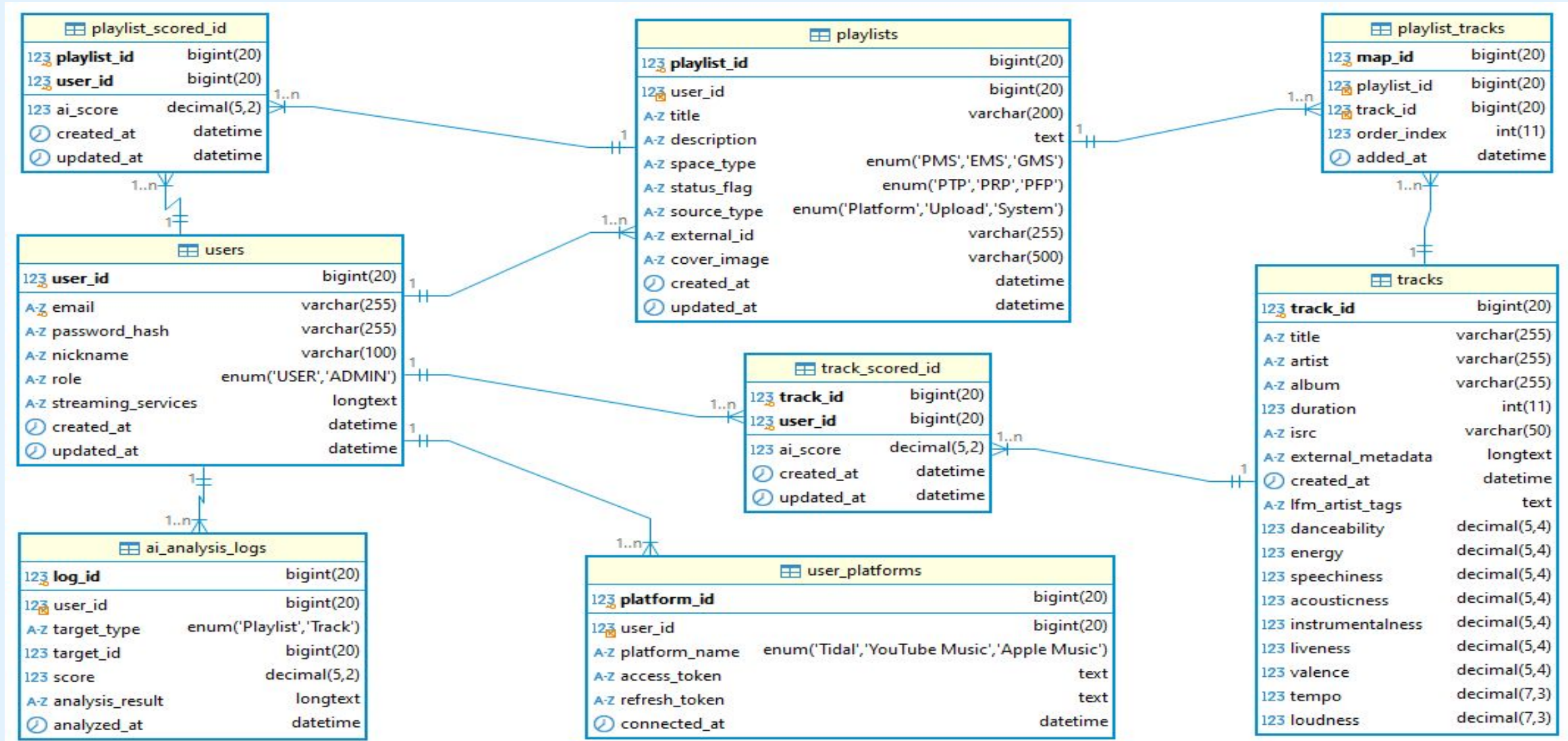


레이어	기술 스택	역할
Frontend	React + NGINX	SPA 서빙 + 리버스 프록시
Backend	Spring Boot	REST API, 인증, 플레이리스트 관리
AI Server	FastAPI	음악 추천
Database	MariaDB	데이터 저장
Cache	Redis	JWT 토큰 캐싱
ID	AI 모델	용도
M1	Ridge	기본 회귀
M2	SVM + SBERT	취향 분류
M3	CatBoost	앙상블 추천
L1	KNN x SBERT + MMR	무드 기반 검색
L2	ChromaDB + Gemini	자연어 검색 + RAG

3.3 ERD (논리적)



3.3 ERD (물리적)



4. 서비스 주요 화면

화면명	회원가입	기능 설명	사용자의 취향(장르)과 이용 중인 스트리밍 서비스를 선택하고, 필수 정보의 유효성을 검증하여 회원가입을 완료하는 페이지
-----	------	-------	--

Full Name

V1

Enter your name

Email Address

V2

Enter your email

Password

V3

Create a password

Confirm Password

V4

Confirm your password

인기 장르 (7)

A1

K-POP

팝

J-POP

인디

인디팝

애니메이션

OST

일렉트로닉 (9)

락/메탈 (7)

스트리밍 서비스

구독 중인 서비스를 선택해주세요. (선택사항)

Tidal

YouTube

Apple

Spotify

이외

☐

I agree to the Terms of Service and Privacy Policy

A4

Create Account

Action		
번호	버튼명	기능
A1	장르	원하는 장르들을 선택
A2	서비스	구독중인 서비스 선택
A3	동의	이용약관에 동의한다고 선택
A4	생성	입력한 정보를 데이터베이스에 저장하고 로그인 페이지로 이동

Validation		
번호	버튼명	기능
V1	이름	유저의 이름을 기입
V2	이메일	유저의 이메일 기입(@이 포함된 이메일 형식)
V3	비밀번호	유저의 비밀번호 기입(6글자 이상의 비밀번호)
V4	비밀번호 확인	비밀번호에 기입한 같은 내용의 비밀번호 기입

화면명	로그인	기능 설명	등록된 이메일과 비밀번호로 본인을 인증하며, 로그인 유지 및 비밀번호 찾기 등의 편의 기능과 서비스 선택을 거쳐 메인 페이지로 진입하는 화면
-----	-----	-------	--

Email Address

V1

Enter your email

Password

V2

Enter your password

Remember me

A2

Forgot password?

A1

함께해요

구독 중인 서비스를 선택해주세요

T

Tidal

YouTube

Apple

Spotify

...

이외

A3

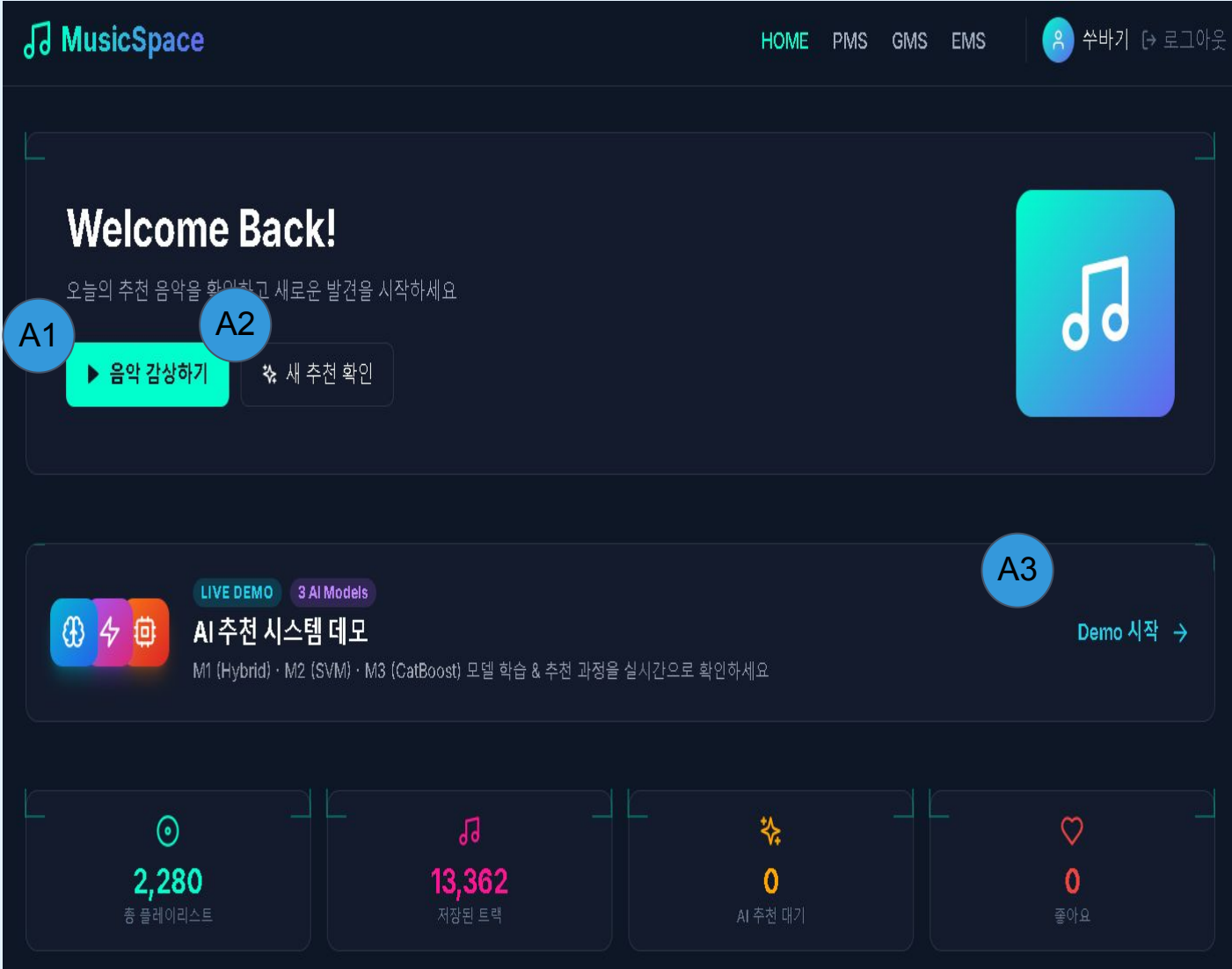
A4

Sign In

Action		
번호	버튼명	기능
A1	Remember	로그인 정보를 기억해 상태를 유지 유무 선택
A2	Forgot	잊어버린 비밀번호를 찾는 페이지로 이동
A3	Service	사용중인 구독서비스를 선택
A4	Sign in	로그인 성공 후 메인 페이지로 이동

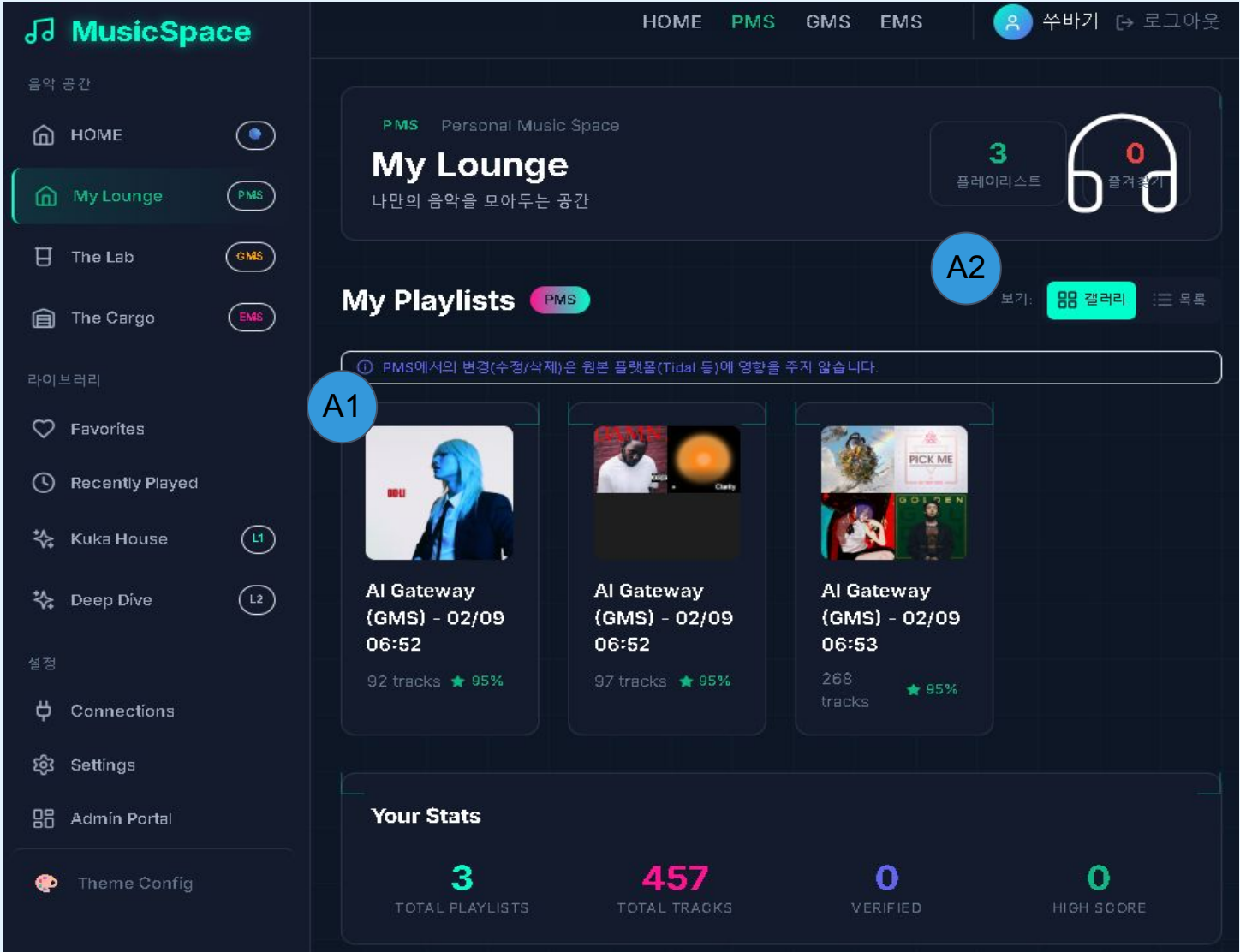
Validation		
번호	버튼명	기능
V1	이메일	데이터베이스에 등록된 이메일 기입
V2	비밀번호	데이터베이스에 등록된 비밀번호 기입

화면명	홈	기능 설명	개인 플레이리스트 확인 및 AI 추천 시스템을 통한 음악 감상이 가능하며, 추천 모델 선택 및 서비스 통계를 한눈에 확인하는 메인 대시보드
-----	---	-------	--



Action		
번호	버튼명	기능
A1	음악 감상하기	본인의 플레이리스트 PMS 화면으로 이동한다.
A2	추천 확인	추천 요청한 플레이리스트 GMS 화면으로 이동한다
A3	모델 선택	모델을 선택하는 페이지로 이동

화면명	PMS	기능 설명	사용자가 저장한 개인 플레이리스트 목록을 갤러리 또는 목록 형태로 자유롭게 확인하고 관리할 수 있는 맞춤형 음악 보관소 화면
-----	-----	-------	---



Action		
번호	버튼명	기능
A1	Playlist	선택한 playlist 목록 확인
A2	view 선택	갤러리 형태로 볼지 목록 형태로 볼지 선택 가능

화면명	GMS	기능 설명	개인 음악 공간(PMS)을 분석하여 AI가 생성한 맞춤형 추천 플레이리스트를 검토하고, 승인 또는 거절을 통해 추천 모델을 학습시키는 화면
-----	-----	-------	---



Action		
번호	버튼명	기능
A1	추천	pms를 분석해서 사용자가 흥미를 느낄 플레이리스트 추천
A2	gms playlist	추천 받은 playlist 직접 확인

화면명	GMS playlist	기능 설명	AI가 추천한 특정 플레이리스트에 포함된 전체 곡 리스트를 확인하고, 개별 트랙 또는 전체 트랙을 선택하여 즉시 감상할 수 있는 상세 화면
-----	--------------	-------	---



AI Gateway (GMS) - 02/09 06:52

AI 추천 플레이리스트 (M1 모델)

92 Tracks • PMS Space

A1

Drowning

WOODZ

4:05

2

O+O

한로로

4:53

3

그대 작은 나의 세상이 되어

카더가든

0:00

4

사랑하게 될 거야

한로로

2:53

5

Love Love Love (feat. 용진)

에픽하이

5:07

6

가까운 듯 먼 그대여

카더가든

0:00

🗑️

7

nosho

식구

0:00

8

Feel This Moment (feat. Christina Aguilera)

Pitbull, Christina Agu...

3:50

9

NOT QUITE ANYMORE

아인리

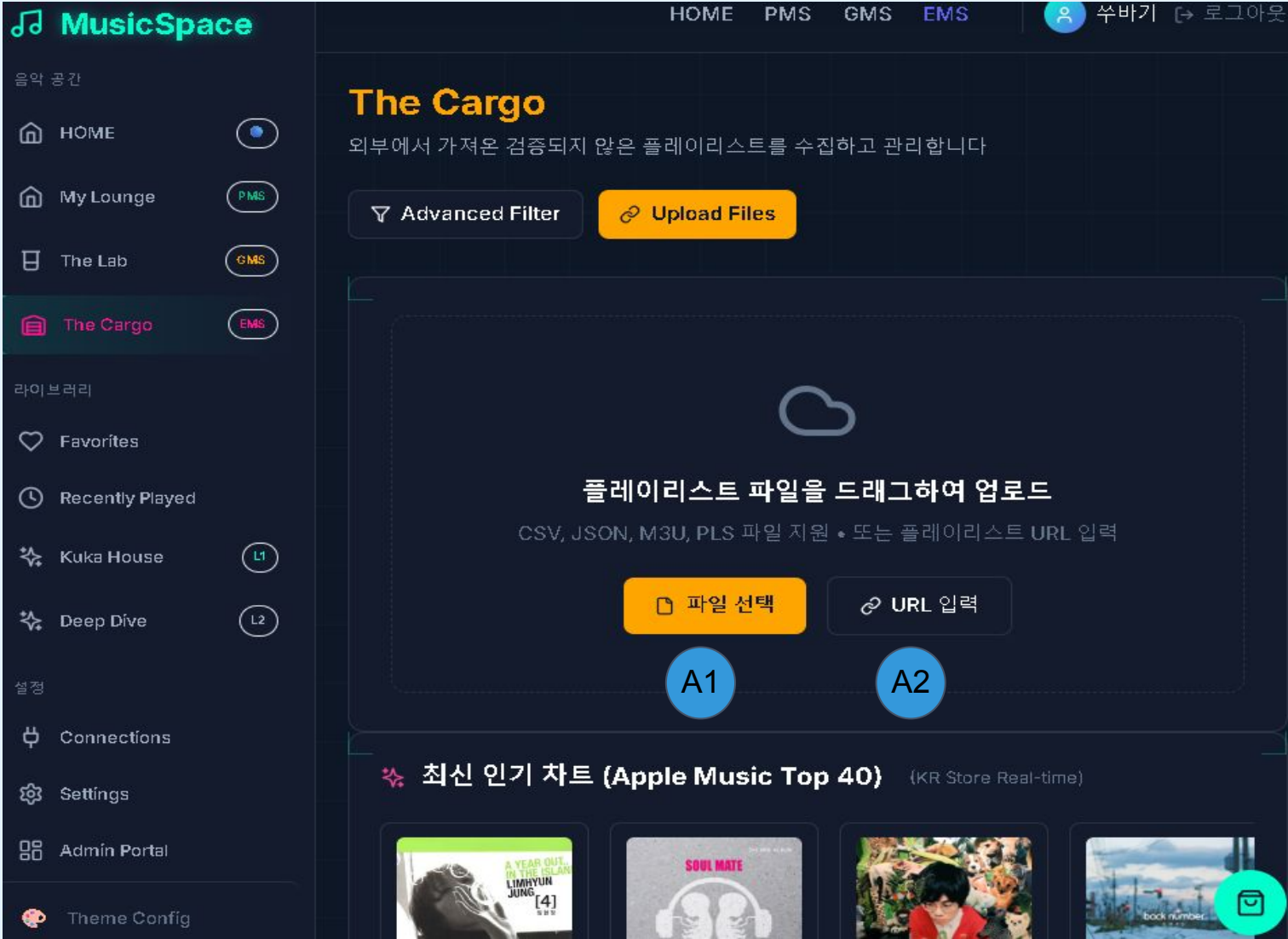
5:46

A2

▶ Play All

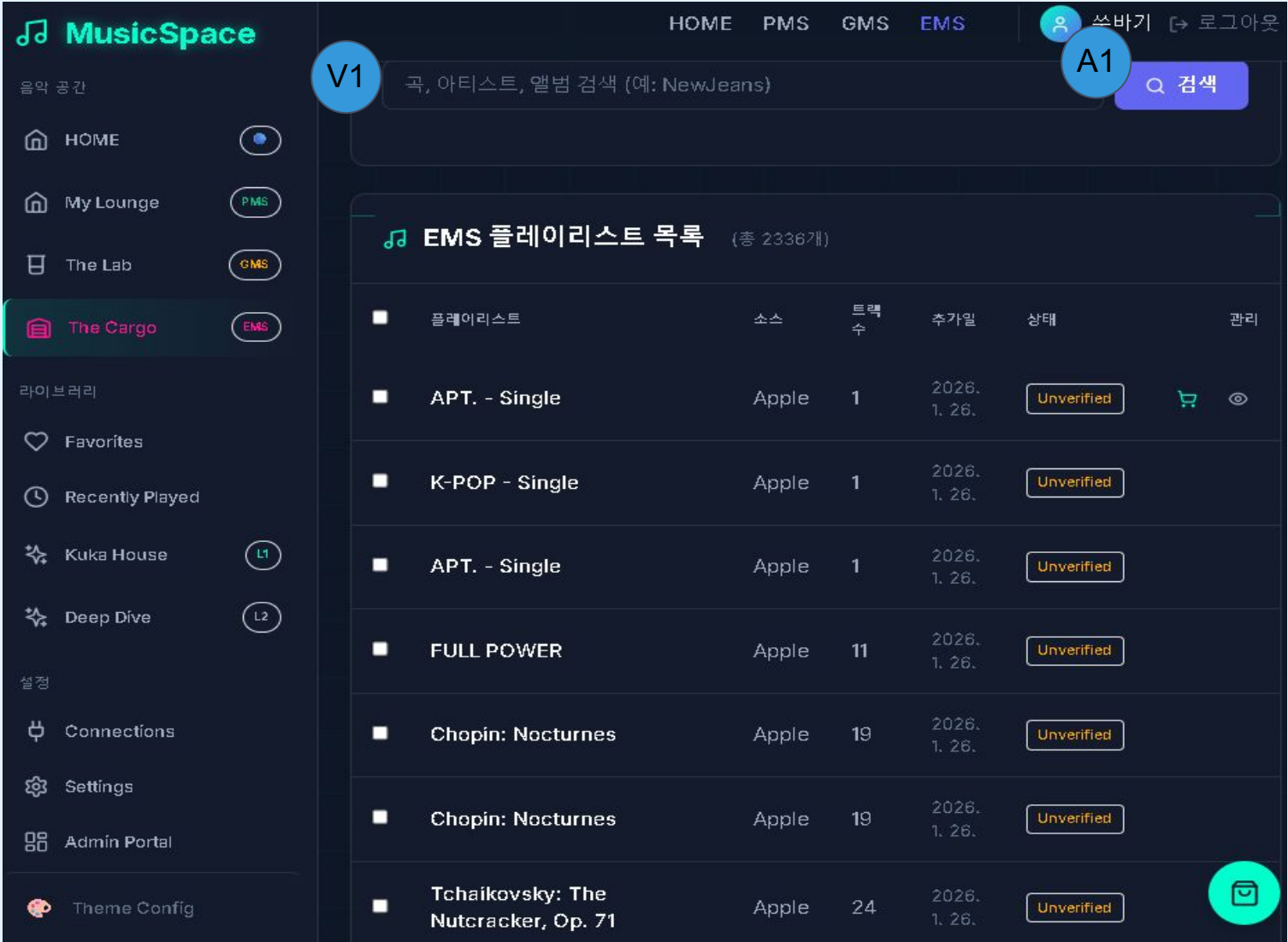
Action		
번호	버튼명	기능
A1	트랙	원하는 트랙을 선택해서 감상
A2	play all	플레이리스트의 모든 트랙 감상

화면명	EMS	기능 설명	외부 플레이리스트 파일(CSV, JSON 등)이나 URL을 업로드하여 검증되지 않은 음악 데이터를 시스템에 수집하고 관리하는 화면
-----	-----	-------	--



Action		
번호	버튼명	기능
A1	파일 선택	csv, json, M3U, PLS 파일 지원
A2	URL 입력	플레이리스트 URL 입력

화면명	EMS playlist	기능 설명	수집된 외부 플레이리스트들의 전체 목록을 확인하고, 검색 기능을 통해 특정 아티스트나 앨범이 포함된 트랙을 신속하게 조회하는 화면
-----	--------------	-------	--



Action		
번호	버튼명	기능
A1	검색	입력된 정보로 검색하여 원하는 트랙을 조회

Validation		
번호	버튼명	기능
V1	검색	원하는 아티스트 또는 앨범을 입력

화면명	좋아요 모음	기능 설명	사용자가 '좋아요'를 표시한 곡들을 한데 모아 관리하며, 전체 곡을 한번에 재생하거나 라이브러리 통계를 확인할 수 있는 개인 취향 저장소 화면
-----	--------	-------	---

음악 공간

HOME

My Lounge

The Lab

The Cargo

라이브러리

Favorites

Recently Played

Kuka House

Deep Dive

설정

Connections

Settings

Admin Portal

Theme Config

HOME

PMS

GMS

EMS

쭈바기
로그아웃

COLLECTION

My Favorites

1 tracks liked

A1

Play All

Liked Songs

PMS

보기:

그리드

목록

Don Pacifico

The Vampires

Favorites Stats

1

TOTAL LIKED

1

ARTISTS

1

ALBUMS

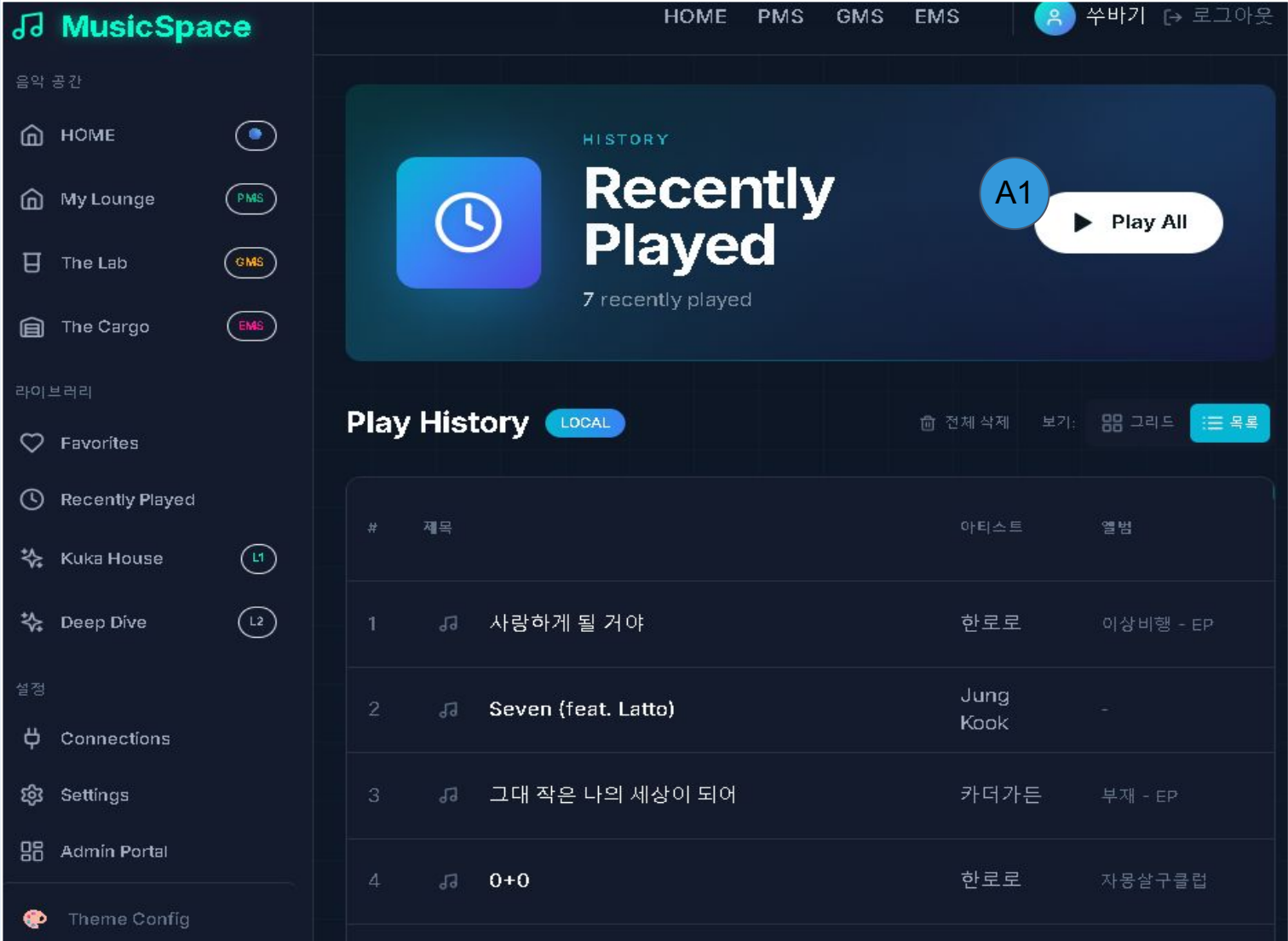
4m

TOTAL TIME

"Seven (feat. Latto)" 재생 소스를 찾을 수 없습니다

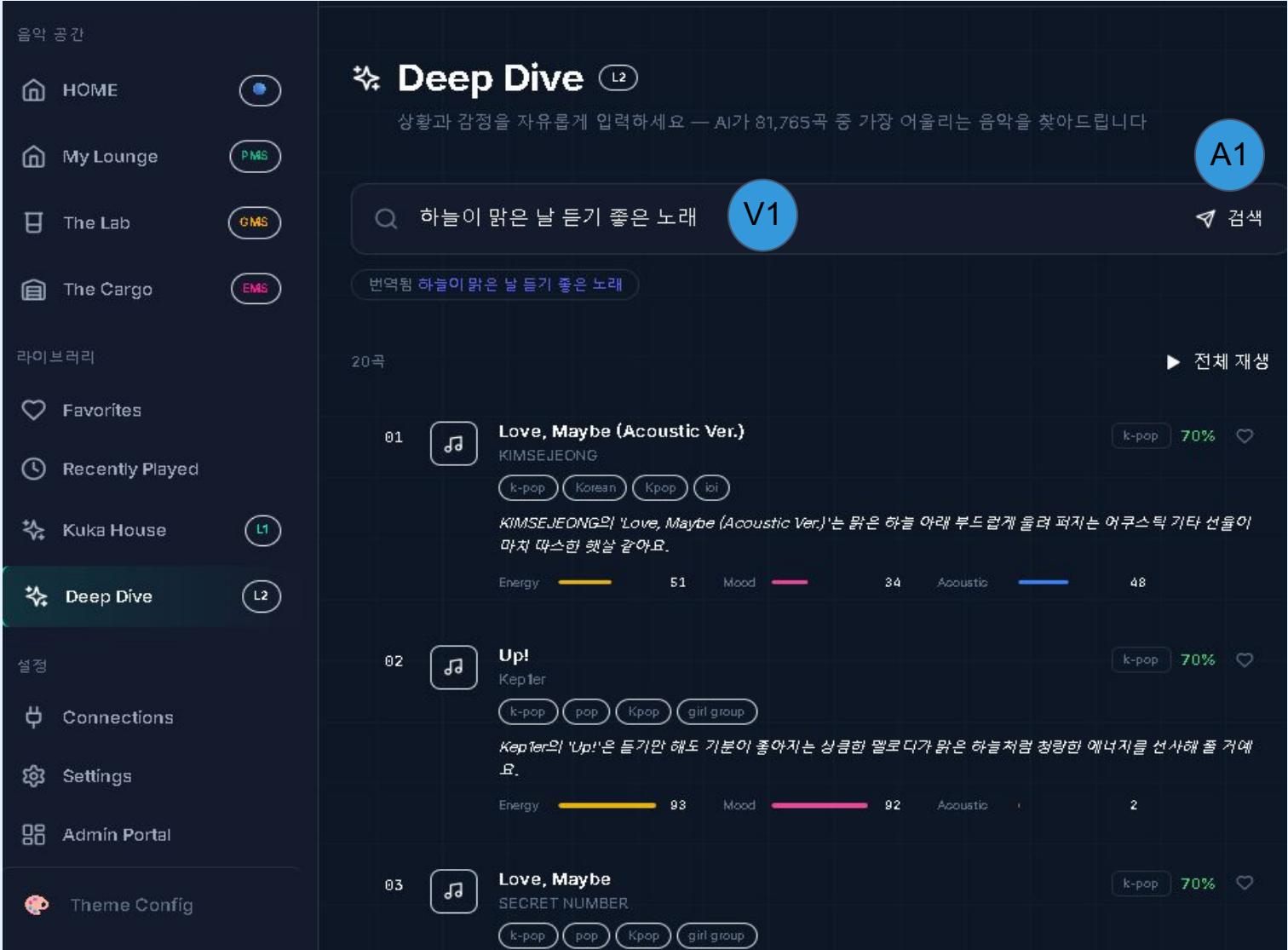
Action		
번호	버튼명	기능
A1	play all	좋아요 누른 곡을 모두 재생

화면명	최근 재생 목록	기능 설명	사용자가 최근에 감상한 음악들의 히스토리를 확인하고, 해당 곡들을 다시 재생하거나 전체 목록을 한 번에 감상할 수 있는 기록 관리 화면
-----	----------	-------	---



Action		
번호	버튼명	기능
A1	play all	최근 재생한 모두 재생

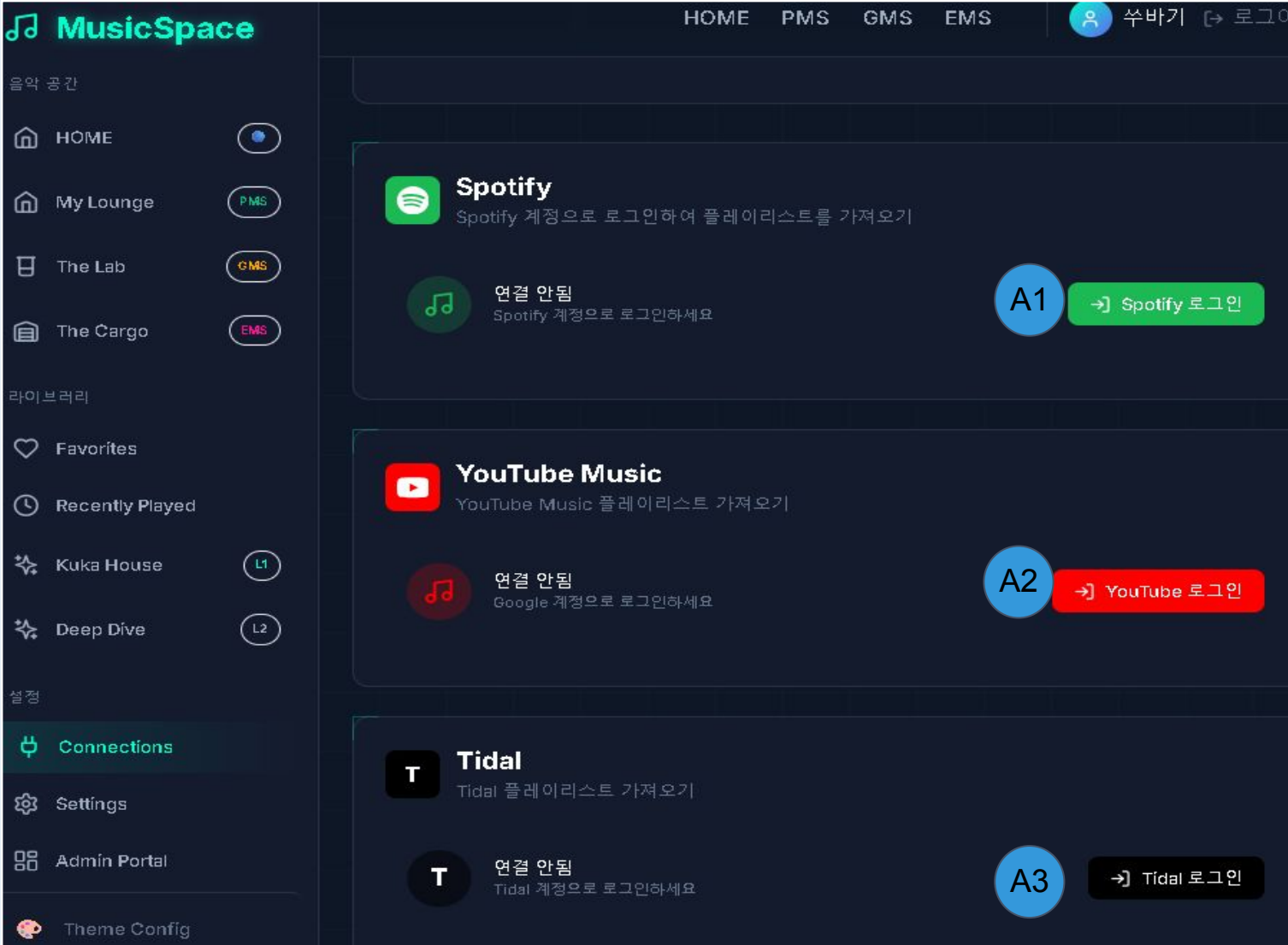
화면명	deep dive	기능 설명	사용자가 입력한 특정 상황이나 감정을 AI가 분석하여, 보유한 방대한 곡 중 가장 어울리는 음악을 맞춤형 해설과 함께 추천해 주는 화면
-----	-----------	-------	---



Action		
번호	버튼명	기능
A1	검색	상황과 감정을 입력하여 검색

Validation		
번호	버튼명	기능
V1	검색	상황과 검색을 입력

화면명	연동	기능 설명	Spotify, YouTube Music, Tidal 등 외부 스트리밍 서비스 계정을 연결하여 개인 플레이리스트 데이터를 가져오고 통합 관리할 수 있는 계정 동기화 화면
-----	----	-------	--



Action		
번호	버튼명	기능
A1	Spotify 연동	Spotify 연동하는 페이지로 이동
A2	Youtube 연동	YouTube로 연동하는 페이지로 이동
A3	Tidal 연동	Tidal로 연동하는 페이지로 이동

5.1 학습 데이터 소개

사용 데이터 셋 URL : <https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>

구분	컬럼명	설명
기본 정보	track_id, artists, album_name, track_name	곡의 고유 ID, 아티스트, 앨범명, 곡 제목
인기도 및 환경	popularity, duration_ms, explicit	곡의 인기도(0~100), 곡 길이(ms), 성인용 가사 여부
음악적 색채	danceability, energy, loudness	춤추기 적합한 정도, 에너지/활동성, 전체적인 음량(dB)
음악적 분위기	valence, acousticness, instrumentalness	곡의 밝기(긍정적 감정), 어쿠스틱 수치, 보컬 없는 연주곡 비율
녹음/공연 정보	liveness, speechiness, tempo, key, mode	라이브 공연 느낌, 말소리 비중, BPM(속도), 조(Key), 선법(장/단조)
기타	track_genre	해당 곡이 속한 장르 카테고리

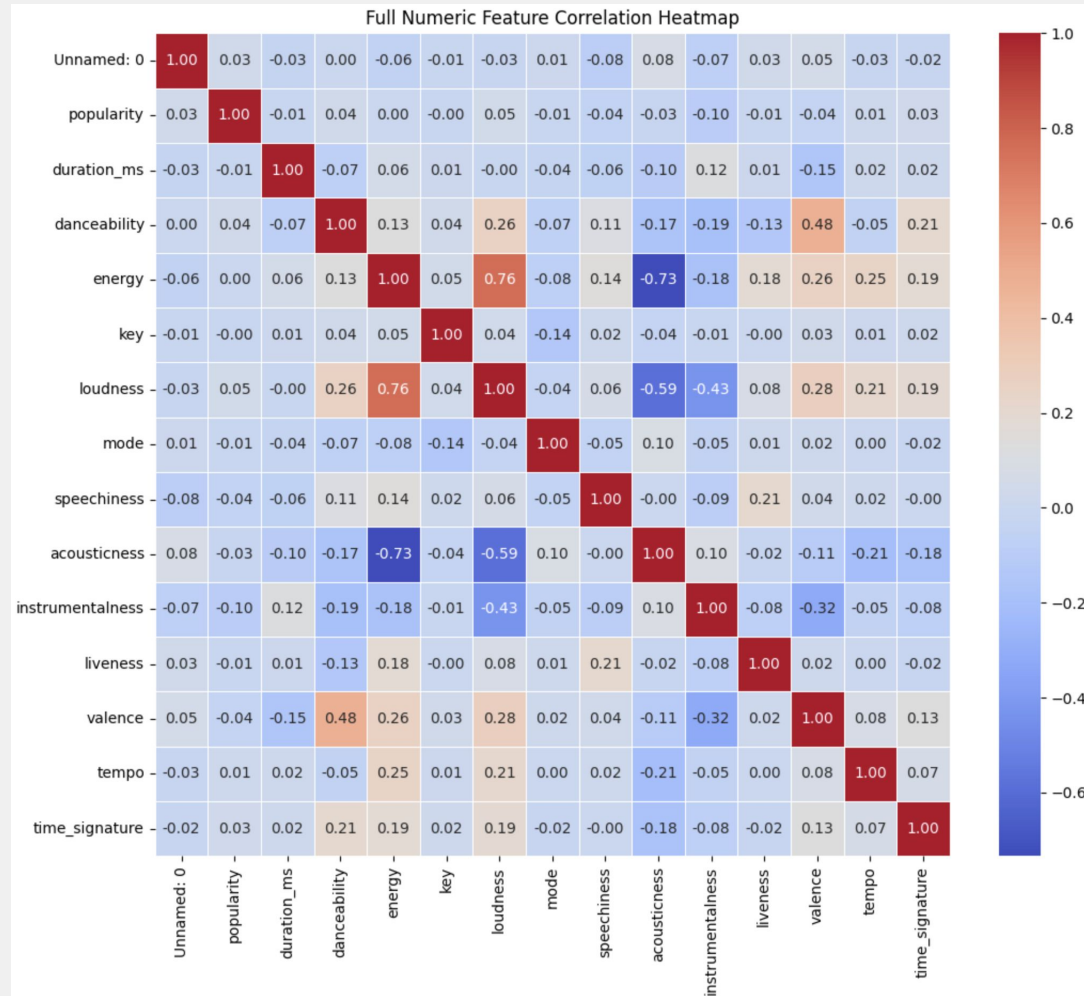
5. 모델 구조 상세

(5.2 모델 학습)

5.2 모델 학습

사용 모델	주요 역할 및 정의	핵심 특징 및 성과
CatBoost Regressor	사용자의 음악적 취향 좌표(DNA)를 생성하고 점수화하는 회귀 모델	• Multi-output Regression: 11가지 오디오 수치를 동시에 예측하여 추천 일관성 확보 • 범주형 데이터 처리: 별도 인코딩 없이 아티스트/장르 정보를 손실 없이 학습
Ridge Regression	메타데이터(장르, 아티스트)를 기반으로 오디오 특성을 예측하는 최종 선정 모델	• 과적합 방지: L2 정규화를 통해 새로운 데이터에 대한 일반화 성능 극대화 • 높은 안정성: 6개 비교 모델 중 가장 안정적인 R^2 달성 (평균 0.3905)
GBR + SVM 하이브리드	메타데이터 기반 오디오 특성 추론(GBR) 및 하이브리드 벡터 기반 사용자 취향 정밀 분류(SVM)	• 고효율 추론: GBR로 API 없이 9개 피쳐 역추론 (DL 대비 전처리 360배 속도, 메모리 1/12 절감) • 정밀 변별: 393D 하이브리드 벡터 및 'Hard Negative' 전략으로 가수 매칭을 넘어선 고차원 취향 검증
KNN + Random Forest	유저 취향과 유사한 곡들을 군집화하고(KNN), 랜덤 포레스트를 통해 중요 피쳐별로 곡을 최종 분류/정제함	• 정밀 군집화: KNN을 통해 유저의 기존 선호 곡과 가장 유사한 K개의 후보곡을 도출하여 추천 정확도 향상 • 데이터 변별력 강화: Random Forest의 의사결정 나무 구조를 활용해 추천 리스트 내 불필요한 노이즈를 효과적으로 필터링

CatBoostRegressor 모델



✓ 상관관계 히트맵

- 설명 : 데이터셋에 포함된 모든 수치형 데이터 간의 상관계수를 시각화하여 변수간의 선형적 관계를 분석함.
- 해석: 음악 분위기를 결정하는 **energy, loudness, acousticness** 등의 상호 간에 높은 상관성을 보이며 유의미한 군집을 형성하고 있음.

결과: 히트맵 순서를 통해 음악적 취향 좌표를 생성하는 데 기여도가 낮은 인기도와 곡 길이 등을 제외하고, 실제 곡의 질감을 표현하는 **11가지 핵심 오디오 특성**을 **최정 학습 feature**로 사용


```
# 1. 데이터 로드
df = pd.read_csv('dataset/dataset.csv')

# 2. 결측치(비어있는 값)가 있는 행 제거
df = df.dropna()

# 3. 중복된 트랙 제거 (track_id 기준)
df = df.drop_duplicates(subset=['track_id'])

# 스케일링할 오디오 속성 11가지
audio_features = [
    'danceability', 'energy', 'key', 'loudness', 'mode',
    'speechiness', 'acousticness', 'instrumentalness',
    'liveness', 'valence', 'tempo'
]

# 4. StandardScaler 적용
scaler = StandardScaler()
df[audio_features] = scaler.fit_transform(df[audio_features])
```

✓ 데이터 정제 및 스케일링

- 설명: 히트맵 분석을 통해 선발된 11가지 핵심 오디오 특성에 대해 StandardScaler를 적용하여 표준화.
- 해석: tempo와 danceability처럼 값의 범위가 다른 변수들을 동일한 스케일로 맞추어, 유클리드 거리 계산 시 특정 변수가 과도한 영향을 미치지 않도록 사용.

결과: 전체 수치 데이터를 전수 조사한 결과, 음악적 맥락과 무관한 메타데이터를 배제하고 곡의 개성을 잘 나타내는 11가지 오디오 속성을 최종 학습 특성으로 선정.

```
from sklearn.model_selection import train_test_split
from catboost import CatBoostRegressor
import numpy as np

X = remaining_df[features]
y = remaining_df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    shuffle=True
)

model = CatBoostRegressor(
    iterations=1000,
    learning_rate=0.1,
    depth=6,
    loss_function='MultiRMSE',
    eval_metric='MultiRMSE',
    random_seed=42,
    verbose=10
)

model.fit(
    X_train, y_train,
    cat_features=cat_features,
    eval_set=(X_test, y_test),
    early_stopping_rounds=50
)
```

✓ 다중 출력 회귀 (Multi-output Regression)

- 설명: 댄스빌리티, 에너지 등 11가지 오디오 수치를 동시에 예측하기 위해 MultiRMSE 손실 함수를 적용.
- 해석: 각 수치를 개별적으로 예측하는 대신, 11개 수치 사이의 관계를 보존하며 하나의 벡터(취향 좌표)로 학습하여 추천의 일관성을 확보.

✓ 효율적인 범주형 데이터 처리

- 설명: cat_features 파라미터를 통해 아티스트 및 장르 정보를 모델에 직접 전달.
- 해석: 별도의 인코딩 작업 없이 텍스트 데이터를 활용함으로써 아티스트 특유의 스타일 정보를 손실 없이 모델에 반영함.

결과: 학습 데이터에만 치중되는 현상을 막아 처음 보는 곡에 대해서도 유연하고 정확하게 취향점수를 계산할 수 있도록 일반화 성능을 극대화.

```
from sklearn.ensemble import IsolationForest

# 1. Isolation Forest를 이용한 이상치 탐지
iso_forest = IsolationForest(contamination=0.1, random_state=42)
outlier_labels = iso_forest.fit_predict(pms_audio_data)

# 정상 데이터(1)와 이상치(-1) 분리
pure_pms = pms_all[outlier_labels == 1]
outliers = pms_all[outlier_labels == -1]

# 2. 정제된 취향 중심점(User Taste Vector) 계산
user_taste_vector = pure_pms[audio_features].mean().values.reshape(1, -1)

# 3. 중심점 기준 최적 거리 곡 100개 추출
predicted_audio = model.predict(X_test)
distances = euclidean_distances(user_taste_vector, predicted_audio).flatten()

# 거리 기준 상위 100곡 선정
top_100_indices = np.argsort(distances)[:100]
gms_recommendations = X_test.iloc[top_100_indices].copy()
```

✓ Isolation Forest 기반 이상치 제거

- 설명: 사용자의 플레이리스트(PMS) 내 오디오 속성을 분석하여 평균적인 흐름에서 벗어난 상위 10%의 곡을 이상치로 규명.
- 해석: 단순 데이터 전체가 아닌 '가장 일관성 있는 취향'의 데이터만 선별함으로써, 노이즈가 섞이지 않은 순수 취향 중심점을 확보.

✓ 취향 중심점 기반 추천(GMS)

- 설명: 정제된 데이터의 평균값인 `user_taste_vector`와 테스트 곡들 사이의 유클리드 거리를 계산.
- 해석: 사용자의 메인 장르 감성과 가장 유사한 오디오 속성을 가진 곡들을 수학적으로 우선 배치하여 추천의 정밀도를 극대화.

```
# 1. 재학습 데이터 구성 (기존 데이터 + 추천 제외곡 10개)
X_feedback = pd.concat([X_train, deleted_gms[features]])
y_feedback = pd.concat([y_train, deleted_gms[target]])

# 2. 가중치 설정 (부정 피드백 데이터에 50배 가중치 부여)
# 기존 데이터 가중치 1 vs 제외 곡 가중치 50
weights = np.concatenate([np.ones(len(X_train)), np.full(len(deleted_gms), 50)])

# 3. 가중치(sample_weight)를 적용한 모델 v2 학습
model_v2.fit(
    X_feedback, y_feedback,
    cat_features=cat_features,
    sample_weight=weights, # 핵심: 50배 페널티 적용
    early_stopping_rounds=50
)
```

✓ 부정적 피드백의 수치화

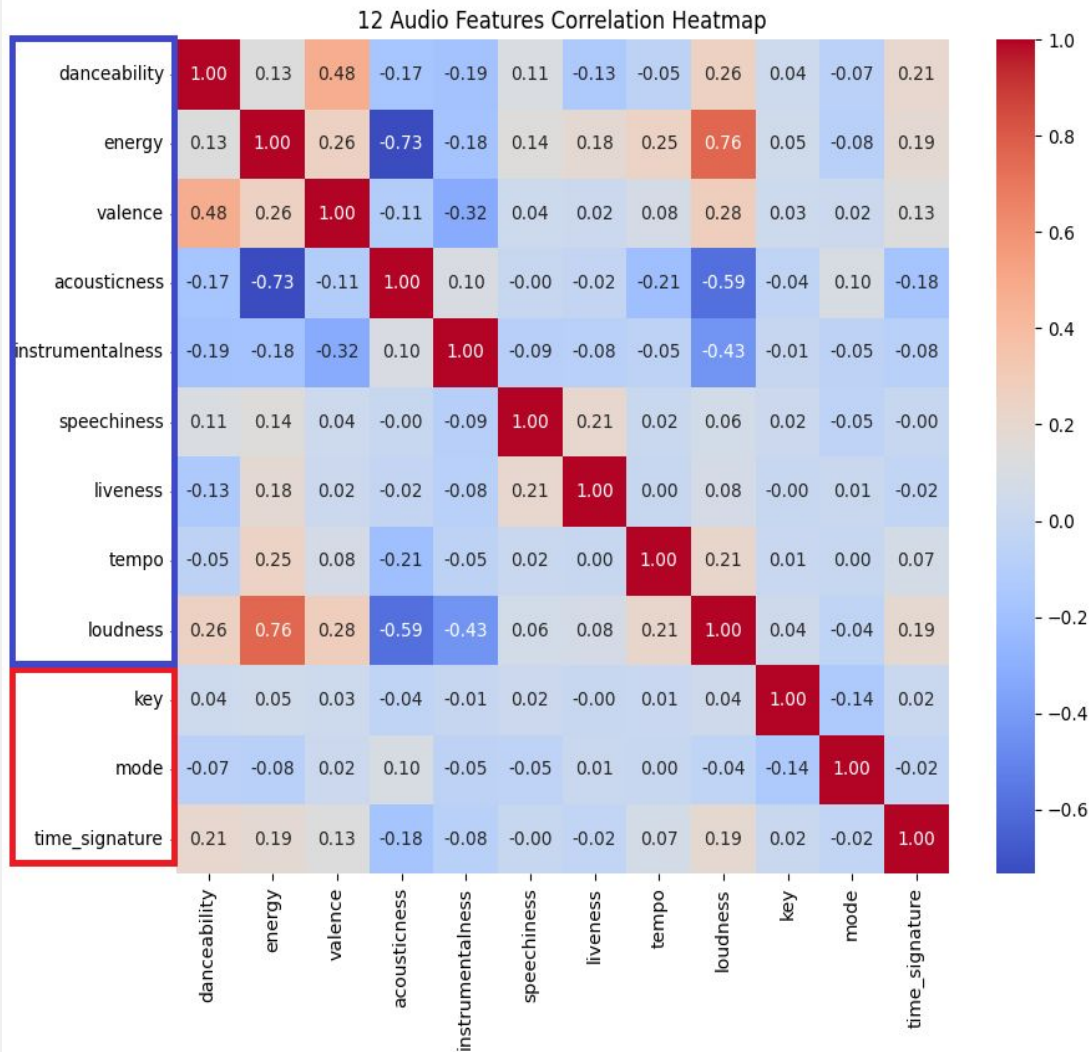
- 설명: 추천 리스트에서 거리가 멀어 제외된 10곡을 '사용자가 선호하지 않는 스타일'로 정의하고, 학습 시 50배의 가중치(Sample Weight)를 부여.
- 해석: 모델이 해당 곡들의 특징을 학습할 때 발생하는 오차를 50배 더 크게 인식하게 함으로써, 유사한 스타일의 곡을 추천 리스트에서 강력하게 배제하도록 유도.

✓ 개인화 모델의 진화 (Model v2)

- 설명: 기존의 일반적인 장르 학습 데이터에 사용자의 실시간 거절 데이터를 결합하여 재학습을 수행.
- 해석: 정적인 모델이 아닌, 사용자의 피드백에 따라 취향의 경계선을 실시간으로 수정하는 동적 추천 시스템을 구축함.

결과: 단순히 '좋은 곡'을 찾는 것을 넘어 '싫어하는 스타일'을 정확히 회피함으로써 추천의 정밀도와 사용자 만족도를 동시에 확보.

Ridge Regression 모델



✓ 오디오 특성 상관관계 분석

- 설명: 12개 오디오 특성 간 상관관계를 히트맵으로 시각화.
- 해석: energy-loudness($r=0.76$), energy-acousticness($r=-0.73$) 등 강한 상관관계를 발견하여, 도메인 지식 기반 12개 → 9개 특성

✓ 데이터 전처리

- 설명: 결측치: track_name, artists, album_name 각 1개 → 'unknown' 대체, 수치형 컬럼 → 중앙값 대체
이상치: instrumentalness(22.15%), speechiness(11.59%), liveness(7.58%) 발견.
- 해석: 음악 데이터 특성상 이상치는 실제 데이터로 판단하여 유지 (예: instrumentalness=0인 보컬곡)

결과: 최종 피처 9개로 선정


```
=====
[Step 3c] 데이터 분할 (70% 학습 / 30% 시뮬레이션)
=====
```

```
[1/2] 데이터 분할 완료
```

- 전체: 114,000곡
- df_train (70%): 79,800곡 → 역추적 모델 학습용
- df_test (30%): 34,200곡 → PMS/EMS 서비스 시뮬레이션용

```
[2/2] 분할 검증 (장르 분포)
```

장르	전체 비율	Train 비율	Test 비율
acoustic	0.88	0.86	0.91
afrobeat	0.88	0.88	0.87
alt-rock	0.88	0.86	0.92
alternative	0.88	0.89	0.86
ambient	0.88	0.91	0.80

```
→ 장르 분포가 Train/Test 간 유사하게 유지됨을 확인
```

```
✅ 데이터 분할 완료!
```

```
df_train: 모델 학습에만 사용 (오디오 특성 보유)
df_test: 오디오 특성 제거 후 PMS/EMS로 활용
```

✅ Train/Test 분할

- 설명: 전체 114,000곡을 70:30 비율로 분할
 - df_train: 79,800곡 (모델 학습용)
 - df_test: 34,200곡 (서비스 시뮬레이션용)
- 해석: 학습 데이터로 모델을 훈련하고, 테스트 데이터로 실제 서비스 환경(PMS/EMS/GMS)을 시뮬레이션

✅ 분할 검증

- 설명: Train/Test 간 장르 분포가 동일하게 유지되는지 확인
- 해석: 데이터 편향 없이 공정한 모델 학습 및 평가 보장
 - 특정 장르가 한쪽에 몰리면 모델 성능이 왜곡될 수 있음

결과: df_train 79,800곡 / df_test 34,200곡
(장르 분포 유지 확인)

5.2 모델 학습-초기 모델 선택(LightGBM)

작성자: 정혜원

LightGBM

Gradient Boosting 기반 트리 모델

선택 이유

- 대용량 데이터 빠른 학습
- 범주형 변수 자동 처리
- 114,000곡 처리 적합 예상

기대 효과

- 비선형 관계 학습
- 피처 중요도 자동 파악
- Histogram 기반 메모리 효율

20,000곡 학습 완료

⚠ 그러나 문제 발생...

✓ LightGBM을 먼저 선택한 이유

- 설명: Gradient Boosting 기반의 트리 모델로, 대용량 데이터에서 빠른 학습 속도, 높은 예측력으로 유명
- 해석: Kaggle 등 머신러닝 대회에서 검증된 성능, 114,000곡 대용량 데이터에 적합할 것으로 기대

✓ 기대 효과

- 설명: 복잡한 비선형 관계 학습, 피처 중요도 자동 파악, Histogram 기반으로 메모리 효율적
- 해석: 장르/아티스트와 오디오 특성 간의 복잡한 패턴을 트리 구조로 학습할 수 있을 것으로 예상

결과: 20,000곡으로 LightGBM 모델 학습 완료 → 그러나 문제 발생...

◆ STEP 12: 예측 정확도 평가

🎧 오디오 속성 예측 정확도:

```
=====
danceability      :  $R^2$  = -0.177
energy            :  $R^2$  = -0.190
valence           :  $R^2$  = -0.209
acousticness      :  $R^2$  = -0.124
instrumentalness   :  $R^2$  = -0.359
speechiness       :  $R^2$  = -0.454
liveness          :  $R^2$  = -0.335
tempo             :  $R^2$  = -0.064
loudness          :  $R^2$  = -0.194
=====
```

💡 R^2 해석:

- 0.8 이상: 매우 정확
- 0.6-0.8: 정확
- 0.4-0.6: 보통
- 0.4 미만: 낮음

✓ 문제 발견 시점

- 설명: Step 12(예측 정확도 평가)에서 학습에 사용하지 않은 500개 곡으로 최종 평가 시 R^2 음수 발견
- 해석: Step 6(학습 단계) 내부 검증에서는 괜찮아 보였으나, 실제 새로운 데이터에서는 완전히 실패 (과적합 착시)

✓ R^2 음수 결과

- 설명: 9개 오디오 특성 전부 R^2 음수
 - danceability: -0.177 / energy: -0.190 / valence: -0.209
 - acousticness: -0.124 / speechiness: -0.454 / 평균: -0.234
- 해석: R^2 음수 = 단순 평균값 예측보다 못함
모델이 데이터를 "이해"하지 못하고 "암기"만 함

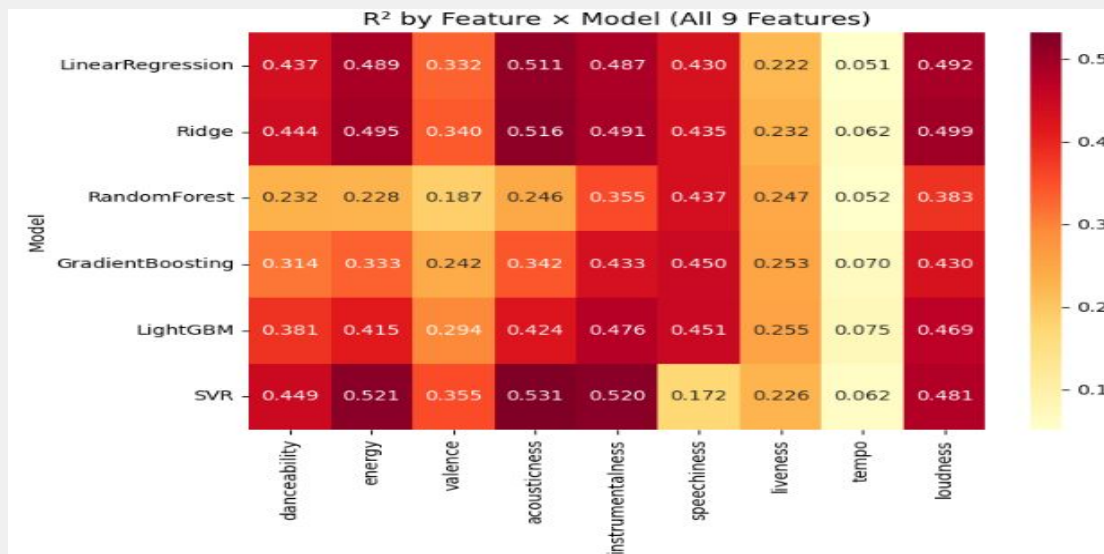
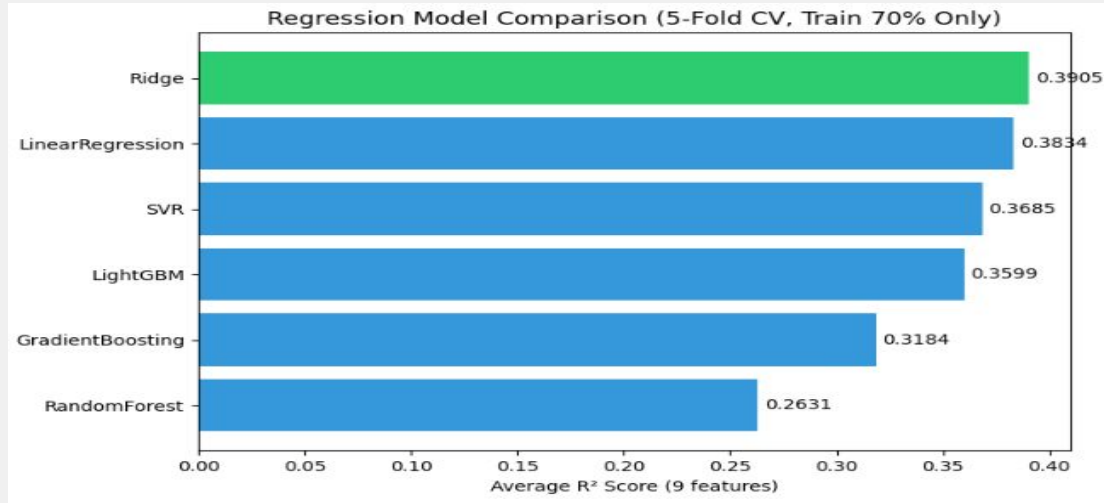
✓ 원인 분석

- 설명: 학습 데이터(30,000개)와 평가 데이터(500개)가 완전히 다른 곡들로 구성 → 분포 불일치
- 해석: LightGBM이 학습 데이터의 세부 패턴을 암기(과적합)하여 본 적 없는 아티스트/앨범에 대해 일반화 실패

결과: 모델 변경 필요 → 6개 회귀 모델 비교 실험 진행

5.2 모델 학습-6개의 회귀 모델 비교 분석

작성자: 정혜원



✓ 비교 대상 모델

- 설명: LinearRegression, Ridge, RandomForest, GradientBoosting, LightGBM, SVR 총 6개 모델
- 해석: 선형 모델부터 앙상블 모델까지 다양하게 비교하여 메타데이터 기반 예측에 가장 적합한 모델 탐색

✓ 평가 방법

- 설명: `df_train(79,800곡)`에서 20,000곡 샘플링 후 5-Fold Cross Validation으로 R^2 와 RMSE 측정
- 해석: 교차 검증으로 과적합 여부를 확인하고 일반화 성능이 좋은 모델 선정

✓ 비교 결과

- 설명: Ridge(0.3905) > LightGBM(0.3599) > RandomForest(0.2631)
- 해석: Ridge가 9개 오디오 특성 전체에서 가장 안정적인 R^2 달성

결과: Ridge 모델이 평균 R^2 0.3905로 1위 → 최종 모델로

5.2 모델 학습-최종 모델 선택(Ridge Regression)

작성자: 정혜원

오디오 특성	초기 (LightGBM)	최종 (Ridge)	개선폭
danceability	-0.177	0.444	+0.621
energy	-0.190	0.495	+0.685
valence	-0.209	0.340	+0.549
acousticness	-0.124	0.516	+0.640
instrumentalness	-0.359	0.491	+0.850
speechiness	-0.454	0.435	+0.889
liveness	-0.335	0.232	+0.567
tempo	-0.064	0.062	+0.126
loudness	-0.194	0.499	+0.693
평균	-0.234	0.390	+0.624

✓ Ridge를 선택한 이유 1: 최고 성능

- 설명: 9개 오디오 특성 평균 R^2 0.3905로 6개 모델 중 1위
- 해석: acousticness(0.52), energy(0.49), loudness(0.50) 등 주요 특성에서 안정적인 예측력 확보

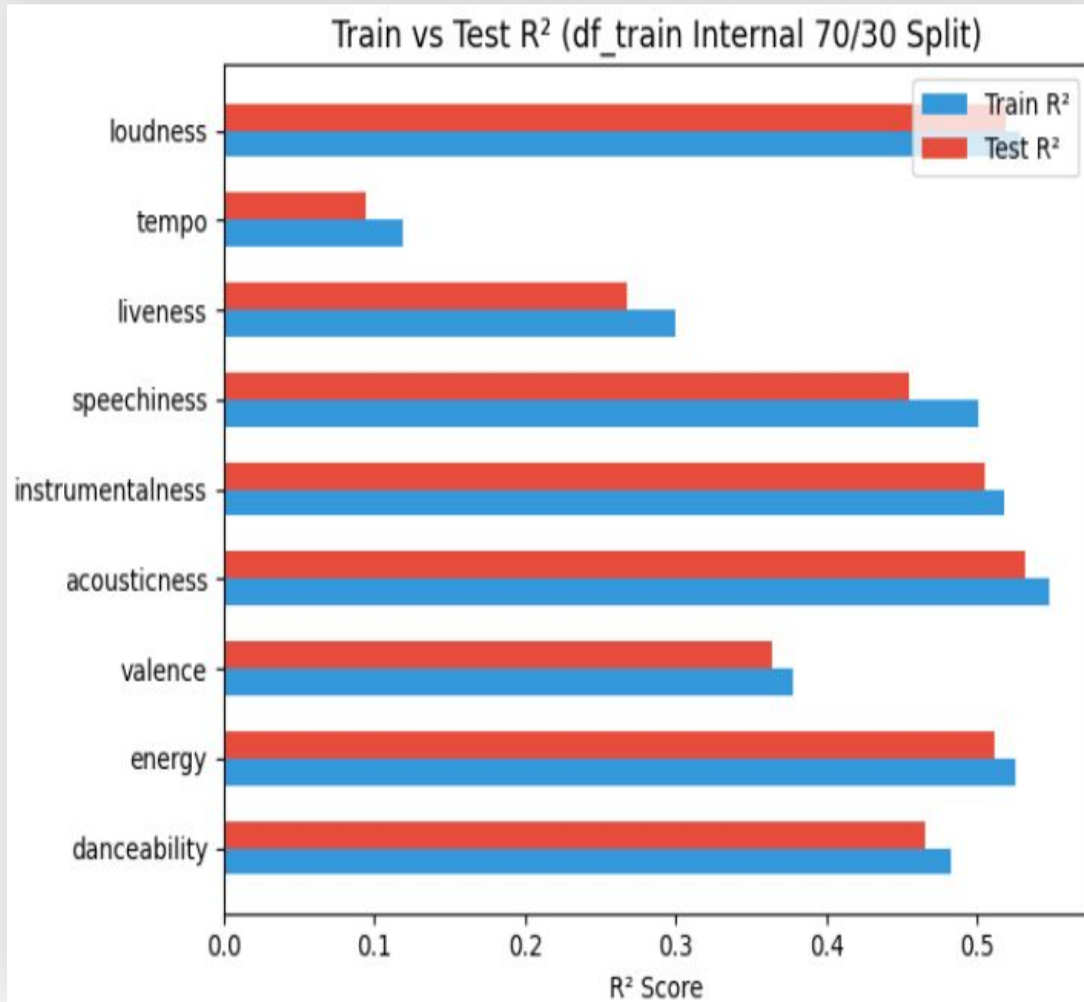
✓ Ridge를 선택한 이유 2: 과적합 방지

- 설명: L2 정규화(가중치 제곱합 페널티)로 과적합 억제
- 해석: 가중치가 너무 커지는 것을 방지하여 새로운 데이터에도 일반화 성능 유지

✓ Ridge를 선택한 이유 3: 선형 관계에 적합

- 설명: 메타데이터(장르, 아티스트)와 오디오 특성 간 관계가 복잡한 비선형보다 단순 선형에 가까움
- 해석: 트리 모델은 불필요하게 복잡한 패턴을 학습하려 했으나 Ridge는 핵심 선형 관계만 포착

결과: R^2 -0.234(LightGBM) → 0.3905(Ridge)로 대폭 개선



✓ 예측이 잘 되는 특성 ($R^2 > 0.45$)

- 설명: acousticness(0.52), energy(0.49), loudness(0.50), instrumentalness(0.49)
- 해석: 장르/아티스트 정보와 강한 연관성 보유
예) 어쿠스틱 장르 → 높은 acousticness 예측 가능

✓ 예측이 보통인 특성 ($R^2 0.3 \sim 0.45$)

- 설명: danceability(0.44), speechiness(0.44), valence(0.34)
- 해석: 메타데이터와 어느 정도 연관성 있으나 변동 폭 존재

✓ 예측이 어려운 특성 ($R^2 < 0.3$)

- 설명: liveness(0.23), tempo(0.06)
- 해석: tempo는 메타데이터와 상관관계가 거의 없어 예측 한계 → 추천 시스템에서 tempo 가중치 축소로 대응

결과: 9개 중 7개 특성에서 $R^2 > 0.2$ 달성, 실용적 예측 가능

※ R^2 수치는 5-Fold CV 기준

df_test (34,200곡)

1 PMS 생성

Personal Music Space

발라드 장르 100곡 선정

사용자 취향 프로필 생성용

2 EMS 준비

External Music Space

나머지 34,100곡

추천 후보 선별 대상

3 오디오 특성 제거

서비스 환경 시뮬레이션

9개 오디오 컬럼 삭제

메타데이터만 보유

PMS 100곡 / EMS 34,100곡 (메타데이터만)

✓ PMS (Preference Music Set) 생성

- 설명: df_test(34,200곡)에서 발라드 장르 100곡을 사용자 취향 곡으로 선정
- 해석: 사용자가 "좋아요"를 누른 곡들로 가정, 이 곡들의 오디오 특성 패턴으로 취향 프로필 생성

✓ EMS (Evaluation Music Set) 준비

- 설명: df_test에서 PMS 제외 나머지 약 34,100곡
- 해석: AI가 평가하여 추천 후보를 선별할 대상 곡 풀
실제 서비스에서 신곡/미청취곡에 해당

✓ 오디오 특성 제거 (서비스 환경 시뮬레이션)

- 설명: PMS/EMS에서 실제 오디오 특성 9개 컬럼 삭제
- 해석: 실제 서비스 환경에서는 Spotify API 없이
메타데이터만 존재하는 상황을 시뮬레이션

결과: PMS 100곡 / EMS 34,100곡 (메타데이터만 보유)



✓ PMS/EMS 오디오 특성 예측

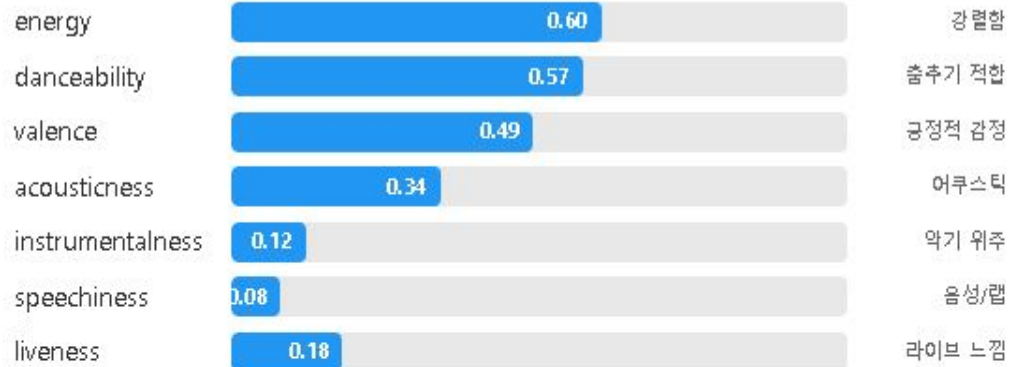
- 설명: Ridge 모델을 활용해 PMS(100곡)와 EMS(34,100곡)의 오디오 특성 9개를 메타데이터 기반으로 예측
- 해석: 오디오 특성이 없는 곡들에 AI가 자동으로 energy, danceability, valence 등을 부여

✓ 예측값 DB 업데이트

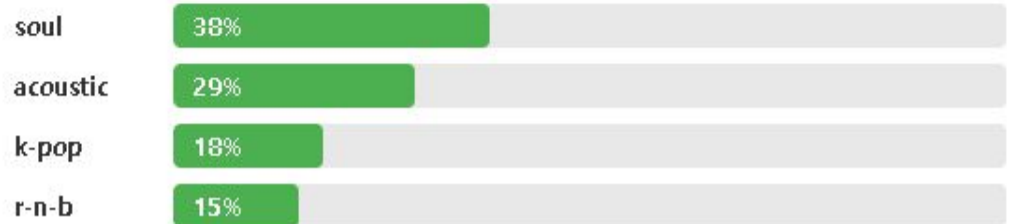
- 설명: 예측된 오디오 특성값을 각 곡의 메타데이터에 추가 후 MySQL 데이터베이스에 저장
- 해석: 실제 서비스에서 신곡이나 외부 곡에도 AI 예측 특성을 활용하여 추천 시스템 작동 가능

결과: PMS/EMS 모든 곡에 AI 예측 오디오 특성 반영 완료

PMS 100곡 기반 사용자 오디오 특성 프로필



Top 장르 분포 (PMS 기준)



✓ 오디오 특성 통계 분석

- 설명: PMS 100곡의 예측된 오디오 특성에 대해 평균, 표준편차, 중앙값, 사분위수 계산
- 해석: 사용자가 선호하는 **energy, danceability, valence** 등의 수치적 취향 패턴을 정량화

✓ 장르/아티스트 분포 분석

- 설명: PMS 내 장르별 비율과 고유 아티스트 목록 추출
- 해석: 사용자의 장르 선호도(예: 발라드 38%, soul 28%)와 좋아하는 아티스트 정보를 추천에 활용

✓ 프로필 요약

- 설명: 9개 오디오 특성 평균 + Top 장르 4개 + 아티스트 수
- 해석: 이 프로필을 기준으로 EMS 곡들과 유사도 비교

**결과: 사용자 취향 프로필 생성 완료
(9개 특성 + 장르 + 아티스트)**

```
weights = {  
    # 오디오 유사도 35% 예측된 오디오 특성(danceability, energy 등)과 사용자 취향의 유사도  
    'predicted_audio': 0.35,  
    # 질적 정보 25% 플레이리스트명·장르 기반 컨텍스트 추론  
    'qualitative': 0.25,  
    # 장르 15% 사용자 선호 장르와의 매칭 점수  
    'genre': 0.15,  
    # 아티스트 10% 기존에 들은 아티스트인지 여부  
    'artist': 0.1,  
    # 분류기 학습 시 적용 (선택적)  
    'preference': 0.15 if self.preference_classifier and self.preference_classifier  
}
```

✓ 코사인 유사도 계산

- 설명: 사용자 취향 벡터와 EMS 각 곡의 벡터 간 코사인 유사도(0~1) 계산
- 해석: 벡터 방향이 비슷할수록 높은 점수, 취향과 유사한 곡일수록 상위 랭크

✓ 하이브리드 스코어링 방식

- 설명: 기존 4가지 유사도 합산 (85%) + 피드백 반영시 preference 15% 추가 → 총 100%
- 해석: 단순 오디오 유사도만이 아닌 다양한 요소를 종합하여 편향되지 않은 추천 제공

✓ GMS 후보 선정

- 설명: EMS 34,100곡 중 추천 점수 0.7 이상인 곡을 GMS 후보로 선별
- 해석: AI가 사용자 취향에 70% 이상 부합한다고 판단한 곡들

결과: EMS → GMS 후보 약 140곡 선별

📁 의외성 주입 (Unexpectedness Injection)

[Before] GMS 후보: 140곡
- 점수 범위: 0.704 ~ 0.745

[After] 의외성 주입 완료 (Level: MEDIUM)
- 최종 GMS: 140곡
- 의외성 비율: 20%

📊 의외성 주입 효과 비교:

구분	평균 점수	최소 점수	최대 점수
Before	0.7261	0.7038	0.7454
After	0.7287	0.7134	0.7454

✅ 의외성 주입 완료! gms_candidates가 업데이트되었습니다.

🎵 의외성 주입 후 Top 10:

1. Wherever I Go	by OneRepublic	(0.714)
2. I'll Keep You Safe	by Vluestar	(0.723)
3. Guaranteed - Humming Version	by Eddie Vedder	(0.742)
4. Hypnotized	by Unodavid	(0.723)
5. The Remedy (I Won't Worry)	by Jason Mraz	(0.741)
6. Whiny Bitch	by Harley Poe	(0.743)
7. Say Something	by A Great Big World	(0.717)
8. Breakfast With the Moon	by Powfu	(0.723)
9. Heaven	by OneRepublic	(0.714)
10. I Can Hear Your Voice	by Instrumental Worship Project	

✅ 의외성 필요성

- 설명: 추천 결과가 너무 비슷한 곡들로만 구성되는 "필터 버블(Filter Bubble)" 문제 발생 가능
- 해석: 항상 같은 스타일만 추천하면 사용자가 새로운 음악을 발견할 기회를 잃음

✅ 의외성 주입 방식

- 설명: LOW/MEDIUM/HIGH 3단계 수준 선택 가능
상위 추천곡 일부를 다른 장르/스타일 곡으로 교체
- 해석: 취향과 다소 다르지만 좋아할 가능성이 있는 곡 노출
사용자의 음악적 탐색 범위 확장

✅ 적용 결과

- 설명: MEDIUM 수준 적용 시 Top 10 중 2~3곡이 새로운 장르의 곡으로 교체됨
- 해석: 발라드 취향 사용자에게 어쿠스틱/인디 곡 추천

결과: 추천 다양성 확보, 필터 버블 방지

- 1 **피드백 수집**
사용자가 GMS 추천곡에서 삭제(싫어요)한 곡 정보 수집
명시적 부정 피드백을 통해 싫어하는 오디오 특성 패턴 파악

- 2 **프로필 벡터 보정**
삭제된 곡의 오디오 특성을 사용자 프로필에서 차감

```
adjusted_profile = pms_features - 0.3 * (deleted_features - pms_features)
```


싫어하는 방향의 특성을 프로필에서 멀어지게 조정 (alpha=0.3, 30% 피드백 감도)

- 3 **EMS 재평가**

```
cosine_similarity(ems_audio, adjusted_profile.reshape(1, -1))
```


보정된 프로필로 코사인 유사도 기반 재정렬 (0.7+ 임계값)

피드백 반영 결과
프로필 벡터 보정 적용 (삭제곡 반영)

✓ 피드백 수집

- 설명: 사용자가 GMS 추천곡에서 삭제(싫어요)한 곡 정보 수집
- 해석: 명시적 부정 피드백을 통해 사용자가 싫어하는 오디오 특성 패턴 파악

✓ 프로필 벡터 보정

- 설명: 삭제된 곡의 오디오 특성을 사용자 프로필에서 차감
- 코드: `adjusted_profile = pms_features - 0.3 * (deleted_features - pms_features)`
- 해석: 싫어하는 방향의 특성을 프로필에서 멀어지게 조정

✓ EMS 재평가

- 설명: 보정된 프로필로 코사인 유사도 재계산
- 코드: `cosine_similarity(ems_audio, adjusted_profile)`
- 해석: 업데이트된 취향 벡터와의 유사도 기반 재정렬

결과: 프로필 벡터 보정 적용 (삭제곡 반영)

📁 실행 결과 요약:

1. 데이터 전처리: 결측치/중복 처리, EDA 완료
2. 피처 선별: 12개 오디오 특성 → 9개 연속형 특성 (근거 기반)
3. 피처 분석: 상관관계 히트맵, 분포 분석, 피처 선정
4. 데이터 분할: 70% 학습 (79,800곡) / 30% 시뮬레이션 (34,200곡)
5. 모델 비교: 6개 회귀 모델 × 9개 오디오 특성 (5-Fold CV, df_train)
6. 역추적 모델 학습: df_train 79,800곡 기반 (내부 7:3 분리)
7. PMS 생성: 100곡 (df_test에서 발라드 추출, 오디오 특성 제거)
8. EMS 생성: 34,100곡 (df_test - PMS, 오디오 특성 제거)
9. 코사인 유사도 추천: GMS 140곡 생성
10. 유저 피드백: 13곡 삭제
11. 프로필 보정: 삭제 피드백 반영 완료

📁 생성된 파일:

- app/models/audio_predictor.pkl (역추적 모델)
- app/models/metadata.json (모델 메타데이터)

🔑 핵심 설계:

- 데이터 분리: 70% 학습 / 30% 서비스 시뮬레이션 (Data Leakage 방지)
- 모델: Ridge 회귀 (audio_predictor.pkl) 1개
- 추천: 코사인 유사도 기반 (별도 분류 모델 없음)
- 피드백: 프로필 벡터 보정 방식

✅ 모든 단계 완료!

✅ 데이터 처리

- 설명: 114,000곡 전처리 및 70%/30% 분할
(79,800곡 학습 / 34,200곡 시뮬레이션)
- 해석: Data Leakage 방지를 위한 명확한 데이터 분리

✅ 피처 선별

- 설명: 12개 → 9개 오디오 특성 선별 (상관관계 분석 기반)
- 해석: 예측 정확도 및 모델 효율성 향상

✅ 모델 선정

- 설명: 6개 회귀 모델 비교 후 Ridge 선정 (5-Fold CV)
- 해석: 메타데이터만으로 오디오 특성 예측 가능

✅ 추천 생성

- 설명: PMS 100곡 → GMS 140곡 생성 (코사인 유사도)
- 해석: 사용자 프로필 기반 개인화 추천

✅ 피드백 반영

- 설명: 13곡 삭제 피드백 → 프로필 벡터 보정 (alpha=0.3)
- 해석: 프로필 보정을 통한 즉시 재추천 및 정확도 향상

결과: 모든 단계 완료 (6 Phase, 16 Step)

GBRegressor+SVM 모델

5.2 모델 학습-문제 정의(왜 오디오 역추적 모델이 필요한가?)

작성자: 최두석

✓ Spotify API의 정책 변경-오디오 특성 제공 중단

- 추천 시스템 고도화를 위해 필수적인 9가지 오디오 특성 (Audio Features) 확보 난항
- **Audio Features**란? : (Danceability, Energy, Valence, Tempo, Loudness, Acousticness, Instrumentalness, Speechiness, Liveness) 음원의 리듬, 에너지, 밝기, 보컬 특성 등을 수치로 표현한 정형 데이터입니다.
- 곡명, 가수이름, 앨범명만 100% 확보가 가능. -> **자료가 미흡하다고 판단**

✓ 해결 아이디어 -> 오디오 역추적 모델 (Audio Backtracking Model) 도입

- 발상의 전환: "오디오 없이 오디오 특성을 알 수 있을까?"
- 예측 방법론: 메타데이터(텍스트)를 기반으로 오디오 특성 값을 역으로 추론
- 114K 음악 데이터(9가지 오디오 특성 보유) -> 역추적 모델 생성
- 역추적 모델
 - Input: 아티스트명(Artist), 곡명(Track Title)
 - Output: 9가지 오디오 피쳐 수치 예측 (Regression)

✓ INPUT 데이터(단순 곡명/아티스트명)만으로는 정보 부족

- '설명적 태그(Descriptive Tags)' 필요

✓ API 선정 과정

- 'MusicBrainz 등 음악사이트들의 일반 메타데이터 API'
 - 정보 수집률(Coverage) 50% 미만 (부적합)
- **Last.fm API**: 사용자 기반 **artist_tag** 확보율 92.1% (채택)
 - 사용자가 직접 단 태그(Rock, india, brasil, 90s, Gangsta rap, Mellow, Electronic 등)로 곡의 분위기를 설명하는 핵심 Feature로 작용

태그 추가 효과			
항목	태그 없음 (Base)	태그 추가 (Applied)	개선율 (Improvement)
평균 R2 Score	0.36	0.4549	26.40%

✓ 태그 커버리지: 92.1%

핵심 인사이트:

- **Last.fm** 태그가 오디오 예측에 큰 도움
- 텍스트 정보의 풍부함이 정확도 향상 키라는 것을 증명


```
def get_lastfm_artist_tags(artist_name, retries=3):
    """Last.fm API로 아티스트 태그 조회"""
    url = 'http://ws.audioscrobbler.com/2.0/'
    params = {
        'method': 'artist.getinfo',
        'artist': artist_name,
        'api_key': LASTFM_API_KEY,
        'format': 'json'
    }

    for attempt in range(retries):
        try:
            resp = requests.get(url, params=params, timeout=10)
            if resp.status_code == 429:
                time.sleep(2)
                continue
            if resp.status_code != 200:
                return None

            data = resp.json()
            if 'error' in data:
                return None

            artist_data = data.get('artist', {})
            tags = artist_data.get('tags', {}).get('tag', [])
            tag_names = [t['name'] for t in tags] if isinstance(tags, list) else []

            return ', '.join(tag_names) if tag_names else None
        except Exception as e:
            if attempt < retries - 1:
                time.sleep(1)
                continue
            return None

    return None
```

✓ 데이터셋 구성 (Dataset Composition)

- Volume: 총 114,000 곡 (Spotify 114K Dataset)
- Input Data (X - Text):
 - 기본 정보: 아티스트명, 곡명, 앨범명
 - 핵심 정보: Last.fm Artist Tags (장르, 분위기, 악기 등)
- Target Data (Y - Label):
 - 9가지 오디오 특성 (Danceability, Energy, Valence, Tempo 등)

✓ 수집 파이프라인 (Collection Pipeline)

- Ground Truth 확보: Spotify API를 통해 **114K 곡의 정답지 (오디오 특성) 수집**
 - Train: 91,200곡 (80%)
 - Test: 22,800곡 (20%)
- Feature Enrichment: **Last.fm API** 연동하여 해당 곡들의 **아티스트 태그 정보 매핑**
- Preprocessing: 데이터 병합 및 **CSV 저장** → 분석 엔진 (FastAPI) 학습용 데이터로 가공

✓ 핵심 기술: **TF-IDF** (Term Frequency - Inverse Document Frequency)

- 목적: 텍스트(Artist, Track, Tags)를 기계가 이해할 수 있는 수치(Vector)로 변환
- 차원 구성 (Total 8,003 D):
 - Artist (3,000) + Track (2,000) + Album (2,000) + Tags (1,000) + Numeric (3)
- 1단계: TF (단어 빈도): 예시 : "Beatles"
 - 114,000곡 중: "Beatles": 100번 등장
 - > 계산: $TF("Beatles") = 100 / 114,000 = 0.0009$
- 2단계: IDF (역문서 빈도):
 - "Beatles": 드물게 등장 → IDF 높음 (더 중요!)
 - > 계산 : $IDF("Beatles") = \log(114000 / 100) = 7.04$
- 3단계: **TF-IDF 계산** :
 - $TF-IDF = TF \times IDF$
 - > 계산 : $TF-IDF("Beatles") = 0.0009 \times 7.04 = 0.0063$

✓ 최적화 전략: **Sparse Matrix** (희소 행렬) 활용

- 효과: 8,000차원 중 0이 아닌 값만 저장(ex.아티스트 3000차원 중에 "Beatles만 0이 아닌 값)하여 **메모리 사용량 1/100로 절감** (가수 한명당 => 3000차원x8bytes=24KB) vs (1x12bytes=12bytes)

✓ 왜 TF-IDF를 사용?

- 장점 : **간단하고 빠름(TF-IDF 계산) + 메모리 효율적(Sparse Matrix) -> 114K 데이터에도 빠른 학습 가능**

VS Embedding을 하여야 하는지 고민하였습니다.

Embedding 장점: 의미 이해, 문맥 파악 / 단점: 느림, 무거움 => **114K데이터+단순 단어 조합**이므로

=> TF-IDF 가 온라인 서비스에 적합하다고 판단

30k곡 비교 실험 결과 (A 선택)

비교 항목	Model A (Selected)	Model B (Deep Learning)	Model C (Hybrid)
알고리즘	TF-IDF + GBR	Embedding + MLP	Embedding + GBR
데이터 형태	Sparse (1500D)	Dense (384D)	Dense (384D)
R ² (30K)	0.1909	0.2884 (1위)	0.223
학습 시간	49.7초 (Fast)	56.3초	4983.1초 (83분) ✗
비고	메모리 효율 최상	배포 복잡도 증가	학습 불가 판정

✓ 딥러닝을 해야 하는가? 딥러닝(Embedding) vs 통계적 방법(TF-IDF)

- “정형 데이터, 비선형 관계” 분석에는 이론적으로 GradientBoostingRegressor가 적합하여 후보로 선정
- 신경망이 정말 필요한가? 에 대한 고민을 하였습니다. 그래서 “얇은 신경망” 모델로 테스트 하여, 성능이 월등하면 딥러닝 모델도 본격적으로 비교하려고 계획
-> “비선형 학습 가능”한 얇은 신경망 모델인 “MLPRegressor”를 후보로 두고 비교하였습니다.
- 비교 실험
 - Model A: TF-IDF + GradientBoostingRegressor
 - Model B: Embedding + MLPRegressor
 - Model C: Embedding + GradientBoostingRegressor

✓ 1차 비교 : 30,000곡만으로 모델 성능 비교

- 모델 C는 학습 속도가 너무 느려서 제외
- 딥러닝 기반의 Model B가 R² 0.28로 1위를 차지
- 하지만 모델 A를 선택 (뒷 페이지에서 이유를 설명)

114k곡 통한 테스트 결과 (Model A 선택)

비교 지표	Model A (선택 모델)	Model B (비교 모델)
전처리 시간	3.02초	1,089.07초
피크 메모리	259.9 MB	3,162.6 MB
평균 R Score	0.4488	0.4947

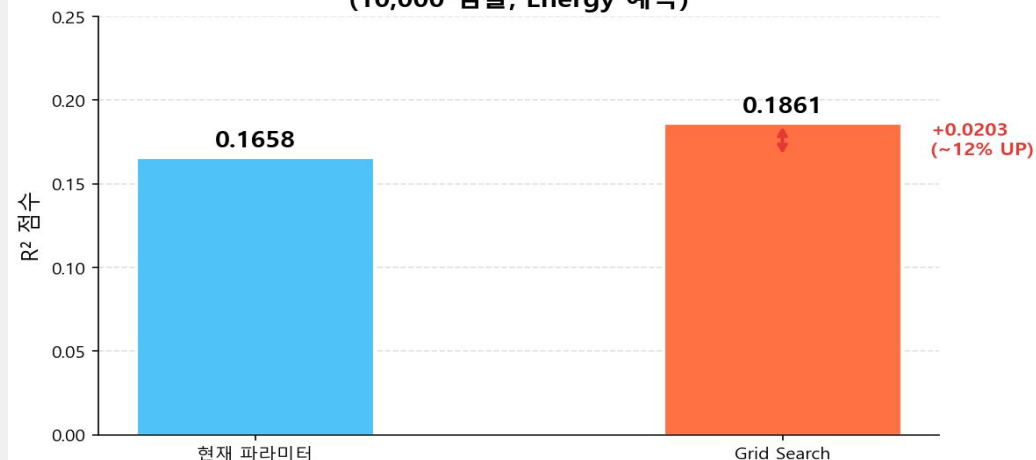
✓ 역추적 모델은 온라인 환경에서 오디오 특성 빠르게 채워 넣는 보조 모델이어서, 빠른 실시간 추론, 대규모 데이터 유입에 대비하여 전처리 속도 & 메모리 부하가 중요하다고 판단

1. 성능 차이 제한적 : R2 0.4488 vs 0.4947 (4.64% 차이) 모델B 승리
2. 전처리 시간 : (360배의 차이) 모델A 승리
Model A (TF-IDF): 단순 통계 기반 -> 3.02초
Model B (Embedding): 텍스트를 이해하기 위해 BERT나 Sentence-Transformers 같은 거대 모델을 통과시켜야 합니다. -> 1089.07초
3. 메모리 피크 차이: 259.9 MB vs 3162.6 MB (12.17배 차이) 모델A 승리
4. 배포 환경 경량화 : 모델A 승리
 - Model A: Scikit-learn 라이브러리 하나 필요. 가볍고 빠름
 - Model B: 딥러닝(PyTorch, TensorFlow, Transformers 라이브러리 필요)
 - 문제: 도커(Docker) 이미지 크기가 수 GB로 커지고, 서버 부팅 속도가 느려지며, API 응답 속도에도 영향을 줍니다. 오디오 역추적은 '보조 기능'인데, 이를 위해 주객전도가 일어나는 것을 막고자 했습니다.

✓ 오디오 역추적은 추천을 위한 “입력 피처를 생성하는 보조 모델”이므로, '최고 성능'보다 '최적 효율(Model A)'을 선택.

파라미터	기본값	사용값	변경 이유 및 효과
n_estimators	100	200	더 많은 트리(Tree)-> 모델 예측 정확도 향상
max_depth	3	5	트리의 깊이를 늘려 데이터 내의 더 복잡한 패턴을 학습
subsample	1	0.8	데이터의 80%만 무작위 활용하여 모델의 과적합(Overfitting) 방지

Grid Search vs 현재 파라미터 — R² 비교
(10,000 샘플, Energy 예측)



✓ Grid Search 없이 경험적 최적값 적용

- 역추천 모델은 보조 모델로서의 효율성 추구
- Gradient Boosting은 하이퍼파라미터 영향도가 크지 않은 모델
- 경험적 사용값의 중간값 선택

✓ 파라미터 적용 단계

1단계: scikit-learn 공식 기본값 확인



2단계: 경험적 사용의 중간값 적용
(n_estimators ↑, max_depth ↑)

✓ 비교 검증

- 실제로 경험적 최적값 내에서 grid-search를 한 결과는 R2 score에서 2%내 차이만 보였습니다.

✓ 기존 방식의 한계(문제 정의)

- 방식: "장르가 Rock이면 Rock 추천", "가수가 아이유면 아이유 추천"
- 한계:
 - 다양성 부족 (Filter Bubble)
 - "같은 아티스트라도 슬픈 곡은 싫다"**는 미묘한 취향 반영 불가

✓ 우리의 접근 (AI Learning)

- 핵심 질문: **"사용자가 이 곡을 왜(Why) 좋아하는가?"**
- 해결책: Hybrid Vectorization (393D)
 - 텍스트**(Meaning): 사용자가 작성한 **아티스트 태그**, **곡/앨범명**에서 추론되는 분위기 활용 (384차원)
 - 오디오(Feeling): 실제 들리는 **소리의 특성** 활용 (9차원)

✓ PMS와 EMS를 활용한 샘플링(학습 데이터 구성)

① 데이터 분포 (Total 1,751 Samples)

- Positive (24.6%): 431곡
 - Source: **PMS** (My Lounge) - 사용자가 직접 승인하고 찜한 곡
- Negative (75.4%): 1,320곡 (1:3 비율)
 - Source: **EMS** (사용자에게는 미지의 곡)에서 무작위 추출
- 1:3은 추천시스템에서 가장 일반적인 비율 (Bayesian Personalized Ranking 논문 기반)

② Negative 샘플링 전략 (Key Logic)

- Hard Negative (50%): 사용자가 좋아하는 아티스트의 '다른 곡'
 - 목적: "이 가수를 좋아한다고 무조건 추천하지 마라" (**변별력** 학습)
- Random Negative (50%): 완전히 다른 아티스트/다른 장르의 곡
 - 목적: 전반적인 비선호 영역 학습

L6 모델과 비교한 모델들	차원	주요 특징	L6 모델과 비교 사항
paraphrase-multi lingual-MiniLM-L 12-v2	384D	다국어, 가벼움	L6보다 느낌 태그/메타데이터 조합에는 과함
intfloat/multilingual-e5-small	384D	최신 비대칭 검색 최적화	FastAPI 단순 파이프라인에는 복잡도 ↑
jhgan/ko-srobert a-multitask	768D	한국어 특화	768D → 메모리·연산 비용 2배

- ✓ 피쳐 구성: 텍스트 의미와 오디오 신호의 결합 + 임베딩
모델 선정 과정 1
- ① 텍스트 임베딩 (384D)
- Input: Artist + Track Title + Album + Last.fm Tags
 - Model: **all-MiniLM-L6-v2** (Sentence-Transformers) 선정
 - 선정 이유:
 - 온라인 실시간 추론 필요 : 경량화 모델로 빠른 추론 속도 (FastAPI 최적화)
 - 384D로 경량 (768D 모델보다 절반)
 - 사전학습된 모델이므로 소규모 데이터(PMS 개인 선호곡 100곡 내외 가능성 높다.)에서도 Transfer Learning 가능
 - 다국어 지원(다양한 국가의 노래) → 한국어 태그도 처리 가능
 - 속도와 성능의 균형
 - “최고 성능”보다 “항상 안정적으로 빠르게 도는 모델”이 필요
 - Output: 384차원 밀집 벡터 (Dense Vector)

L6 기준 - 모델 비교					
모델명	모델 로드 시간	인코딩 시간	처리량 (Texts/sec)	모델 메모리	AUC-ROC
all-MiniLM-L6	8.65초	23.73초	73.8/초	17.8 MB	0.9781
miniLM-L12	53.01초	23.51초	74.5/초	123.8MB (7x)	0.9985
e5-small	9.98초	41.64초 (1.75x)	42/초	252.5MB (14x)	0.9988
ko-sroberta	9.02초	69.40초	25.2	8.9 MB	-

피쳐 조합별 효용성 비교 (SVM, \$C=10\$)		
피쳐 세트 (Feature Set)	차원 (Dim)	AUC-ROC
텍스트 벡터 + 오디오 특성	393D	0.9987
텍스트 벡터 ONLY	384D	0.998
오디오 특성 ONLY	9D	0.738

✓ 피쳐 구성: 텍스트 의미와 오디오 신호의 결합 + 임베딩
모델 선정 과정 2

- SVM (C=10, gamma='scale', kernel='rbf'), 5-Fold Stratified CV의 동일 조건하에 비교하였습니다.
- all-MiniLM-L6는 모델 로딩 + 전체 데이터 인코딩 기준 총 소요 시간이 32.4초로 가장 빨랐습니다.
- e5-small보다 인코딩이 1.75배 빠름 (41.64초 vs 23.73초)
- ko-sroberta보다 인코딩이 2.9배 빠름 (69.40초 vs 23.73초)
- 메모리 소요: all-MiniLM-L6(17.8MB) vs e5-small(252.5MB) → 14배 차이

② 오디오 특성 (9D) - The Feeling

- Input: 오디오 역추적 모델(GradientBoosting)의 예측값
- Input Features: Danceability, Energy, Valence, Tempo, Loudness 등 9개
- 역할 : 텍스트가 설명하지 못하는 '리듬감', '음색' 보완

③ [① + ②] 393D SVM 벡터

- Formula: 텍스트 벡터 (384) + 오디오 특성 (9) = 사용자 취향 반영한 벡터 (393)

5.2 모델 학습-SVM 취향 추천 모델(모델 선정)

작성자: 최두석

모델 성능 비교 (AUC-ROC 기준)				
모델 (Algorithm)	AUC-ROC	Log Loss	CV 표준편차	비고
SVM (RBF Kernel)	0.9987	0.041	± 0.0013	최종 선택 (성능 1위)
Logistic Regression	0.9972	0.051	± 0.0021	복잡한 취향 경계 표현 한계
MLP (128, 64)	0.9957	0.062	± 0.0014	데이터 대비 과한 모델
Gradient Boosting	0.9917	0.1091	± 0.0045	고차원 연속 벡터에 비효율적
Random Forest	0.9848	0.2171	± 0.0068	차원 커질수록 분산 증가

✓ AUC-ROC가 높은 것은 사용자가 좋아하는 곡을 사용자가 싫어하는 곡보다 더 높게 점수 매길 확률이 높다는 의미를 가집니다.

-> 성능이 높다. svm 1위

✓ 교차 검증 (5-Fold Stratified): - AUC-ROC: 0.9987 +/- 0.0013

-> cv 표준편차(+0.0013)는 fold마다 나온 성능 값들이 얼마나 흔들리는가를 의미한다. ->cv 표준편차(+0.0013)는 데이터를 어떻게 나뉘어도 성능이 거의 같았다는 의미이다. -> 성능이 안정적이다. svm 1위

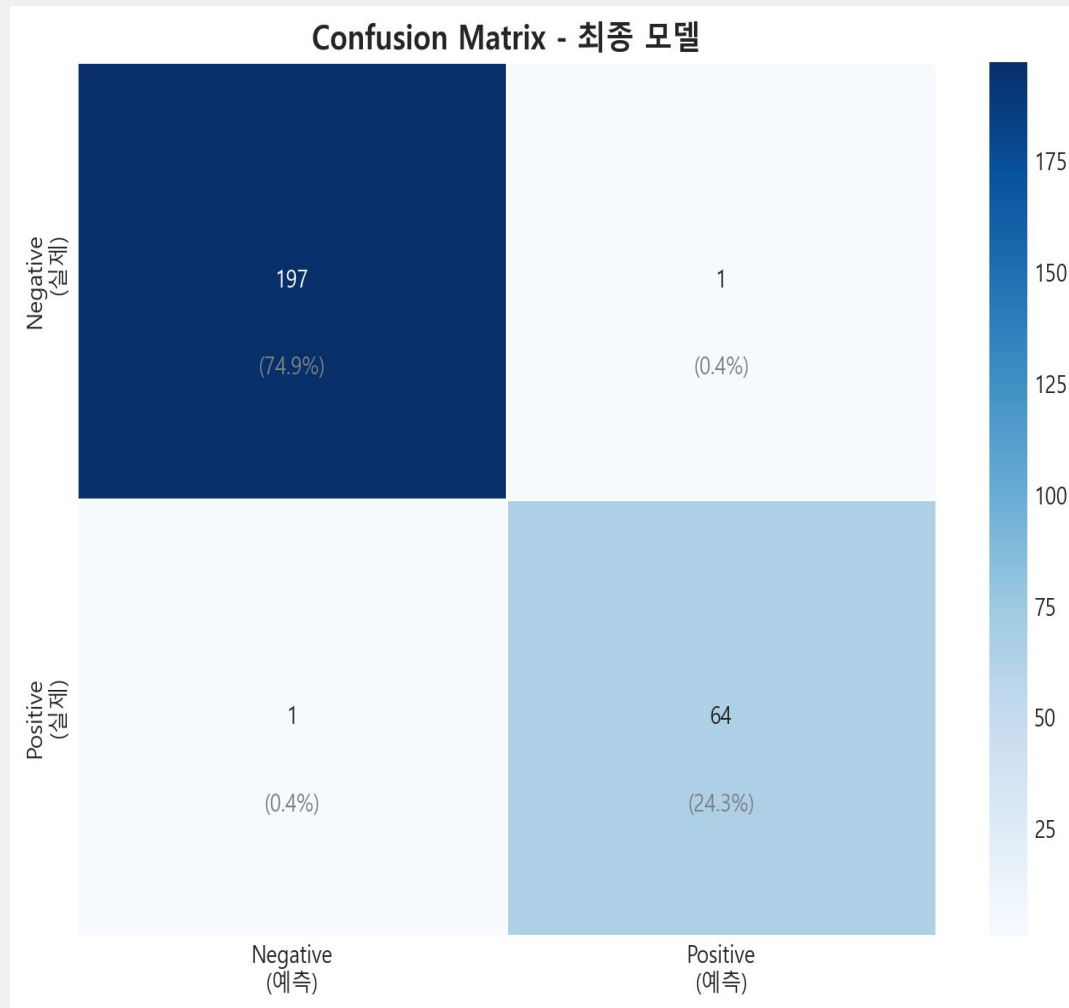
✓ GroupKFold (아티스트 기반 분할) : AUC = 0.9986

-> 아티스트 단위로 데이터를 분리하여, 같은 아티스트의 곡이 train과 test에 동시에 등장하지 않도록 검증한 방식이며, 이 환경에서도 AUC 0.9986을 유지해 아티스트를 암기하는 학습이 아님을 확인했다. = 일반화 성능이 좋다

✓ 현재 피쳐 구조(고차원 임베딩(393D) + 소규모 데이터 (사용자 선호 곡은 몇백곡 내외))에서 가장 안정적인 일반화 성능과 효율을 동시에 제공했기 때문에 SVM을 선택했다.

5.2 모델 학습-SVM 취향 추천 모델(모델 선정)

작성자: 최두석



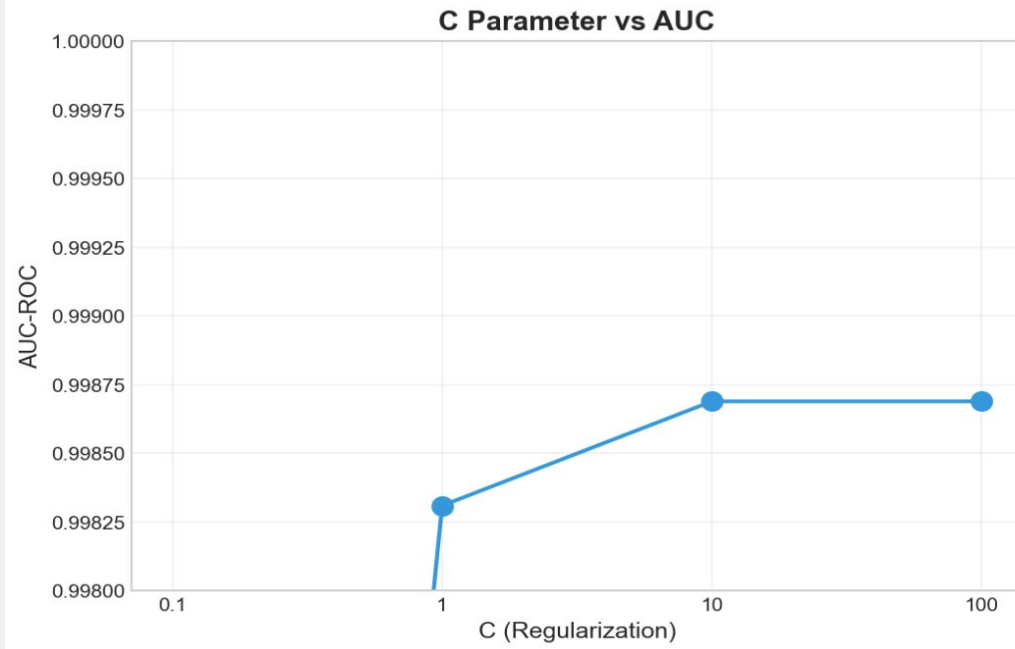
✓ Confusion Matrix 해석

- True Negative: 197개 (74.9%) - 싫어하는 곡을 정확히 거름
- True Positive: 64개 (24.3%) - 좋아하는 곡을 정확히 추천
- 오분류: 단 2건 (0.8%)
 - False Positive: 안 좋아할 곡을 추천한 경우
 - False Negative: 좋아할 곡을 놓친 경우

5.2 모델 학습-SVM 취향 추천 모델(하이퍼파라미터 튜닝)

작성자: 최두석

C (규제 매개변수)	gamma (커널 계수)	AUC-ROC (성능)	비고
0.1	scale	0.9971	-
1	scale	0.9983	-
10	scale	0.9987	최적의 조합 (1위)
100	scale	0.9987	성능 동일 (과적합 위험 증가)
10	auto	0.9985	-
10	0.01	0.9982	-



✓ 5-Fold CV를 사용한 그리드 서치(Grid Search)

- 테스트된 파라미터:
 - C: [0.1, 1, 10, 100] (오분류 허용도)
 - gamma: ['scale', 'auto', 0.01, 0.001] (영향 범위)
 - kernel: ['rbf']

- 최적 파라미터: C=10, gamma='scale'

✓ C=10 선택 이유

- C=10과 C=100 성능 동일 (0.9987)
- C=100은 과적합 위험 → 더 작은 C=10 선택

✓ gamma='scale'이 가장 안정적 (auto보다 높은 AUC)

5.2 모델 학습-SVM 취향 추천 모델(최종 모델 사양)

작성자: 최두석

✓ 모델 유형: 서포트 벡터 머신(Support Vector Machine, SVM)
커널: RBF (방사 기저 함수, Radial Basis Function) - 비선형 패턴 학습용

✓ 하이퍼파라미터 (튜닝 후):

- C (정규화 파라미터): 10
- gamma: 'scale' ($1 / (n_features * X.var())$)
- kernel: 'rbf'
- probability: True

✓ 피쳐 세트: embedding_plus_audio_clean

- 총 차원: 393D
- 텍스트 임베딩 (miniLM): 384D
 - 입력: artist, track_name, album_name, lfm_artist_tags
 - 모델: sentence-transformers/all-MiniLM-L6-v2
- 오디오 특성 (예측값): 9D
 - danceability, energy, speechiness, acousticness
 - instrumentalness, liveness, valence, tempo, loudness
 - 114K Spotify 데이터셋으로 학습된 GradientBoostingRegressor로 예측

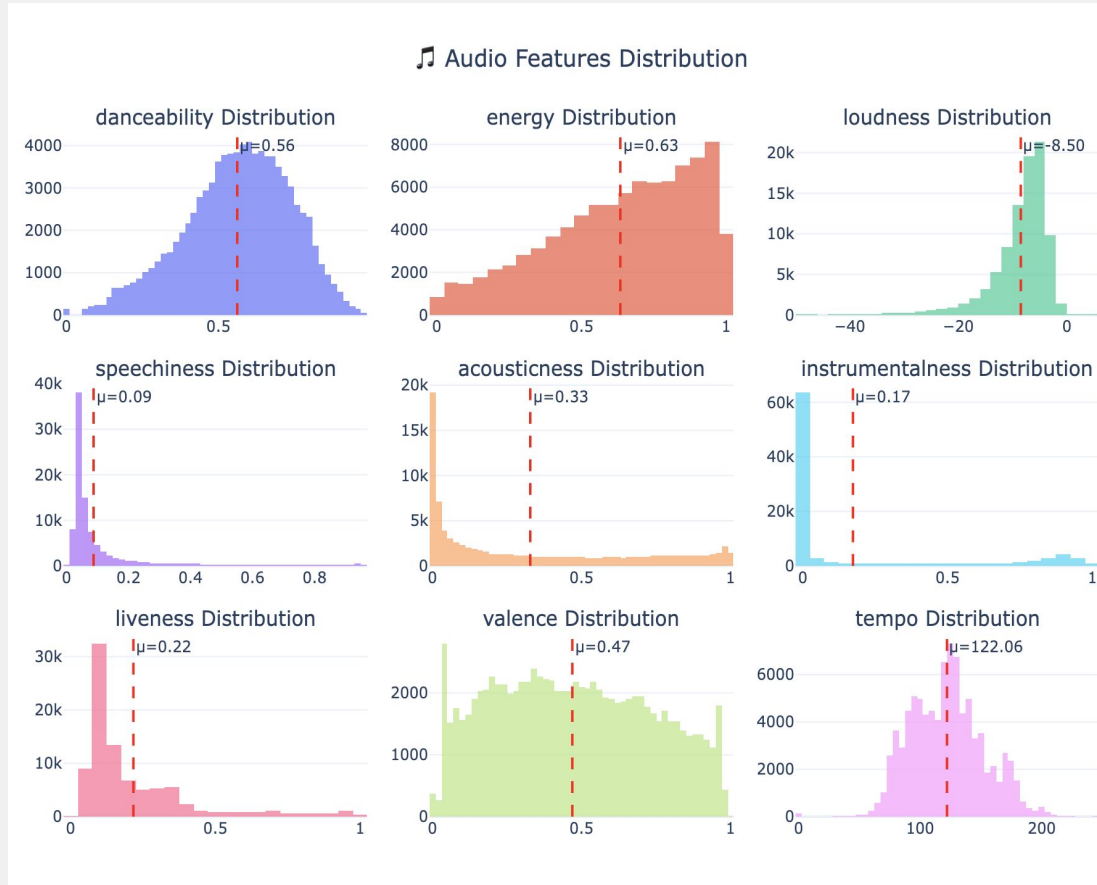
✓ 성능 지표

- 교차 검증 (5-Fold Stratified):
 - AUC-ROC: 0.9987 +/- 0.0009
- 테스트 세트 성능 (15% 홀드아웃, n=263):
 - 정확도(Accuracy): 0.9924
= 전체 예측 중 맞힌 개수 / 전체 예측 수
 - 정밀도(Precision): 0.9846
= 추천한 것 중 실제 좋아한 비율
 - 재현율(Recall): 0.9846
= 좋아할 곡 중 추천한 비율
 - F1-Score: 0.9846
 - AUC-ROC: 0.9995
 - Log Loss: 0.0300

KNN+RandomForest

5.2 모델 학습-89,740곡의 DNA— 9개 숫자로 보는 음악

작성자: 김기열



✓ 1. 모든 곡은 9개 숫자로 표현된다

- Spotify API는 모든 곡을 **9개 오디오 피쳐**로 요약한다
- 이 숫자만으로 "이 사람이 좋아할 곡"을 맞출 수 있을까? — 이것이 실험의 출발점
- 원본 114,000행에서 중복 제거 → **89,740곡**

✓ 2. 분포를 보면 전처리 전략이 보인다

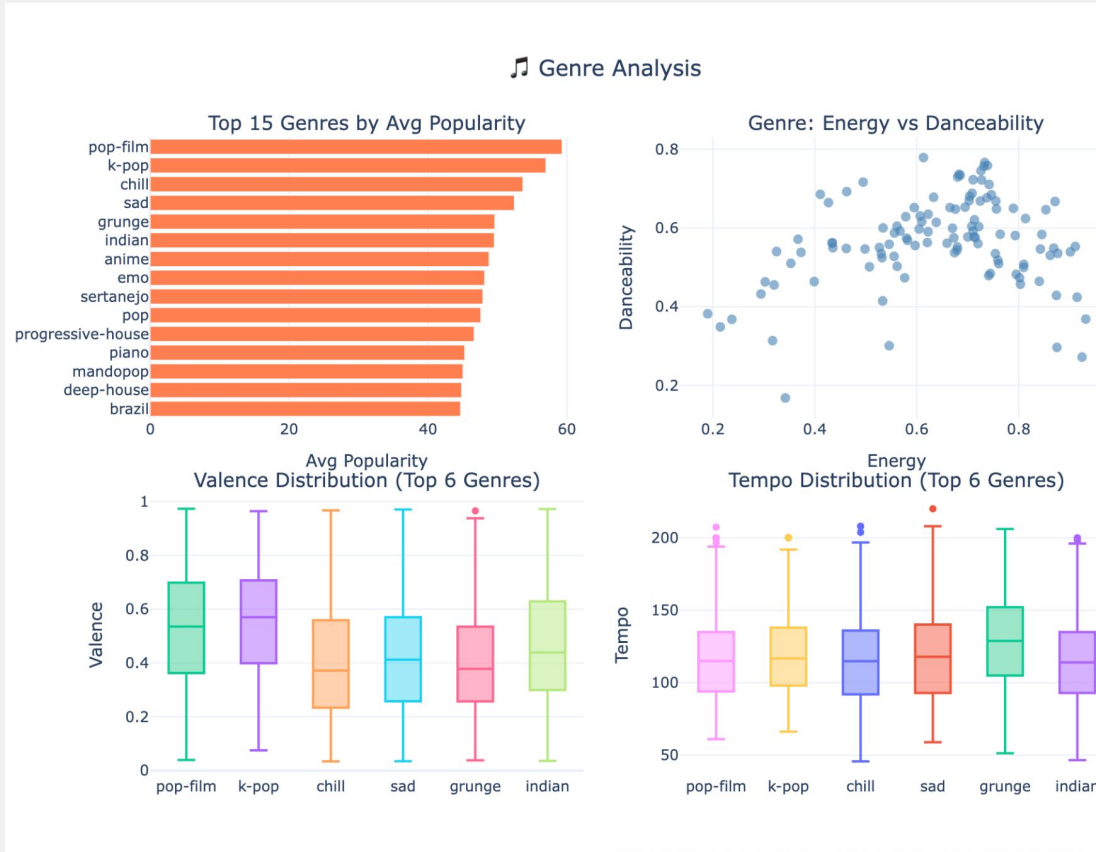
- 양극화된 피쳐가 핵심: acousticness는 0 아니면 1에 몰림 (어쿠스틱 vs 일렉트로닉)
- instrumentalness도 0에 극도로 집중 — 보컬곡 vs 기악곡이 극명하게 갈림 → 이런 **극단적 분포가 장르를 구분하는 시그널**

✓ 3. 스케일 문제

- 7개 피쳐는 0에서 1이지만, loudness(-49에서 4dB)와 tempo(0에서 244BPM)는 스케일이 다름 → 이 **2개만 MinMaxScaler** 적용 (나머지는 불필요)
- 거리 기반 추천에서 스케일 차이는 유사도 왜곡의 직접 원인

5.2 모델 학습-장르별로 보면, 숫자에도 성격이 있다

작성자: 김기열



✓ 4. 장르마다 피쳐 패턴이 다르다

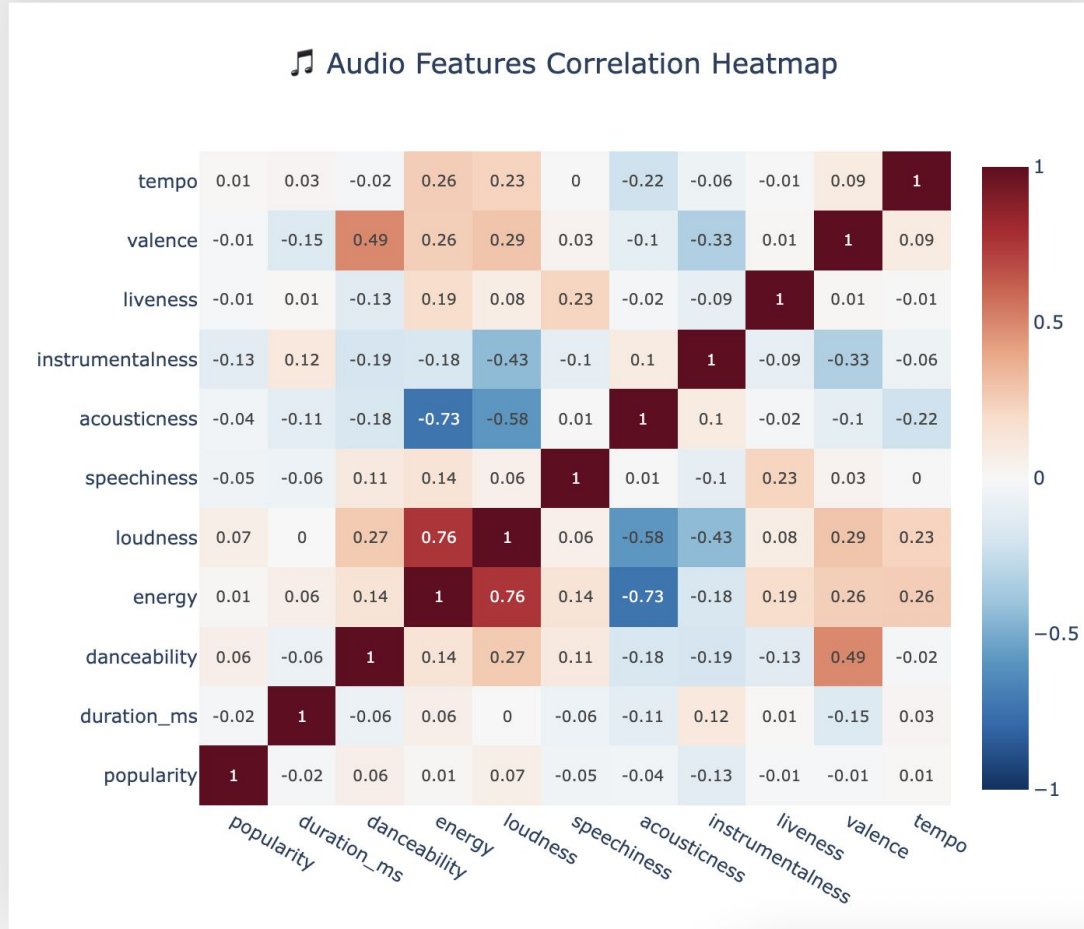
- K-pop: 높은 danceability + energy → **신나는 곡**
- Jazz: 높은 acousticness + instrumentalness → **어쿠스틱 기악**
- Rock: energy는 높지만 다른 장르와 겹치는 구간이 많음
- 장르 간 **피쳐 겹침 정도**가 추천 난이도를 결정한다

✓ 5. 다중 레이블 구조

- 한 곡이 여러 장르에 속함: 1개(81.5%), 2개(13.0%), 3개 이상(5.5%)
- "이 곡은 Pop이면서 Dance" — 장르 경계가 모호한 곡이 18.5% 존재
- 추천 평가 시 **multi-label 구조를 반드시 고려**

✓ 6. 이상치 판단: 전부 유지

- instrumentalness에서 IQR 기준 21.9%가 "이상치"
→ 실은 **기악곡의 정상 범위**
- 이것 제거하면 기악곡 전체를 잃음
- **데이터를 고치기 전에, 도메인을 먼저 이해하라**



✓ 7. 주요 상관관계

- energy ↔ loudness: **$r=0.76$** → 에너지 높은 곡이 음량도 큼
- energy ↔ acousticness: **$r=-0.73$** → 에너지 높으면 어쿠스틱 아님
- 일반 ML에서는 $r>0.7$ 이면 제거 검토 → **그러나 여기서는 유지**

✓ 8. 왜 제거하지 않았는가

- **의미가 다르다**: loudness = 물리적 음량(dB), energy = 음악적 강렬함(종합)
- "조용한 하드코어"도, "시끄러운 어쿠스틱"도 존재
→ 상관이 깨지는 곡이 있음
- 9차원뿐인 피쳐 공간에서 하나를 빼면 **정보 손실이 더 크다**
- **차원이 낮을 때는, 공선성보다 정보 보존이 우선**
- (참고: ablation 미수행 — 저차원 정보 손실 리스크를 고려한 보수적 판단)



✓ 9. 9,447곡(10.5%)의 인기도가 0

- 오디오 피처를 비교하면 다른 곡들과 **거의 동일**한 분포
- 음악적 품질 문제가 아님 — Spotify 인기도 = 최근 재생 수 기반
- 오래된 곡, 비주류 곡이 0으로 집계되는 구조

✓ 10. 제거하지 않은 이유

- Content-based 추천의 장점: **인기도에 편향되지 않고** 음악적 유사성으로 추천
- "아무도 안 듣지만 당신 취향엔 맞는 곡" — 이것이 content-based의 가치

✓ 11. 최종 전처리 결과

- **선택**: 9개 오디오 피처 (danceability, energy, loudness 등)
- **제외**: popularity(인기 편향), key(순환 구조), mode(2값), duration_ms(포맷 차이) 등
- **최종**: 89,740곡 × 9피처

```
AUDIO_FEATURES = [
    'danceability', 'energy', 'loudness', 'speechiness',
    'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo'
]

SCENARIOS = {
    'kpop_pop': {
        'name': 'K-pop/Pop',
        'artists': ['BTS', 'Ed Sheeran', 'Taylor Swift', 'Ariana Grande', 'Billie Eilish']
    },
    'rock_metal': {
        'name': 'Rock/Metal',
        'artists': ['Metallica', 'AC/DC', 'Nirvana', 'Led Zeppelin', 'Linkin Park']
    },
    'jazz_acoustic': {
        'name': 'Jazz/Acoustic',
        'artists': ['Miles Davis', 'Norah Jones', 'John Coltrane', 'Billie Holiday', 'Chet Baker']
    }
}
```

```
N_RANDOM = 5
N_LIKED = 30
SEED = 42
```

```
@dataclass
class Scenario:
    """평가 시나리오 1개"""
    name: str
    liked_indices: np.ndarray # 30개 (추천 입력)
    relevant_indices: np.ndarray # 같은 아티스트의 다른 곡들 (평가 대상)
    candidate_indices: np.ndarray # liked 제외 전체 (~89,710개)
```

```
@dataclass
class ModelResult:
    """모델 1개 x 시나리오 1개 평가 결과"""
    scenario: str
    model: str
    precision_k: float
    recall_k: float
    ndcg_k: float
    genre_hit: float
    diversity: float
    time_sec: float
```

✓ 12. 문제: 유저 히스토리가 없다

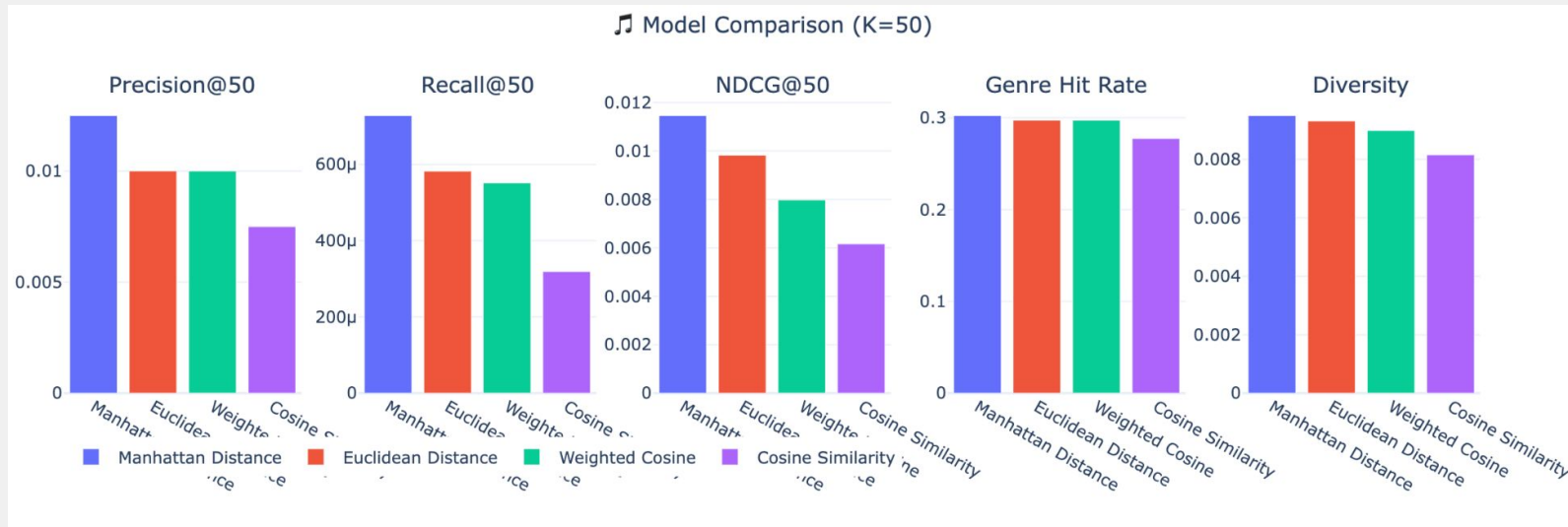
- 일반적 평가: 유저가 들은 100곡 중 10곡 숨기고 맞추기
- 우리는 content-based → 유저 히스토리 없음 → **자체 설계: Artist-Membership 평가**

✓ 13. "BTS 좋아하면, BTS 다른 곡도 추천되어야 한다"

- liked_songs 30곡의 아티스트가 가진 다른 곡이 Top-50에 오는지 측정
- 8개 시나리오: K-pop(relevant=537곡), Rock, Jazz(67곡), Random 5개
- 핵심 메트릭: **NDCG@50** — 상위에 relevant가 올수록 높은 점수

✓ 14. 왜 8개 시나리오인가

- K-pop = 쉬운 문제(곡 많음) / Jazz = 어려운 문제(곡 적음)
- 하나의 **AVERAGE**만 보면 놓치는 정보가 있다 → 시나리오별 분석 필수
- ⚠ proxy metric — 절대 지표가 아닌 **모델 간 상대 비교 도구**로 해석



✓ 15. centroid 기반: 30곡 평균 → 가장 가까운 50곡 추천

모델	NDCG	Genre Hit
Manhattan	0.011	0.302
Euclidean	0.010	0.297
Weighted Cosine	0.008	0.297
Cosine	0.006	0.278

✓ 16. 결과: 전멸

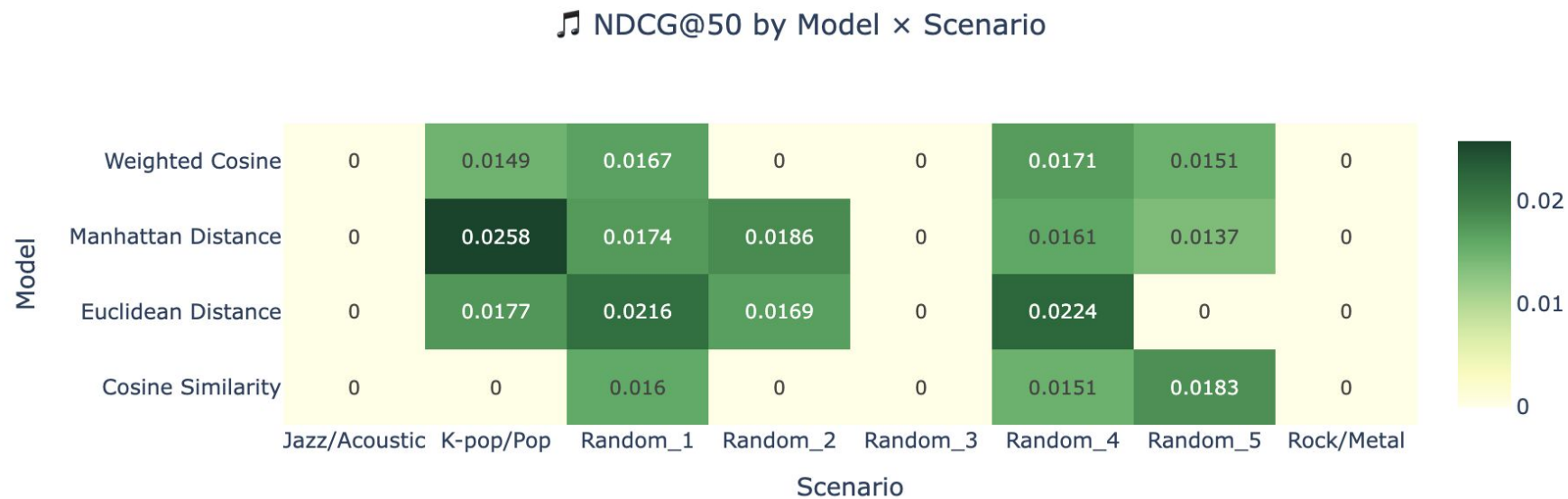
- NDCG 최고 0.011 → 사실상 추천 실패
- Genre Hit 30% — 랜덤(10–20%) 이상이지만 **실용적 수준에 크게 미달**

✓ 17. Weighted Cosine의 교훈

- EDA 기반 가중치를 줬으나 Manhattan에 못 미침
- **"올바른 피쳐 지식도, 잘못된 구조를 보완하지 못한다"**

5.2 모델 학습-시나리오별로 보면 — 더 심각하다

작성자: 김기열



✓ 18. 장르별 성능: 거의 전부 0

- K-pop: Manhattan이 NDCG=**0.026**으로 그나마 최고
- Rock, Jazz: 전 모델 NDCG $\approx$ **0.000** — 완전한 실패
- Random: 간헐적으로 0.01–0.02 → 우연에 가까움

✓ 19. 왜 이렇게 심각한가 — centroid의 구조적 문제

- BTS + Ed Sheeran + Taylor Swift의 **평균 벡터**를 상상해보자
- 각각의 음악적 특성이 다른데, 평균을 내면 **어디에도 속하지 않는 벡터**가 됨

- Pop과 Rock을 둘 다 좋아하는 사람의 "평균 취향"은 **실제로 존재하지 않음**

- 이것이 centroid 방식의 본질적 한계

✓ 20. 이 실패가 중요한 이유

- "안 되는 것을 확인한 것"도 실험의 가치
- **4개 모델이 전멸했다는 사실 자체가 "구조를 바꿔야 한다"는 근거**
- → 다음 질문: "평균 대신, 각각의 곡마다 이웃을 찾으려면?"

5.2 모델 학습-KNN— 평균을 구하지 말고, 각각 물어봐라

작성자: 김기열

```
def recommend_knn(feature_matrix, train_indices, candidate_indices, k,
                  n_neighbors=10):
    """KNN: liked 곡마다 n_neighbors개 이웃을 찾고 투표로 집계"""
    # 후보 피쳐 인덱싱
    candidate_features = feature_matrix[candidate_indices]

    # KNN 모델 학습 (후보 곡 대상)
    knn = NearestNeighbors(n_neighbors=n_neighbors, metric='cosine', algorithm='brute')
    knn.fit(candidate_features)

    # liked 곡별 이웃 찾기
    liked_features = feature_matrix[train_indices]
    distances, local_indices = knn.kneighbors(liked_features)

    # 투표 집계: 가까울수록 높은 점수 (1/distance)
    scores = np.zeros(len(candidate_indices))
    for i in range(len(train_indices)):
        for j in range(n_neighbors):
            local_idx = local_indices[i, j]
            dist = max(distances[i, j], 1e-10)
            scores[local_idx] += 1.0 / dist

    top_k_local = np.argsort(scores)[::-1][:k]
    return candidate_indices[top_k_local]

results_knn = env.run(recommend_knn, 'KNN', k=K)
```

```
KNN 평가 (K=50)...
K-pop/Pop | P@50=0.900 R@50=0.084 NDCG=0.930 Genre=0.900 Div=0.071 [rel=537] (0.139s)
Rock/Metal | P@50=0.260 R@50=0.049 NDCG=0.409 Genre=0.300 Div=0.028 [rel=267] (0.127s)
Jazz/Acoustic | P@50=0.760 R@50=0.567 NDCG=0.831 Genre=0.760 Div=0.088 [rel=67] (0.123s)
Random_1 | P@50=0.400 R@50=0.017 NDCG=0.534 Genre=0.760 Div=0.266 [rel=1146] (0.132s)
Random_2 | P@50=0.100 R@50=0.007 NDCG=0.229 Genre=0.500 Div=0.132 [rel=708] (0.121s)
Random_3 | P@50=0.500 R@50=0.029 NDCG=0.630 Genre=0.660 Div=0.083 [rel=858] (0.120s)
Random_4 | P@50=0.120 R@50=0.003 NDCG=0.184 Genre=0.380 Div=0.161 [rel=1961] (0.063s)
Random_5 | P@50=0.120 R@50=0.007 NDCG=0.238 Genre=0.480 Div=0.126 [rel=856] (0.095s)

AVERAGE | P@50=0.395 R@50=0.095 NDCG=0.498 Genre=0.593 Div=0.119 (0.115s)
```

✓ 21. 발상의 전환

- Baseline: 30곡의 **평균** → 1개 벡터로 전체와 비교
- KNN: 30곡 **각각** → 개별적으로 이웃을 찾음
- BTS곡은 BTS 이웃을 찾고, Taylor Swift곡은 Taylor Swift 이웃을 찾는다
- **"혼합 취향을 평균내지 않고, 각각의 취향을 존중한다"**

✓ 22. 결과

시나리오	Baseline 최고	KNN	배수
K-pop	0.026	0.930	36배
Jazz	≈ 0	0.831	—
Rock	≈ 0	0.409	—
AVG	0.011	0.498	약 43배

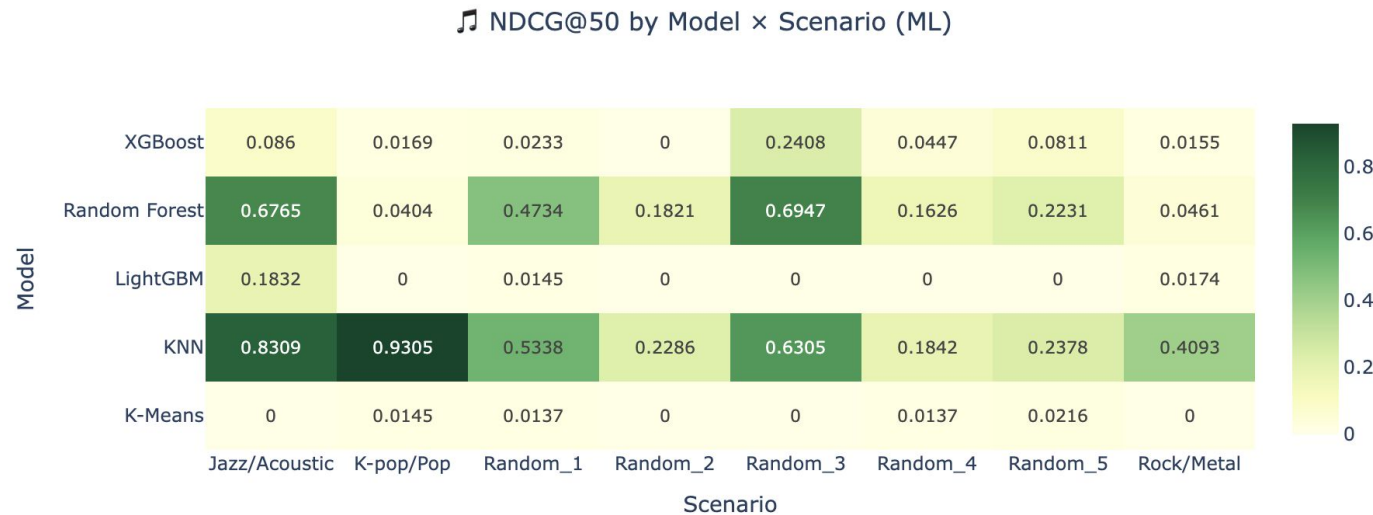
- 구현: cosine 거리 + 1/distance 가중 투표 → 30곡×10이웃 = 300후보 → Top-50

✓ 23. 핵심 메시지

- 알고리즘을 바꾼 게 아니라, **"질문하는 방식"**을 바꿨다
- ⚠ KNN은 평가(Artist-Membership)와 구조적으로 정렬되어 유리한 측면 있음
- → **모델 간 상대 비교로서의 의미에 주목**

5.2 모델 학습-나머지 4개 ML 모델 — 왜 KNN보다 못한가

작성자: 김기열



✓ 24. 결과 요약

모델	NDCG	왜 KNN보다 낮은가
KNN	0.498	—
Random Forest	0.312	180곡 학습 데이터 부족
XGBoost	0.064	180곡으로 과적합
LightGBM	0.027	180곡으로 과적합
K-Means	0.008	다중 취향이 클러스터에 분산

✓ 25. 분류 모델의 근본 문제

- liked 30곡 + 랜덤 150곡 = **180곡으로 학습** (1:5, seed=42)
- 89,740곡 중 150곡만 negative → 전체 분포를 대표하기에 턱없이 부족
- 부스팅(XGB, LGBM)은 **적은 데이터에서 과적합이 치명적**, RF(배깅)는 그나마 나음

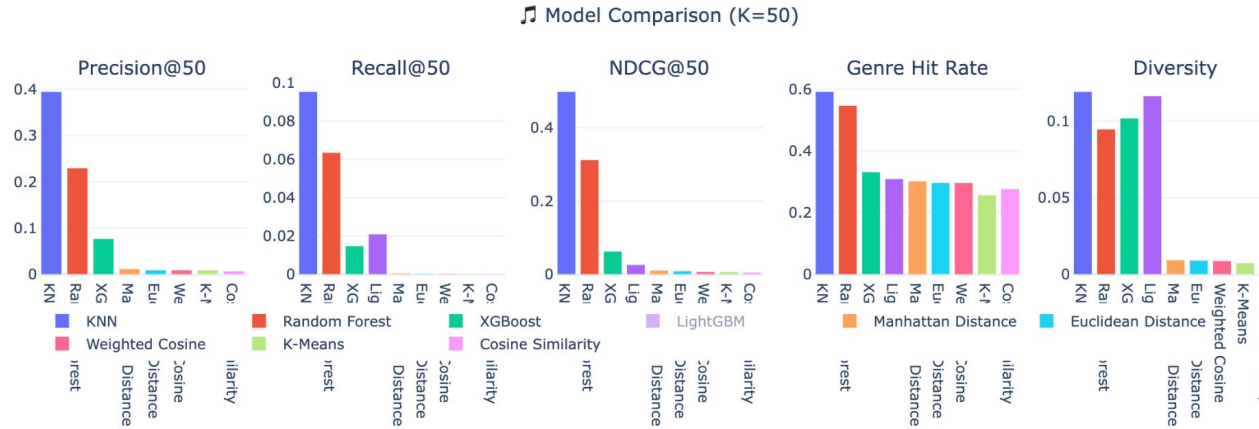
✓ 26. 반직관적 패턴

- KNN(비모수) > RF(배깅) > XGB(부스팅) > LGBM(부스팅)
- **"복잡할수록 성능이 떨어진다"** — 데이터 크기에 맞는 모델 선택이 핵심

5.2 모델 학습-전체 9개 모델 — 한눈에 보기

작성자: 김기열

Baseline vs ML Comparison



✓ 27. 9개 모델 순위

그룹	모델	NDCG
A. 성공	KNN	0.498
B. 부분 성공	Random Forest	0.312
C. 실패	XGB / LGBM / Baseline 4개 / K-Means	0.006–0.064

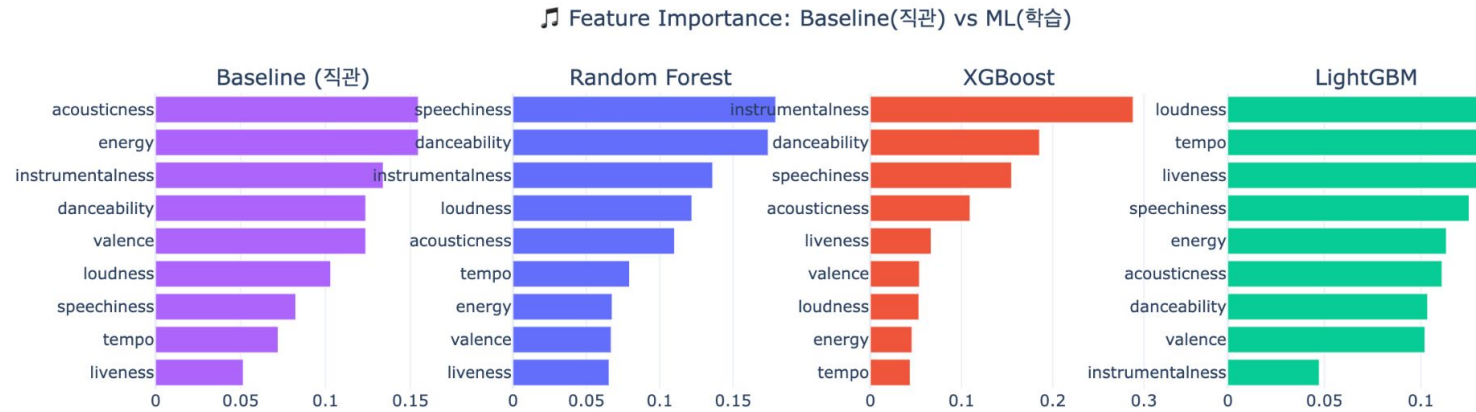
✓ 28. 이 차트가 말하는 3가지

- **구조가 성능을 결정**: centroid(0.011) → 개별 곡(0.498) — 본질적 차이
- **복잡도와 성능이 반비례**: 모델 파라미터가 많을수록 오히려 하락
- **9개 모델의 실패가 KNN 선택의 근거**: 소거법으로 최적 구조 도출

✓ 29. 그러나 KNN도 완벽하지 않다

- P@50=0.395 → **50곡 중 약 20곡**만 취향에 맞춤
- Rock NDCG=0.409 — K-pop(0.930)에 비해 상대적 약점
- **원인: 오디오 피쳐만으로는 장르를 구분할 수 없다**

Feature Importance



✓ 30. EDA 직관 vs ML 학습 (⚠ K-pop/Pop 시나리오 기준)

피쳐	EDA 가중치	ML 학습	일치
energy	1.5 (높음)	상위	✓
acousticness	1.5 (높음)	상위	✓
danceability	1.2 (중간)	상위	✓
liveness	0.5 (낮음)	하위	✓

• 3개 모델(RF, XGB, LGBM) 모두 **비슷한 순위** → 안정적 패턴

• ⚠ 다른 장르에서는 피쳐 중요도 순위가 달라질 수 있음

✓ 31. 그런데 왜 **Weighted Cosine**은 실패했나?

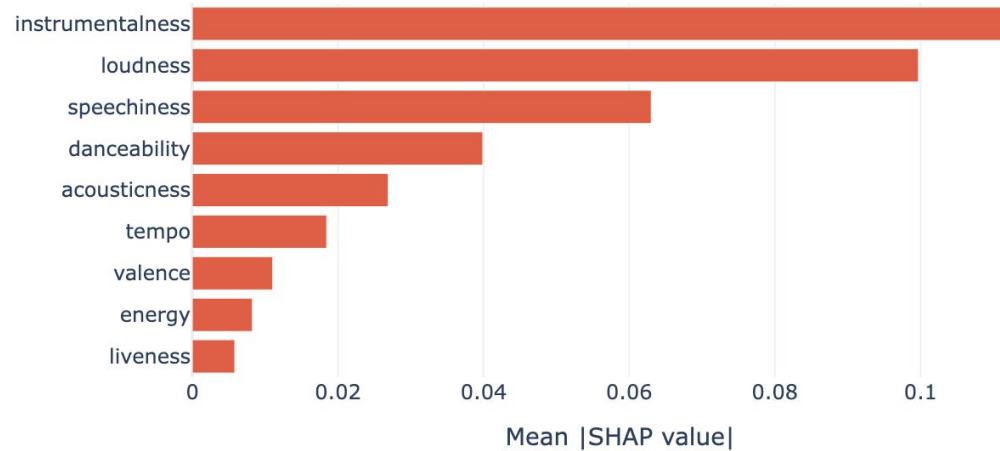
• 직관이 맞았는데 가중치를 줬더니 오히려 Manhattan보다 낮음

• **"어떤 피쳐가 중요한지 아는 것" ≠ "좋은 추천을 하는 것"**

• → 가중치보다 **추천 구조**(centroid vs 개별 곡)가 결정적

SHAP - Feature Contribution

🎵 SHAP — 피처별 평균 기여도 (K-pop/Pop Top 10)



✓ 32. SHAP이란

- 각 추천 곡에 대해 "어떤 피처가 얼마나 기여했는지" 분해(RF 기반, K-pop Top 10곡)

- 빨간색(+) = 추천 확률 높이는 방향 / 파란색(-) = 낮추는 방향

✓ 33. K-pop/Pop 시나리오 해석

- **danceability 높음** → 추천 확률 ↑ (K-pop은 신나는 곡이 많음)

- **acousticness 낮음** → 추천 확률 ↑ (일렉트로닉/댄스 계열)

- **energy 높음** → 추천 확률 ↑

- → "이 곡은 **에너지 높고 댄서블해서** BTS와 유사합니다"

✓ 34. 해석 가능성의 가치

- RF(0.312)는 KNN보다 낮지만, "**왜 이 곡인지**" 설명 가능

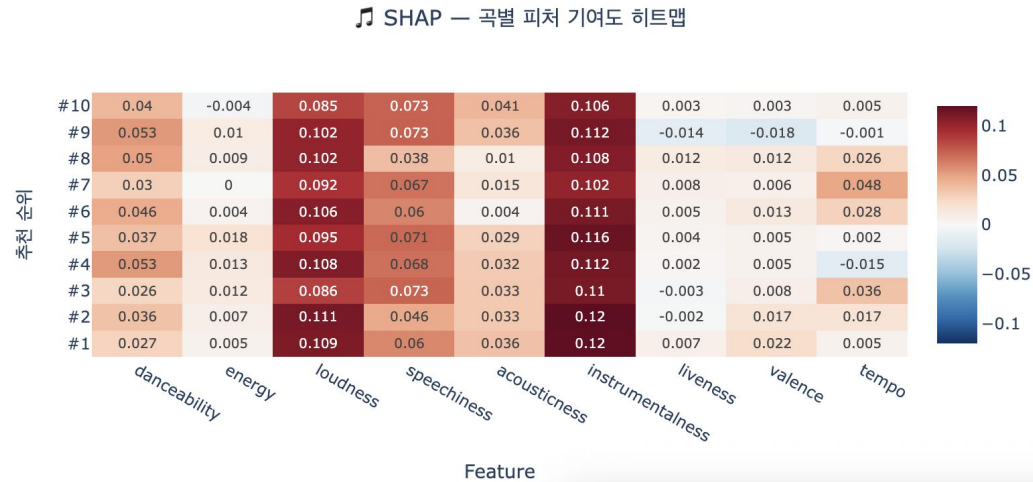
- 블랙박스: "이 곡 들어보세요" vs 설명 가능: "에너지와 댄서빌리티가 비슷해서요"

- 이 해석 가능성을 **더 발전시키면?** → Stage 2 RAG

5.2 모델 학습-최종 정리 — 9개 모델이 말해주는 것

작성자: 김기열

SHAP - Per-Track Heatmap



✓ 35. 성과 요약

메트릭	Baseline (Manhattan)	ML (KNN)	개선
P@50	0.013	0.395	약 31배
NDCG@50	0.011	0.498	약 43배
Genre Hit	0.302	0.593	1.96배

✓ 36. 4가지 교훈

- **추천 구조가 핵심**: centroid → 개별 곡 탐색이 본질적 차이
- **복잡도 ≠ 성능**: KNN > RF > XGB > LGBM
- **직관 가중치의 한계**: EDA가 맞아도 구조가 틀리면 의미 없음
- **실패도 근거**: 8개 모델의 실패가 KNN 선택을 정량적으로 정당화

✓ 37. 남은 한계

- P@50=0.395 → **50곡 중 약 20곡만** 취향에 맞춤
- 오디오 피쳐에는 **아티스트 이름도, 장르도 없다**
- → **숫자에 없는 정보를 어떻게 추가할 수 있을까?**

5. 모델 구조 상세

(5.3 LLM 구현)

- 1) 취향 추천 모델 결과 설명 기능
- 2) 사용자 요청의 RAG 검색 기능

✓ 현재

- 취향 추천 모델: 추천곡 "Ed Sheeran - Perfect", 점수 0.92
- 문제점: 사용자는 0.92라는 숫자의 의미를 모름. "왜 추천했지?" 의문 발생

✓ 개선

- 취향 추천 모델 + LLM: "당신이 좋아하는 잔잔한 어쿠스틱 발라드 스타일이에요. 최근 들은 John Mayer 곡들과 비슷한 감성입니다."
- 해결: 추천의 근거를 자연어로 제공하여 설득력 강화

```
def _interpret_energy(value: float) -> str:
    """에너지 값을 설명으로 변환"""
    if value < 0.3:
        return "잔잔하고 차분한"
    elif value < 0.6:
        return "적당한 에너지의"
    else:
        return "신나고 에너지 넘치는"

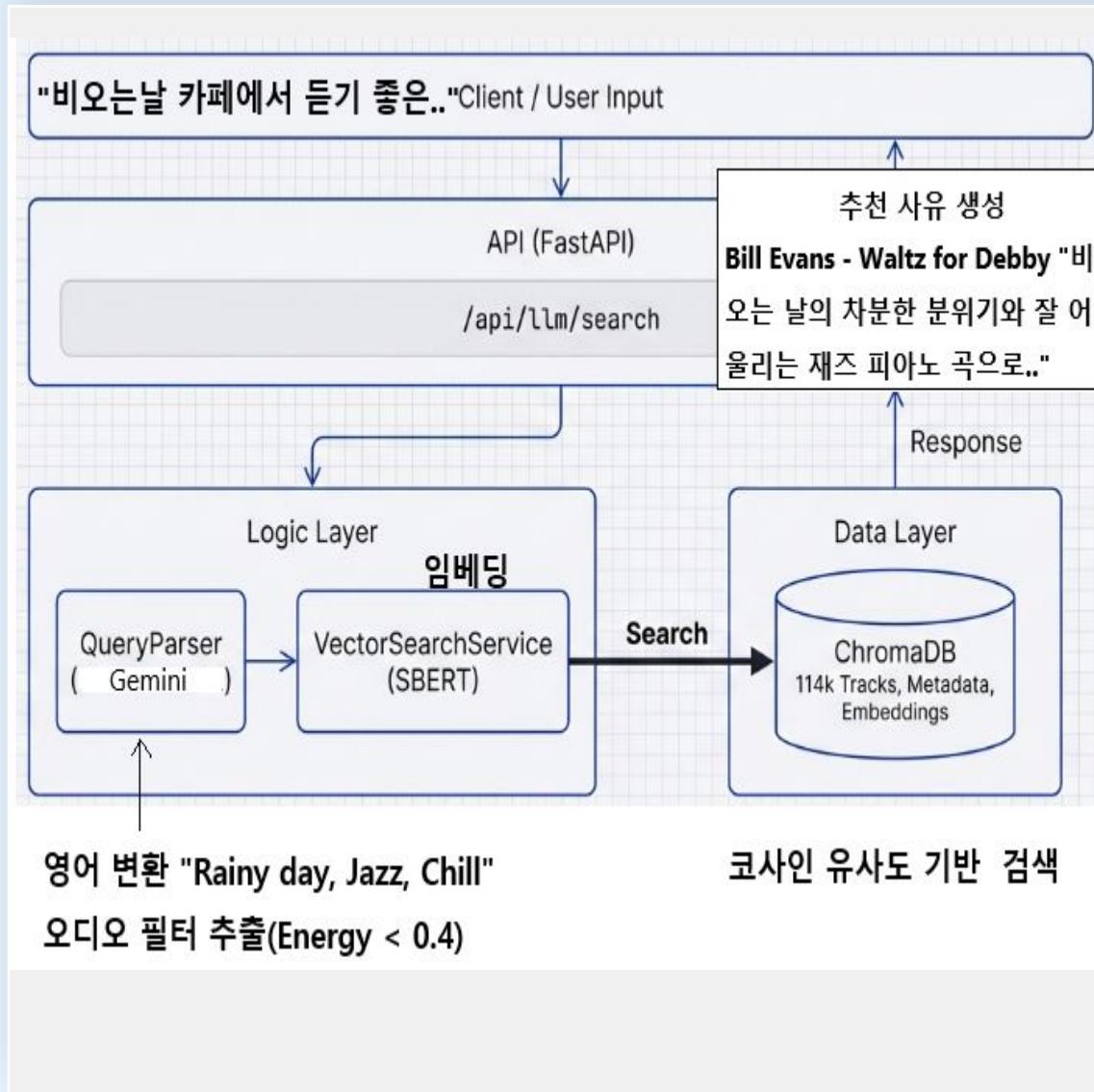
def _interpret_valence(value: float) -> str:
    """발랄함 값을 설명으로 변환"""
    if value < 0.3:
        return "우울하거나 감성적인"
    elif value < 0.6:
        return "중립적인 분위기의"
    else:
        return "밝고 긍정적인"
```

✓ 시스템 구조

- 다이어그램:
 1. 취향추천 모델: 사용자 로그 분석 → 추천곡 선정 (Score 계산).
 2. Data 결합: 추천곡 특징(Audio Features) + 사용자 취향 결합하여 LLM에게 전달
 3. LLM (Gemini): 결합된 데이터를 바탕으로 개인화된 설명 생성.
- 역할 정의:
 - SVM: "좋아할 확률 계산"
 - LLM: "점수의 의미 해석 및 전달"

✓ 데이터 로직: 어떻게 '취향'을 이해하는가?

- 입력 데이터
 - 사용자 취향: 평균 에너지 0.35 (잔잔함 선호), 선호 장르 [Acoustic, Pop], 최근 아티스트 [John Mayer]
 - 추천곡 정보: 곡명 'Perfect', 에너지 0.3, 어쿠스틱 점수 0.8
- 프롬프트 전략:
 - 수치(0.3)를 해석적 표현("잔잔함")으로 매핑하여 LLM에게 제공
 - 근거: 제임스 러셀 이론 ("모든 감정을 energy와 valence의 2차원 평면에 배치할수 있다.")



✓ 기존 방식(Keyword Search)의 한계 인식

- "Jazz" 입력 → Jazz 장르 곡만 출력.
- 개선점 도출: "비 오는 날 카페에서 듣기 좋은 노래" 입력하면, 결과를 보여줄 수는 없을까? (사용자의 감정이나 상황(Context)을 검색을 할 수 있으면 좋겠다!)

✓ 해결 : RAG (Retrieval-Augmented Generation)

- 개념도:
 1. Retrieval (검색): 쿼리(요청) 임베딩하여, Vector DB(ChromaDB)에서 유사한 곡 탐색.
 2. Augmented (증강): 검색된 곡 정보와 사용자 질문을 결합하여 LLM에게 전달할 프롬프트 보강.
 3. Generation (생성): LLM(Gemini)이 최종 추천 및 설명 생성
- 기술 스택:
 - LLM: Google Gemini-2.0-Flash (가성비 및 속도 우수),.
 - Vector DB: ChromaDB (오픈소스, 로컬 구축 용이),.
 - Embedding: Sentence-Transformers (all-MiniLM-L6-v2)

✓ 시스템

1. 쿼리 분석 : 사용자 질문("비 오는 날...") → 영어 변환("Rainy day, Jazz, Chill") + 쿼리에서 추출한 오디오 (Energy < 0.4)
2. Vector Search (SBERT + ChromaDB): 변환된 쿼리 임베딩 → 코사인 유사도 기반 Top-K 검색
3. LLM: 검색 결과 요약 및 추천 사유 생성

5.3 LLM 구현 -2)자연어 음악 검색 기능(임베딩 모델 선정)

작성자: 최두석

유사도 점수

한국어 쿼리	영어 전용	다국어	LLM 번역 + 영어 전용
재즈 음악	0.156	0.664	0.947
록 음악	0.144	0.699	0.845
힙합	0.084	0.712	0.869
슬픈 음악	0.125	0.747	0.831
신나는 음악	0.18	0.516	0.788
잔잔한 음악	0.22	0.469	0.845
비오는 날 듣는 음악	0.131	0.603	0.784
카페에서 듣는 음악	0.15	0.806	0.806
운동할 때 듣는 음악	0.118	0.44	0.814
비오는 날 카페 잔잔한 재즈	0.005	0.703	0.839
우울할 때 감성 발라드	0.128	0.604	0.817
평균 (Average)	0.131	0.633	0.835

구분	모델명 (Model Name)
영어 전용 (English Only)	all-MiniLM-L6-v2
다국어 (Multilingual)	paraphrase-multilingual-MiniLM-L12-v2
한국어→영어 번역	`gemini-2.0-flash`

✓ 크로마DB는 영어로 작성된 114k곡을 임베딩한 결과

✓ 영어 전용 모델은 **Unknown Token**만 반환

1. 영어 모델 : "비오는 날 카페..." → [?, ?, ?, ...] (의미 없는 숫자)
2. 크로마DB : "jazz, lounge..." → [0.5, 0.3, 0.2, ...] (영어 벡터)
 - 유사도 0.005 (한국어 쿼리를 임베딩하기 힘들다.)

✓ 다국어 모델

1. 다국어 모델 : "비오는 날 카페..." → [0.4, 0.2, 0.3, ...] (한국어 벡터)
2. 크로마DB : "jazz, lounge..." → [0.5, 0.3, 0.2, ...] (영어 벡터)
 - 유사도 0.703 (다국어 모델이 한국어와 영어를 비슷한 공간에 배치)

✓ LLM 번역 + 영어 모델

1. "비오는 날 카페..." → LLM 번역 → "Rainy day cafe chill jazz"
2. 영어 모델 : "Rainy day cafe..." → [0.48, 0.32, 0.22, ...] (영어 벡터)
3. 크로마DB : "jazz, lounge..." → [0.5, 0.3, 0.2, ...] (영어 벡터)
 - 유사도 **0.839** (같은 영어 모델이 같은 영어를 처리하므로 정확도가 상승)

해결 : 'LLM을 통한 쿼리 확장(Query Expansion)'을 전처리 단계에 도입하여 언어 장벽 해결

5.3 LLM 구현 -2)자연어 음악 검색 기능(도전, 향후 개선, 이론적 근거)

작성자: 최두석

✓ 고민-LLM 2번 사용하면 응답 속도 느려지지 않을까?

- 사용자 요청(쿼리)를 다국어 모델로 변환 후에 LLM에 전달을 한다면, LLM 1번 호출
- LLM으로 영어 번역 후(크로마DB 저장된 데이터 원문이 영어), 영어 모델로 임베딩 후, LLM에게 전달한다면 LLM 2번 호출.
- LLM 1번 호출에 0.5~1.5 초 정도 소요됨을 생각할 때, 사용자가 서비스에 대해서 불편함을 느낄것에 대한 고민
- “속도냐 정확도냐?” - 멍청한 답변을 내놓는 빠른 검색보다는, 1초정도 더 걸리더라도 정확한 결과를 제공하는 것이 낫다고 판단 -> 다국어모델과 비교시, 앞서 제공된 유사도 점수기반 정확도 개선율은 31.9%

곡 수별 사용자 대기 시간 및 경험 분석

곡 수	예상 소요 시간	사용자 체감 및 경험	비고
3곡	~1.5초	매우 빠름- 거의 즉시 응답	-
5곡	~2.0초	적당함	선택
10곡	~3.0초	약간 느림- 기다림을 인지하기 시작	-
20곡	~5.0초	느림- 서비스 이탈 가능성 높음	-

✓ 고민 - 자연어 검색시 결과를 몇곡씩 보여주어야 하나?

- 결정 : 5곡
- 5곡 결과 보여주기 위한 2초는 사용자가 "AI가 생각 중" 이라고 인식하는 허용 범위로 판단
- 5곡이면 충분한 선택지 제공
- 더 보고 싶으면 "더 보기" 버튼으로 5곡 추가 로드

✓ 향후 개선 계획

- 114K곡이 저장된 크로마DB에서 검색을 하는 방식 입니다. 그러나 새로운 노래가 주입되더라도, 새로운 곡은 검색이 되지 않습니다. 새로운 곡은 매일밤 주기적으로 크로마 DB에 저장되도록 구현할 예정입니다.

● 향후 계획 : 주기적 동기화 (배치)

매일 밤 실행

```
def sync_chromadb():
```

```
    # PMS에는 있지만 ChromaDB에 없는 곡 찾기
```

```
    new_songs = get_songs_not_in_chromadb()
```

```
    # 일괄 임베딩 후 ChromaDB 저장
```

```
    for song in new_songs:
```

```
        embedding = embedding_model.encode(song["tags"])
```

```
        chroma_collection.add(...)
```

- 1)추천 결과를 SHAP/RAG로 왜 이 곡인지 설명
- 2)사용자 질문에 실제 데이터 기반으로 답변

✓ 38. Stage 1 결론에서 이어진다

● KNN은 NDCG 0.498이지만, 오디오 피처에 **장르/아티스트 정보가 없음**

● 형식: "아티스트 - 곡명 [장르]" → Sentence-BERT(MiniLM, 22M, 384d)로 벡터화

✓ 39. 두 모달리티는 정반대 패턴

시나리오	KNN (오디오)	Text (텍스트)
K-pop	0.930	1.000
Rock	0.409	1.000
Jazz	0.831	1.000
Random 평균	0.363	0.045

✓ 40. 텍스트 임베딩으로 해결

● 형식: "아티스트 - 곡명 [장르1, 장르2]" → Sentence-BERT(MiniLM, 22M, 384d)로 벡터화
 ● 장르 시나리오: 텍스트 **완벽(1.000)**, KNN은 Rock에서 0.409로 취약

● Random 시나리오: 텍스트 **사실상 실패(0.045)**, KNN이 압도(0.363)

● 두 모달리티가 **정반대 패턴** → 상보적 결합의 근거

✓ 41. ⚠ Proxy Metric 주의

● 장르 시나리오 Text 1.000 = 아티스트명이 텍스트에 **직접 포함** → 평가와 정렬

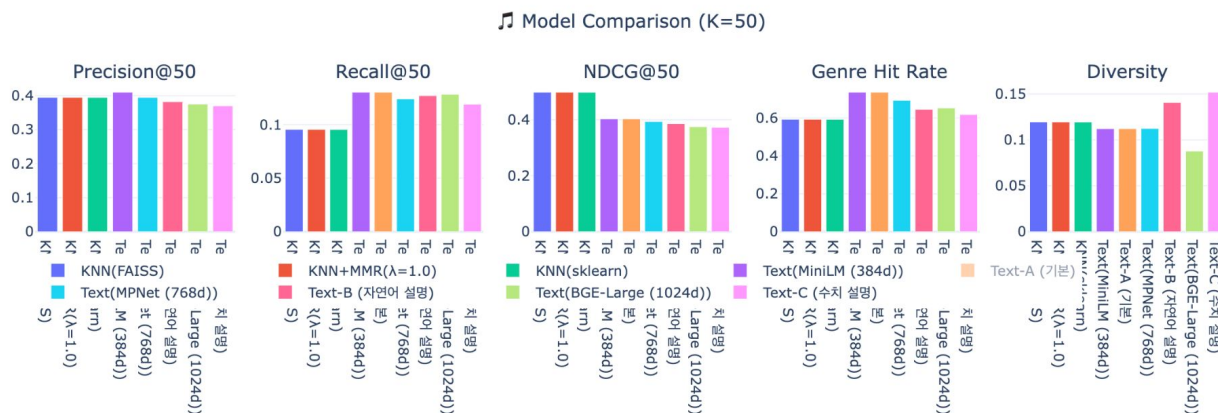
● 실제 추천 서비스(Spotify, YouTube)에서도 아티스트/장르는 핵심 시그널이므로 "부정행위"는 아님

● **진짜 테스트는 Random** → 텍스트 centroid는 산발적 취향에서 실패

5.3 LLM 구현 - 크다고 좋은 게 아니다 — 모델 선택과 텍스트 구성

작성자: 김기열

Model Comparison



42. 임베딩 모델 — 파라미터 15배 키워도 NDCG 7% 하락

모델	파라미터	차원	NDCG
MiniLM	22M	384d	0.403
MPNet	109M	768d	0.394
BGE-Large	335M	1024d	0.375

43. 텍스트 구성 — 정보를 추가할수록 하락

형식	NDCG
A: BTS - Dynamite [k-pop, pop]	0.403
B: + high energy, very danceable	0.386
C: + energy:0.79 tempo:120	0.373

- SBERT는 음악 도메인 특화가 아님 → 오디오 설명이 노이즈로 작용

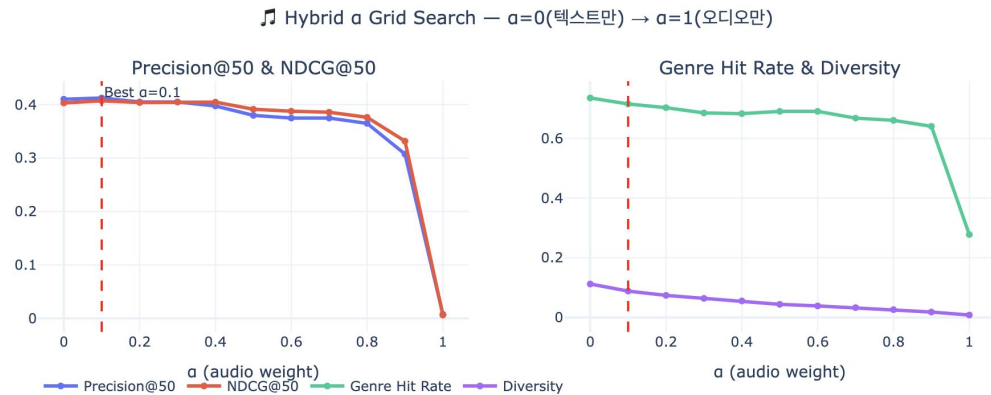
- 빠른 피쳐 엔지니어링이 넣는 것보다 중요할 수 있다

44. 핵심 교훈

- 텍스트는 텍스트답게(메타데이터), 오디오는 오디오답게(수치)

- → 스코어 레벨에서 결합하라

Alpha Grid Search



✓ 45. Hybrid 설계

- $\text{score} = \alpha \times \text{cosine}(\text{audio_centroid}) + (1-\alpha) \times \text{cosine}(\text{text_centroid})$
- 양쪽 모두 **centroid 기반** — 30곡의 평균 벡터로 비교

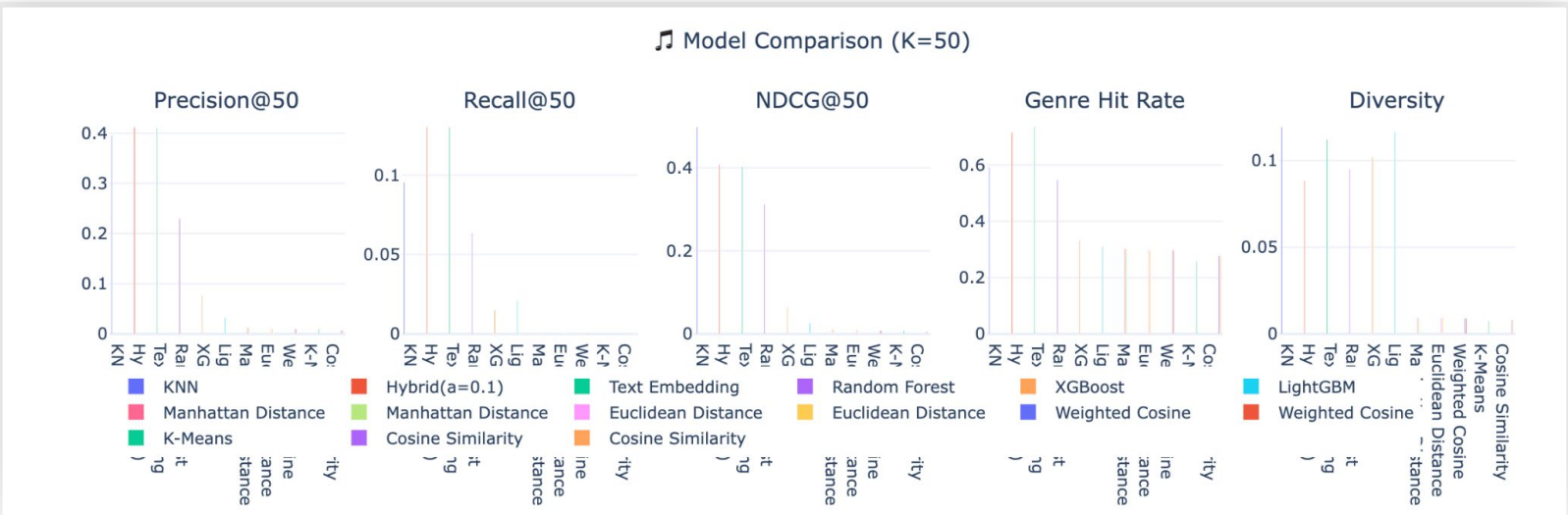
✓ 46. 결과: KNN(0.498)을 못 넘음

α	NDCG	해석
0.1(최적)	0.407	텍스트 비중 90%
0.5	0.391	오디오 비중 커질수록 하락
1.0	0.006	centroid 코사인 = Baseline급

- α 가 커질수록 하락 → **오디오 centroid가 발목**

✓ 47. 전환점

- Stage 1에서 실패한 centroid 구조를 **그대로 반복**한 결과
- 텍스트를 추가하는 것만으로는 부족 → **오디오 스코어링 구조도 바꿔야 한다**
- → "KNN 스코어에 텍스트를 결합하면?"



✓ 48. 11개 모델 순위

순위	모델	NDCG	구조
1위	KNN	0.498	개별 곡 탐색
2위	Hybrid($\alpha=0.1$)	0.407	centroid×2
3위	Text	0.403	centroid
4위	Random Forest	0.312	분류
하위	XGB/LGBM/Baseline	0.006–0.064	—

✓ 49. 패턴: centroid 구조 자체가 병목

- Baseline(오디오 centroid) → 0.011
- Text(텍스트 centroid) → 0.403
- Hybrid(둘 다 centroid) → 0.407
- **KNN(개별 곡 탐색) → 0.498** ← 항상 centroid보다 위

✓ 50. 핵심 전환

- "텍스트 신호는 강력(0.403). KNN 구조도 강력(0.498)."
- 오디오만 **centroid** → **KNN으로 교체**, 텍스트는 centroid 유지 → Ensemble의 설계

✓ 51. Ensemble vs Hybrid — 차이는 딱 하나

	Hybrid (0.407)	Ensemble (0.571)
오디오	centroid 코사인	KNN 스코어
텍스트	centroid 코사인	centroid 코사인 (동일)

✓ 52. α 그리드서치 — Plateau 발견

α	NDCG	해석
0.0 (Text only)	0.408	텍스트만
0.2–0.3	약 0.52–0.57	급격한 상승
0.4–0.9	0.571	Plateau
1.0 (KNN only)	0.498	텍스트 없음

- Baseline 대비 약 50배, KNN 대비 +14.6%

✓ 53. Plateau의 의미

- $\alpha=0.4$ 든 0.9든 결과가 똑같다 → α 튜닝에 신경 안 써도 됨
- 오디오와 텍스트가 서로 다른 정보를 주고 있다는 증거

- ⚠ $\alpha=0.4$ 만 CSV 검증. 0.5–0.9는 런타임 결과

노트 → Hybrid는 오디오 비중(α)을 높이면 성능이 급락했는데, Ensemble은 $\alpha=0.4\sim0.9$ 전부 동일.

- 이유 → KNN 스코어가 워낙 정보량이 풍부해서 비중이 30%만 넘으면 충분히 기여함
- 반대로 $\alpha<0.3$ 이면 KNN 기여가 너무 적어서 급락.
- Plateau가 생기는 메커니즘: α 를 바꿔도 최종 Top-50곡 순위가 안 뒤집힘
→ 같은 50곡이 선택 → 같은 NDCG.

✓ 54. 2단계 파이프라인

- 1단계: $\alpha=0.4$ 로 앙상블 → 상위 200곡 후보 풀
- 2단계: MMR로 다양성 있게 50곡 선택
- $MMR = \lambda \times \text{앙상블점수} - (1-\lambda) \times \text{기선택 곡과의 유사도}$

✓ 55. 결과: NDCG 거의 하락 없이 다양성 확보

λ	NDCG	Diversity	변화
1.0	0.571	0.108	순수 정확도
0.7	0.570	0.115	NDCG -0.2%, Div +7%
0.5	0.566	0.129	NDCG -0.8%, Div +20%

• 사실상 공짜로 다양성을 얻는 구조

- Ensemble은 순수 상태에서 KNN보다 Diversity가 낮으므로(0.108 vs 0.119) → **MMR 적용이 앙상블의 필수 보완책**

✓ 56. ⚠ 앙상블의 트레이드오프 정리

- 정확도: Ensemble(0.571) > KNN(0.498) → **+14.6%** ✓
- 다양성: Ensemble(0.108) < KNN(0.119) → **-10%** ✗
- MMR($\lambda=0.7$) 적용 후: NDCG 0.570 + Diversity 0.115 → **양쪽 다 KNN 이상** ✓

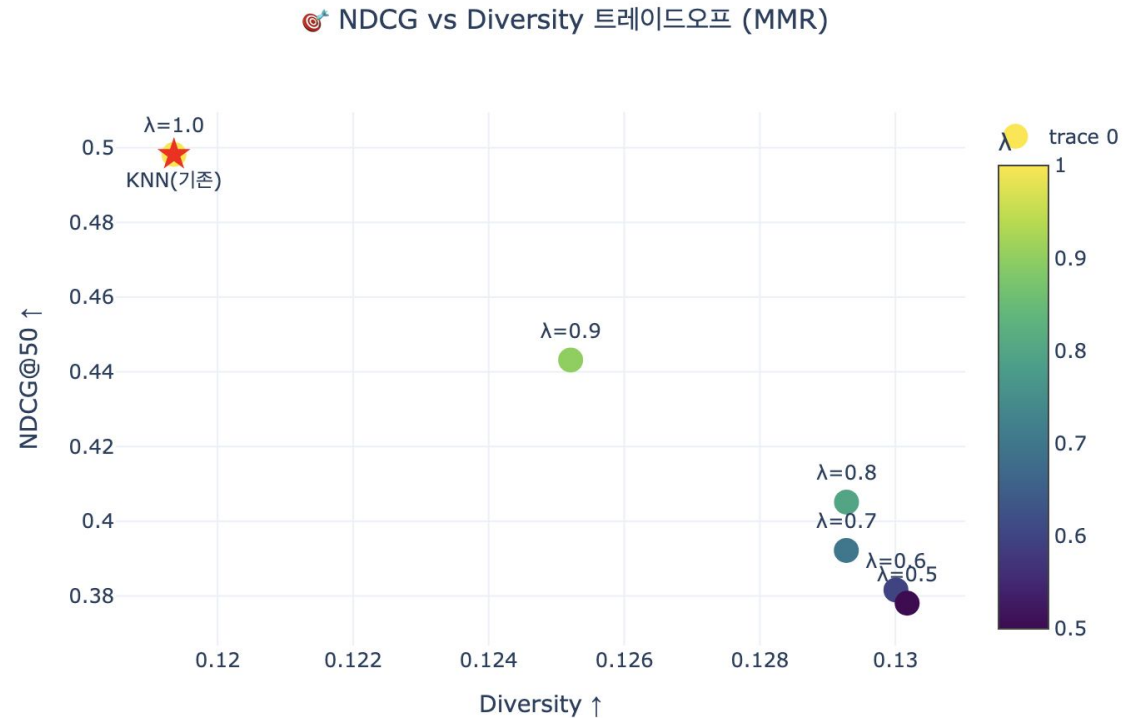
노트

- **Diversity 역전이 발생하는 이유**: 텍스트에 아티스트명이 포함되어 같은 아티스트 곡이 높은 점수를 받음
- **다양성 하락** → 이것이 **MMR 적용의 직접적 동기**이고, $n_{\text{pool}}=200$ 은 $K=50$ 의 4배 휴리스틱, 풀 크기 그리드서치 미수행

5.3 LLM 구현-양상블의 진짜 가치 — 안정성

작성자: 김기열

MMR Tradeoff



✓ 57. 결과: NDCG 거의 하락 없이 다양성 확보

λ	KNN NDCG	하락률	Ensemble NDCG	하락률
1.0	0.498	—	0.571	—
0.8	0.405	-18.7%	0.570	-0.1%
0.5	0.378	-24.1%	0.566	-0.8%

✓ 58. 왜 이런 차이가 나는가

- KNN: 오디오 유사성 **하나의 축** → MMR이 핵심 이웃을 밀어냄 → 급락
- 앙상블: **오디오+텍스트 두 축** → 하나가 밀려도 다른 축이 보완
- 멀티모달 앙상블의 실질적 가치: 정확도가 아니라 **강건성 (robustness)**

✓ 59. 실서비스 관점

- 실서비스에서 MMR은 필수 (동일 아티스트 도배 방지)
- KNN 단독이면 MMR 적용 시 성능 붕괴 — 서비스 불가
- 앙상블은 MMR 적용해도 성능 유지 — **서비스 가능한 유일한 구조**

✓ 60. 모달리티별 강약 패턴

시나리오 유형	Text	KNN	최강
장르 (K-pop/Rock/Jazz)	1.000	0.409–0.930	Text
Random (5개)	0.000–0.142	0.184–0.630	KNN

✓ 61. 성능을 결정하는 요인: 취향 응집도 (cohesion)

- **K-pop** 시나리오: liked곡 전부 K-pop → 높은 응집도 → 모든 모델 강함
- Random_4: 장르 산발적 → 낮은 응집도 → **양쪽 모달리티 모두 최저**
- 응집도가 낮으면 어떤 모델도 어려움 — **태스크 자체의 한계**

✓ 62. 실패의 원인을 구분한다

- 모델의 한계: 오디오로 장르 구분 불가 → 텍스트로 보완 (해결)
- 데이터의 한계: relevant set 극소 (Jazz: 67곡) → 더 큰 데이터셋 필요
- 태스크의 한계: 산발적 취향 → 본질적으로 어려움
- **"모든 실패를 모델 탓으로 돌리지 않는다"** — 이것이 정교한 에러 분석

노트

- 실패 원인을 모델 / 데이터 / 태스크로 구분하는 것이 핵심
- Text가 장르 시나리오에서 1.000인 것은 아티스트명 포함 효과이지만, **Random에서 0.045로 떨어지는 것이 centroid 구조의 한계**를 보여줌
- **왜 실패했는가**를 정확히 짚어야 **어떻게 개선할 수 있는가**가 나온다.

✓ 63. RAG 파이프라인

- **R**: FAISS로 추천곡 ↔ liked_songs 유사곡 Top-3 검색
- **A**: 오디오 피쳐 9차원 + 장르 + 유사도 점수로 프롬프트 구성(약 700자 → 3,400자+)
- **G**: Gemini 2.5 Flash가 데이터 기반 자연어 설명 생성
- ⚠ 현재 **KNN 추천 결과에 적용** — 추천 모델 고정, 설명 방식만 비교하는 실험 설계

✓ 64. 결과: LLM 환각 제거

항목	기본 Gemini	RAG Gemini
수치 인용	0개	24개
설명 근거	LLM 자체 지식	실제 피쳐 데이터
예시	몽환적인 분위기	피쳐 수치 기반 비교

- SHAP(Stage 1) = 피쳐 레벨 설명 / RAG = **사용자 레벨 설명**

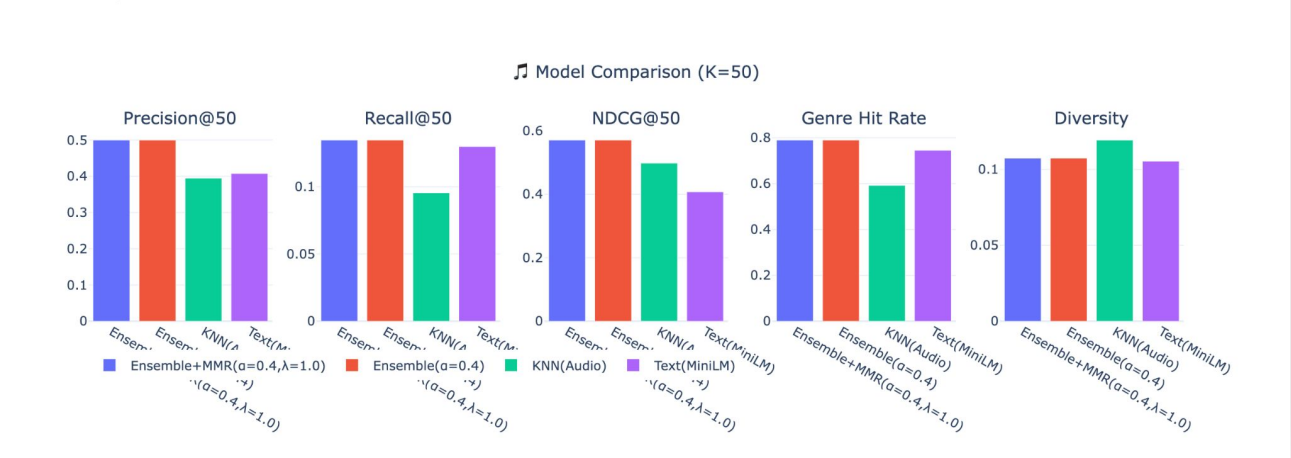
✓ 65. 한계

- 현재 KNN 추천 결과에만 적용 — Ensemble 적용은 향후 과제
- 중복 트랙 미제거 → RAG 설명에서도 동일 곡 반복 발생
- ⚠ 추천 모델 고정, **설명 방식만 비교**하는 실험 설계

노트

- 프롬프트 약 700자 → **RAG 적용 시 3,400자+로 확장**
- KNN 출력에만 적용한 이유는 **실험 설계상 추천 모델을 고정하고 설명 방식만 비교**하기 위함
- Ensemble 출력 적용 시 **텍스트 유사도**도 활용 가능.

Final Comparison



✓ 66. 전체 성과

메트릭	Baseline	KNN	Ensemble
NDCG@50	0.011	0.498	0.571 (약 50배)
P@50	0.013	0.395	0.500
Genre Hit	0.302	0.593	0.790

● KNN→Ensemble +14.6% / MMR 안정성 약 29배

✓ 67. 5가지 핵심 메시지

- 추천 구조가 성능을 결정: centroid→KNN = 43배, +텍스트 = 50배
- 크다고 좋은 게 아니다: 22M > 335M / 기본 텍스트 > 상세 텍스트
- 복잡도 ≠ 성능: KNN > RF > XGB > LGBM
- 앙상블의 진짜 가치는 안정성: MMR 시 KNN -24%, 앙상블 -0.8%
- 실패 분석이 성공보다 설득력 있다: 취향 응집도가 성능을 예측

✓ 68. 20개 모델 체계

Stage	수	모델
1	9개	Cos, Euc, Man, W.Cos, KNN , K-Means, RF, XGB, LGBM
2	9개	Text×3, Text-B, Text-C, Hybrid, Ensemble, KNN+MMR, Ens+MMR
3	2개	RAG(Basic), RAG(FAISS+Gemini)

6. 결과 및 정리

6.1 프로젝트 성과

혁신적인 음악 추천 시스템 구축

1

3단계 필터링 시스템

EMS → GMS → PMS 구조화된 검증 프로세스
수백곡 → 소수 후보 → 개인 라이브러리 자동 정제

2

AI + Human 하이브리드

AI의 자동 점수화와 사용자의 최종 승인을 결합하여 추천 정확도와 만족도를 동시에 확보

3

지속 학습 (Continuous Learning)

사용자 피드백으로 모델을 재학습할수록
더욱 정교해지는 개인화

4

의외성 추천 (Serendipity)

필터 버블(Filter Bubble) 방지 적용
취향을 벗어나지 않으면서 새로운 음악 발견의 기회 제공

5

LLM 기반 설명 (XAI)

"이 곡을 추천한 이유"를 투명하게 설명
설득력 있는 근거 제시로 AI 추천 신뢰도 대폭 향상

6.2 개선점 및 향후 발전 방향

| 모델 성능 개선

- 1 중복 트랙 처리 (RAG 반복 설명 문제 해결)
- 2 데이터셋 확장 (Jazz 67곡 등 소수 장르)
- 3 텍스트 임베딩 고도화 (다국어 지원 등)

| 시스템 아키텍처 개선

- 1 실시간 학습 자동화 강화
- 2 FAISS 최적화 및 캐싱 전략
- 3 RAG를 Ensemble 모델에도 적용

| 서비스 기능 확장

- 1 협업 필터링 통합
- 2 의외성 주입 알고리즘 개선
- 3 감정 기반 추천 기능

| 평가 지표 개선

- 1 실제 사용자 만족도 지표 추가
- 2 Random 시나리오 대응 전략

| 비즈니스 확장성

- 1 플랫폼 확장 (Apple Music, Melon 등)
- 2 수익 모델 다각화 (B2B, 프리미엄)
- 3 모바일 네이티브 앱 개발

6.3 프로젝트 소감

이름	소감
이성훈	팀장으로서 부족한 점이 많았음에도, 묵묵히 제 자리를 채워준 팀원들 덕분에 프로젝트를 완주할 수 있었습니다. 제가 방향을 고민하는 동안 각자의 위치에서 최고의 결과물을 만들어준 여러분의 노력이 이번 프로젝트의 가장 큰 동력이었습니다. 함께 호흡하며 성장할 수 있어 진심으로 영광이었습니다. 모두 고생 많으셨습니다.
정혜원	PL로서 처음 팀을 이끌면서 시행착오도 많았지만, 각자 맡은 역할에서 최선을 다해주신 팀원분들 덕분에 프로젝트를 성공적으로 마무리할 수 있었습니다. 특히 어려운 순간마다 조언과 아이디어를 나눠주셔서 저도 많이 배울 수 있었고, 이 과정을 통해 협업의 가치를 다시 한번 느꼈습니다. 모두 감사합니다.
최두석	이번 프로젝트를 통해 SVM, LLM, 벡터 검색 등 다양한 AI 기술을 실제 서비스에 적용하는 경험을 할 수 있었습니다. 이론을 배울 때보다 실제로 서비스에 적용하면서 더 많은 것을 이해하고, 알게된 것 같습니다. 팀원 분들이 각자 갖고 있는 장점이 다르다 보니, 다른 팀원들에게서도 배우는 점이 있어서 좋았습니다.
조우성	2개월 정도 정말 정신 없이 달려왔습니다. 처음 교실에 들어왔을때 서먹함과 두려움이 이제는 마지막 프로젝트가 되어 열매 맺으려 하고 있습니다. 힘들었지만 최선을 다한 좋은 경험 좋은 추억이 될만한 시간이었고 이 경험과 기술을 바탕으로 팀원 모두 좋은 곳으로 취업해 개발자로서 꽃길만 걸었으면 좋겠습니다.
김기열	20개 모델을 하나씩 실험하면서, 성능이 오르지 않을 때 "왜 안 되는가"를 분석하고 다음 가설을 세우는 과정이 가장 많이 성장한 부분입니다. 특히 centroid 구조의 한계를 발견하고 KNN 스코어로 교체해 +40% 개선을 이끌어낸 경험이, 단순히 모델을 돌리는 것과 구조를 이해하고 설계하는 것의 차이를 체감하게 해줬습니다.

감사합니다.

AI기반 3단계 음악 검증 시스템_별첨

테이블 정의서
업무 흐름도 상세
단위 테스트
통합 테스트



테이블 정의서

테이블 정의서 (상세)

테이블 이름 : users

컬럼ID	컬럼명	데이터타입(크기)	NULL	기본값	PK	FK
user_id	사용자 고유 ID	bigint(20)	N		Y	
email	이메일	varchar(255)	N			
password_hash	비밀번호 해시	varchar(255)	N			
nickname	사용자 닉네임	varchar(100)	N			
role	사용자 권한	enum('USER','ADMIN')	N	USER		
created_at	가입일시	datetime	Y	CURRENT_TIMESTAMP		
updated_at	수정일시	datetime	Y	CURRENT_TIMESTAMP		

테이블 이름 : user_platforms

컬럼ID	컬럼명	데이터타입(크기)	NULL	기본값	PK	FK
platform_id	플랫폼 연동 ID	bigint(20)	N		Y	
user_id	사용자 ID	bigint(20)	N			Y
platform_name	플랫폼 명	enum('Tidal',...)	N			
access_token	엑세스 토큰	text	N			
refresh_token	리프레시 토큰	text	N			
connected_at	연동일시	datetime	Y	CURRENT_TIMESTAMP		

테이블 정의서 (상세)

테이블 이름 : playlist_tracks

컬럼ID	컬럼명	데이터타입(크기)	NULL	기본값	PK	FK
map_id	매핑 ID	bigint(20)	N		Y	
playlist_id	플레이리스트 ID	bigint(20)	N			Y
track_id	트랙 ID	bigint(20)	N			Y
order_index	정렬 순서	int(11)	N	0		
added_at	추가 일시	datetime	Y	CURRENT_TIMESTAMP		

테이블 이름 : playlist_scored_id

컬럼ID	컬럼명	데이터타입(크기)	NULL	기본값	PK	FK
playlist_id	플레이리스트 고유 ID	bigint(20)	N		Y	Y
user_id	사용자 ID	bigint(20)	N		Y	Y
ai_score	AI 추천 점수	decimal(5,2)	Y	0.00		
created_at	생성일시	datetime	Y	CURRENT_TIMESTAMP		
updated_at	수정일시	datetime	Y	CURRENT_TIMESTAMP		

테이블 정의서 (상세)

테이블 이름 : ai_analysis_logs

컬럼ID	컬럼명	데이터타입(크기)	NULL	기본값	PK	FK
log_id	로그 ID	bigint(20)	N		Y	
user_id	사용자 ID	bigint(20)	N			Y
target_type	분석 대상 타입	enum('Playlist','Track')	N			
target_id	대상 식별 ID	bigint(20)	N			
score	분석 점수	decimal(5,2)	Y			
analysis_result	분석 결과 상세	longtext (JSON)	Y			
analyzed_at	분석 일시	datetime	Y	CURRENT_TIMESTAMP		

테이블 이름 : track_scored_id

컬럼ID	컬럼명	데이터타입(크기)	NULL	기본값	PK	FK
track_id	트랙 ID	bigint(20)	N		Y	Y
user_id	사용자 ID	bigint(20)	N		Y	Y
ai_score	AI 추천 점수	decimal(5,2)	Y	0.00		
created_at	생성일시	datetime	Y	CURRENT_TIMESTAMP		
updated_at	수정일시	datetime	Y	CURRENT_TIMESTAMP		

테이블 정의서 (상세)

테이블 이름 : playlists

컬럼ID	컬럼명	데이터타입(크기)	NULL	기본값	PK	FK
playlist_id	플레이리스트 고유 ID	bigint(20)	N		Y	
user_id	사용자 ID	bigint(20)	N			Y
title	제목	varchar(200)	N			
description	설명	text	Y			
space_type	공간 타입	enum('PMS','EMS','GMS')	N	EMS		
status_flag	상태 플래그	enum('PTP','PRP','PFP')	N	PTP		
source_type	출처 타입	enum('Platform',...)	N	Platform		
external_id	외부 ID	varchar(255)	Y			
cover_image	커버 이미지 URL	varchar(500)	Y			
created_at	생성일시	datetime	Y	CURRENT_TIMESTAMP		
updated_at	수정일시	datetime	Y	CURRENT_TIMESTAMP		

테이블 정의서 (상세)

테이블 이름 : tracks

컬럼ID	컬럼명	데이터타입(크기)	NULL	기본값	PK	FK
track_id	트랙 고유 ID	bigint(20)	N		Y	
title	곡 제목	varchar(255)	N			
artist	아티스트	varchar(255)	N			
album	앨범명	varchar(255)	Y			
duration	재생 시간(초)	int(11)	N			
isrc	국제 표준 코드	varchar(50)	N			
external_metadata	메타데이터	longtext (JSON)	N			
created_at	등록일시	datetime	Y	CURRENT_TIMESTAMP		

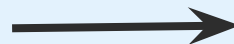
업무 흐름도 상세

기호(Symbol) 설명

시작 이벤트



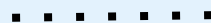
프로세스
진행방향



종료 이벤트



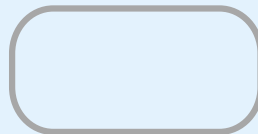
연계
프로세스



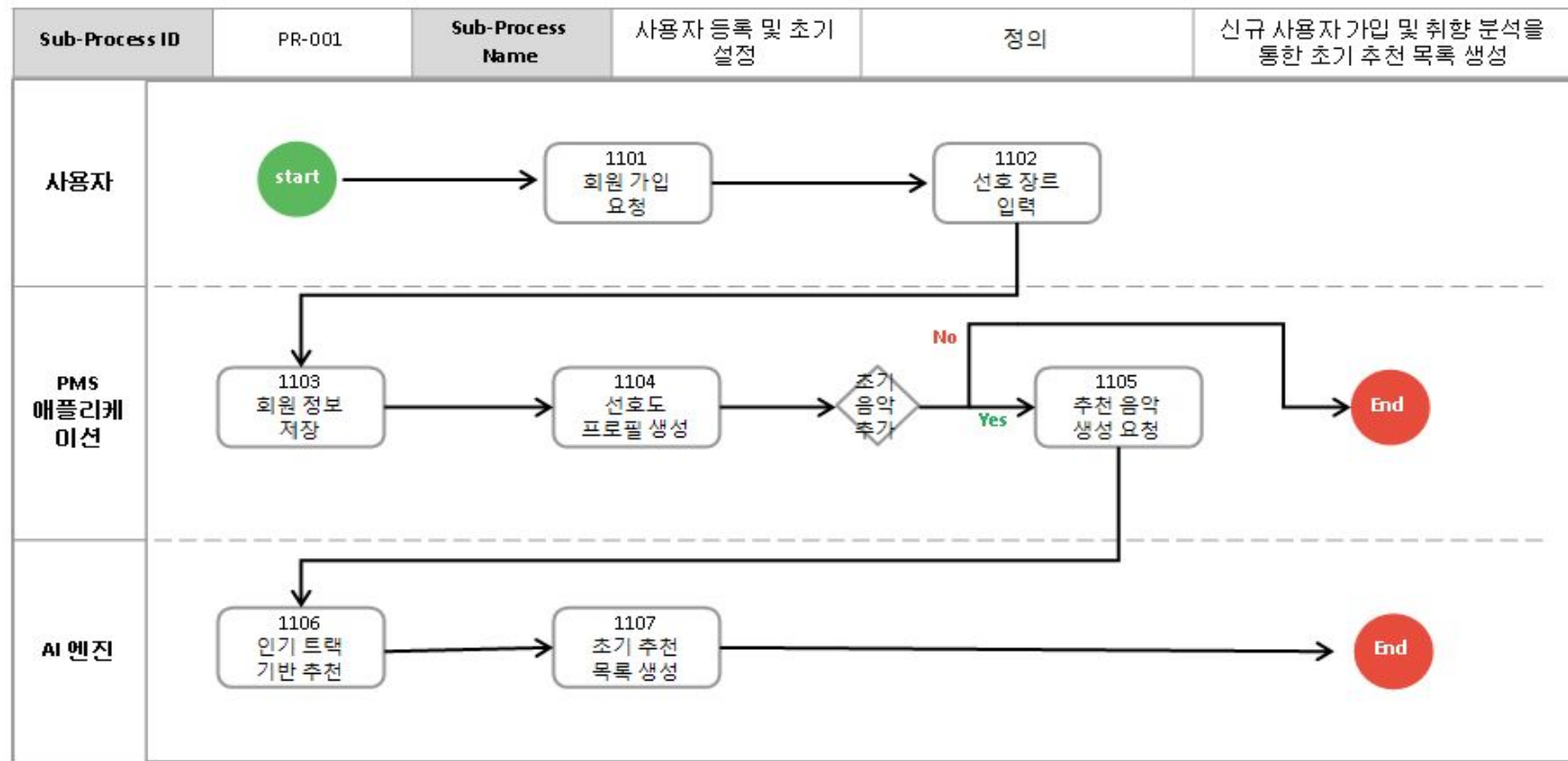
분기 혹은 구분



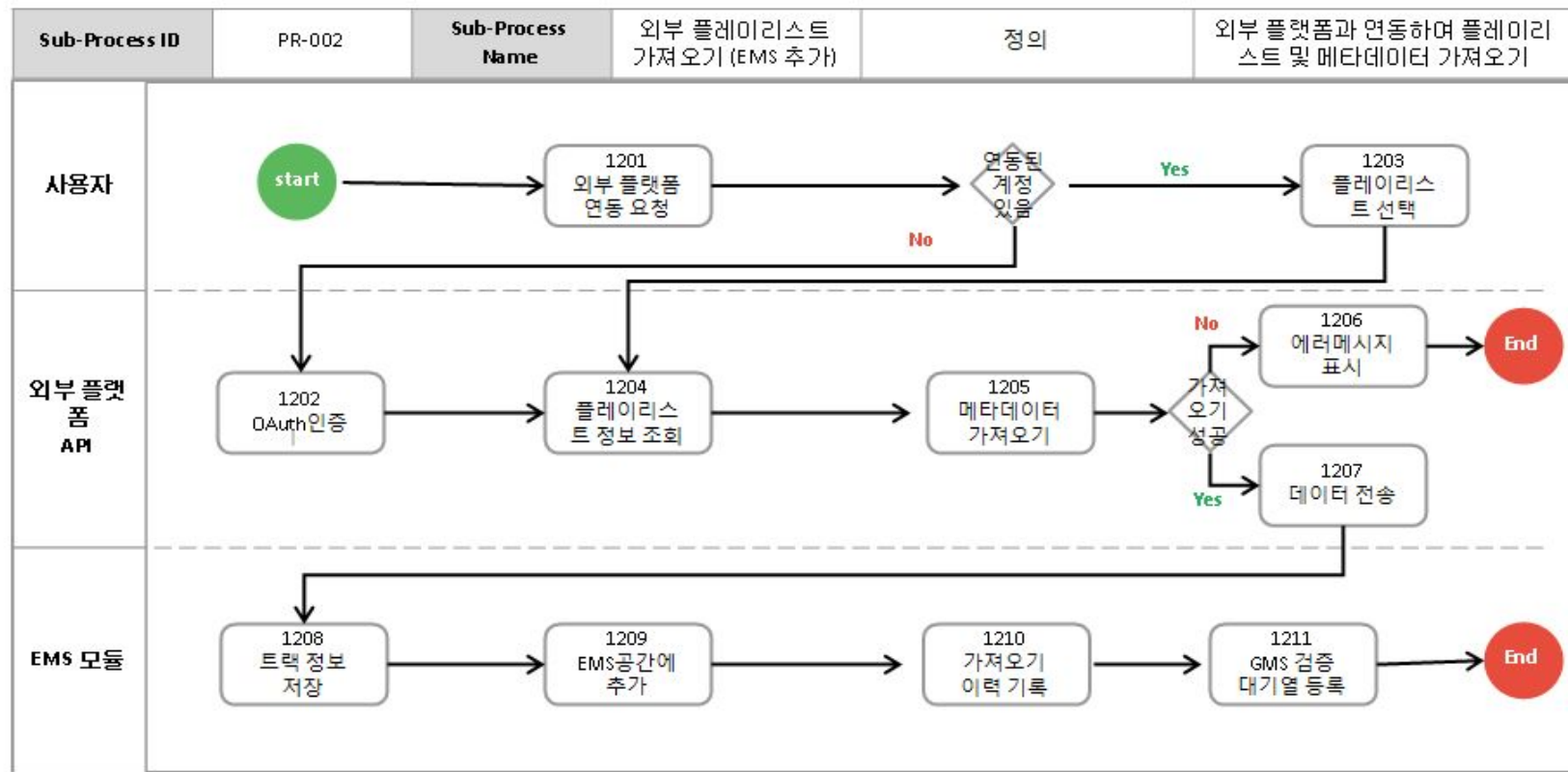
전산
프로세스



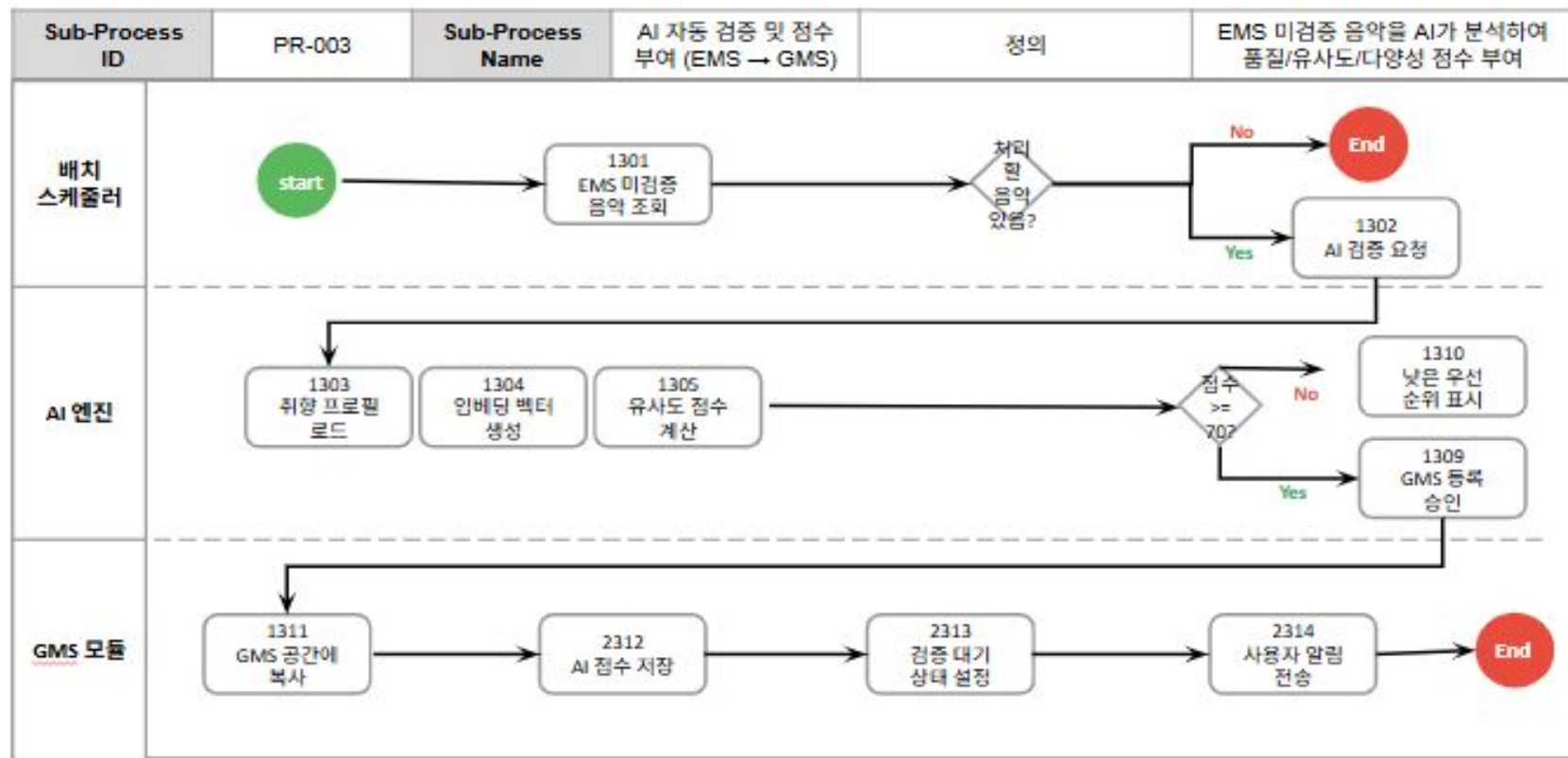
업무 흐름도 상세



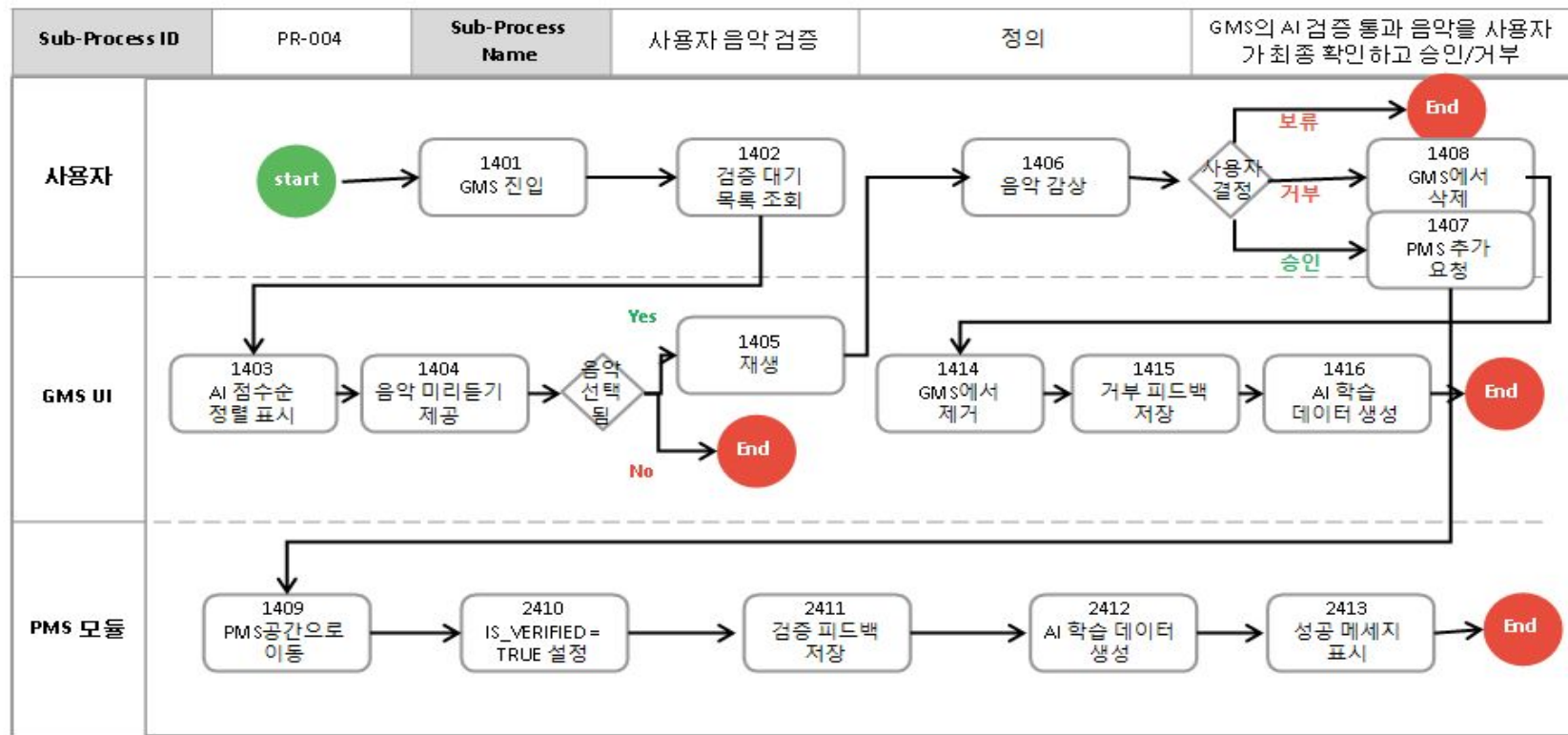
업무 흐름도 상세



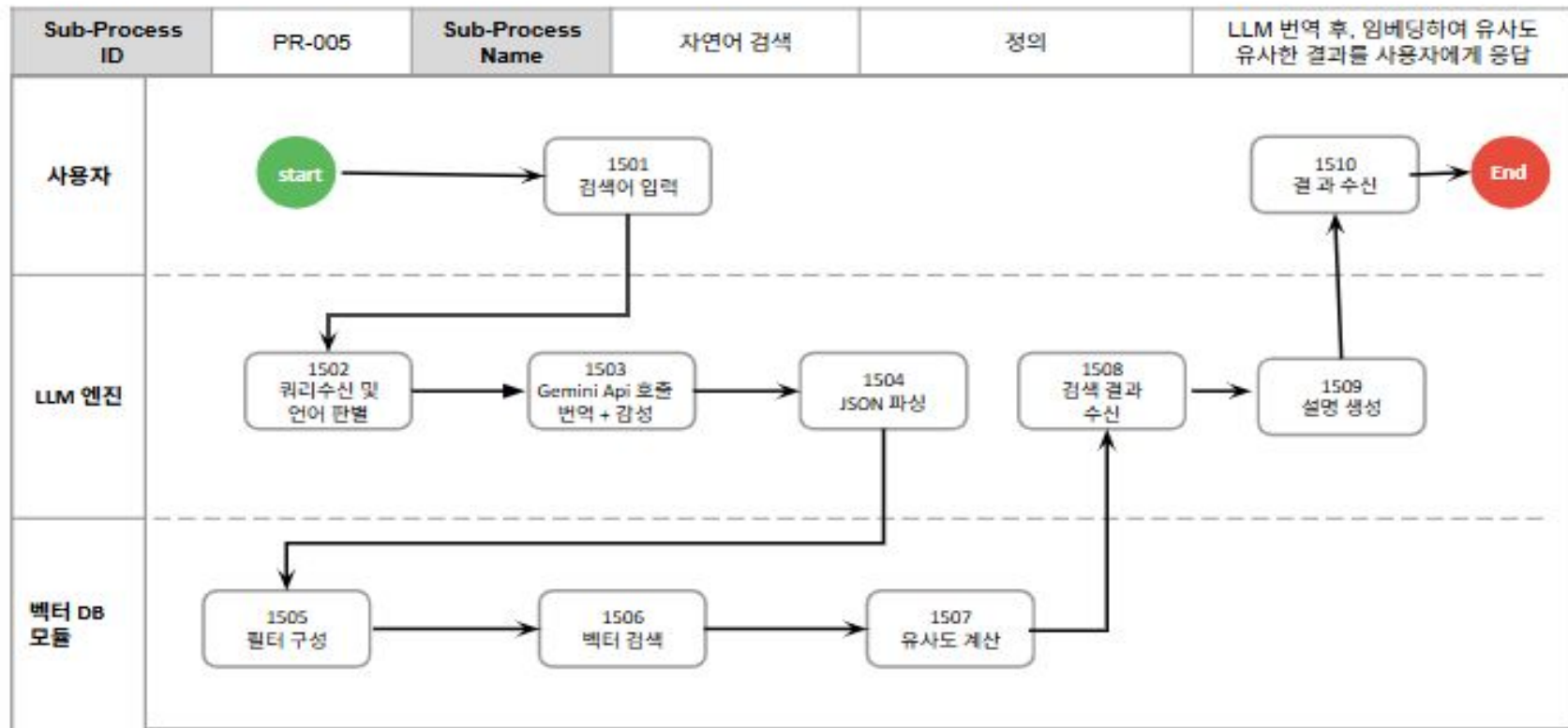
업무 흐름도 상세



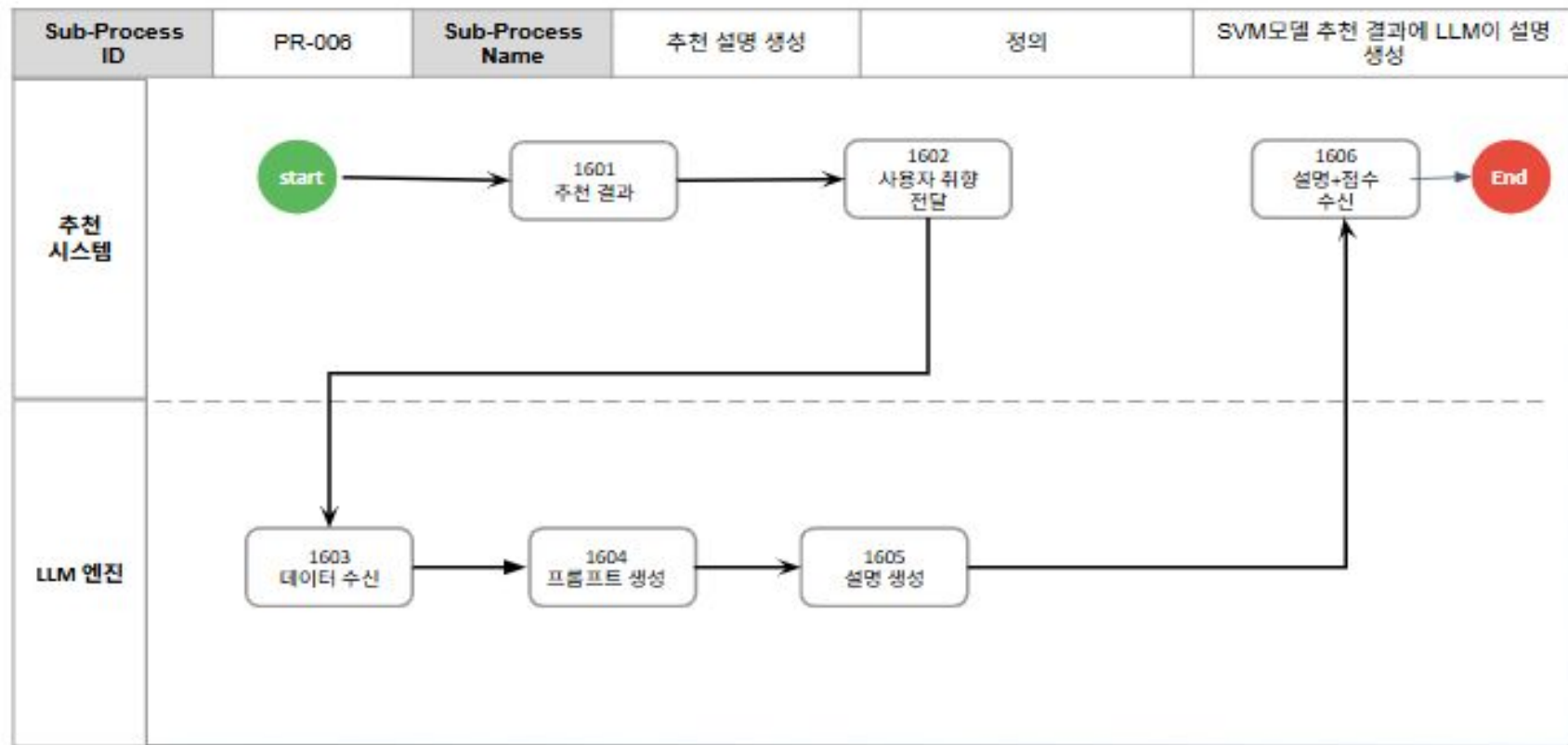
업무 흐름도 상세



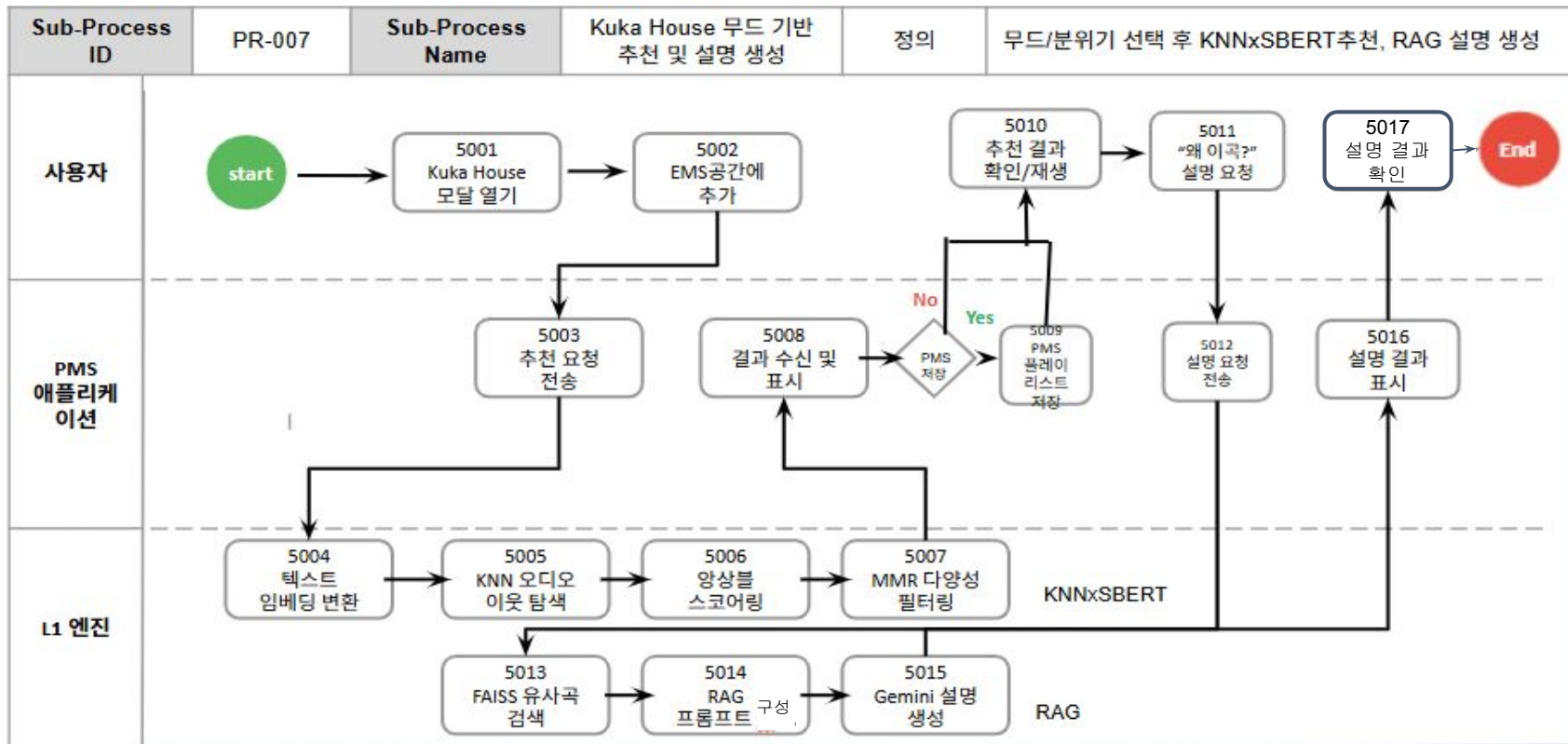
업무 흐름도 상세



업무 흐름도 상세



업무 흐름도 상세



단위 테스트

단위 테스트

No	케이스 명	메뉴 경로	케이스 내용	사전 조건	예상 결과	수행 결과	수행자
1	사용자 취향 분석	데이터 관리	Isolation Forest 이상치 탐지	PMS 데이터 로드	상위 10% 이상치 식별	Pass	조우성
2	취향 중심점 산출	데이터 관리	정제된 취향 벡터 계산	이상치 탐지 완료	일관성 있는 취향 좌표 생성	Pass	최두석
3	Ridge 엔진	추천 메인	다중 출력 회귀 트랙 예측	모델 v1 로드	오디오 수치 (MultiRMSE) 반환	Pass	정혜원
4	유사도 및 정렬	추천 메인	유클리드 거리 계산 및 정렬	중심점 산출 완료	거리순 상위 100곡 추출	Pass	조우성
5	MariaDB 적재	시스템 연동	추천 결과 및 메타데이터 저장	DB 연결 활성화	DB 테이블 정상 입력 확인	Pass	이성훈
6	부정 피드백 분류	모델 고도화	추천 하위 10곡 페널티 분리	100곡 추천 완료	재학습 제외 리스트 확보	Pass	정혜원
7	가중치 설정	모델 고도화	제외 곡에 50배 가중치 부여	제외 리스트 존재	가중치 적용 weight 생성	Pass	최두석

단위 테스트

No	케이스 명	메뉴 경로	케이스 내용	사전 조건	예상 결과	수행 결과	수행자
8	모델 v2 재학습	모델 고도화	피드백 가중치 반영 업데이트	모델 v1, v2 분기	취향 보정된 v2 모델 생성	Pass	최두석
9	LLM 사유 생성	추천 결과	Gemma3 기반 감성 사유 생성	Ollama 구동 확인	한국어 추천 근거 출력	Pass	김기열
10	LangChain 구동	AI 서비스	Prompt+LLM+Parser 체인	LangChain 설정	단계별 파이프라인 통과	Pass	김기열
11	Redis 보안 관리	시스템 보안	JWT 토큰 수명 및 보안 관리	Redis 연결 확인	토큰 만료 및 블랙리스트 작동	Pass	이성훈
12	음악 API 연동	추천 결과	추천 곡 기반 영상 ID 조회	API Key 활성화	비디오 ID 및 정보 반환	Pass	조우성

통합 테스트

통합 테스트

시나리오 ID	시나리오 명	흐름도	검증 포인트	진행 성공률	수행자
GMS_001	회원 가입 및 취향 분석	회원 정보 입력 및 초기 선호도 데이터 수집	1. 회원가입 데이터 저장 여부 2. 초기 PMS 데이터 로드 및 이상치 제거 성공 여부	100%	조우성
GMS_002	개인화 음악 추천	로그인 후 사용자 맞춤형 추천 리스트 생성	1. Ridge모델 v1 연동 확인 2. 취향 중심점 기반 유사 곡 추출 정확도	100%	정혜원
GMS_003	추천 결과 확인 및 재생	추천 리스트 조회 및 유튜브 연동 확인	1. MariaDB 저장 데이터 출력 확인 2. YouTube API를 통한 영상 매칭 여부	100%	조우성
GMS_004	AI 추천 사유 제공	LLM 기반 추천 근거 문장 생성	1. LangChain을 통한 오디오 수치 주입 확인 2. Gemma3의 한국어 추천 사유 출력 결과	100%	김기열
GMS_005	사용자 피드백 및 재학습	추천 제외 곡 설정 및 모델 고도화	1. 부정 피드백 데이터(10곡) 분리 여부 2. 50배 가중치 적용 후 모델 v2 재학습 결과	100%	최두석
GMS_006	보안 및 토큰 검증	API 요청 시 JWT 및 Redis 연동 확인	1. Access Token 만료 시 자동 차단 여부 2. Redis 내 블랙리스트 토큰 유효성 검증	100%	이성훈

통합 테스트

테스트 케이스 ID	테스트 케이스 (절차)	사전조건	테스트 데이터	예상결과	화면 ID	테스트 결과	비고
GMS_001_01	[가입]신규 회원이 정보를 입력하고 가입을 완료한다.	백엔드 서버 활성화	사용자 ID, PW, 이메일	User 테이블에 데이터가 정상적으로 적재됨	signup_page	Pass	POST /api/auth/signup
GMS_001_02	[연동]Spotify API를 통해 사용자의 과거 청취 기록(PMS)을 수집한다.	외부 API 접근 권한	사용자 플레이리스트	PMS 테이블에 곡 정보 및 오디오 피쳐 저장	login_link	Pass	GET /api/external/spotify/sync
GMS_001_03	[정제]Isolation Forest 알고리즘으로 수집된 데이터의 이상치를 제거한다.	PMS 데이터 적재 완료	오디오 수치 샘플	취향에서 벗어난 곡이 필터링된 정제 데이터 생성	analyze_view	Pass	PATCH /api/data/refine/outliers

통합 테스트

테스트 케이스 ID	테스트 케이스 (절차)	사전조건	테스트 데이터	예상결과	화면 ID	테스트 결과	비고
GMS_002_01	[분석] 정제된 데이터로부터 유저의 취향 중심점 (Vector)을 계산한다.	GMS_001 완료	pure_pms_sample	11가지 오디오 속성의 평균 취향 좌표 산출	recommend_main	Pass	POST /api/analyze/user-reference
GMS_002_02	[예측] Ridge 모델 (v1)을 호출하여 각 트랙의 선호도 수치를 예측한다	Ridge v1 모델 로드	전체 곡 Meta 데이터	MultiRMSE 기반의 트랙별 예측 벡터값 반환	recommend_engine	Pass	POST /api/analyze/user-reference
GMS_002_03	[선별] 중심점과 예측값 사이의 유클리드 거리를 계산하여 상위 곡을 추출한다.	예측 벡터값 존재	거리 계산 알고리즘	거리 기준 최적의 추천 곡 100건 리스트업	recommend_list	Pass	POST /api/analyze/user-reference

통합 테스트

테스트 케이스 ID	테스트 케이스 (절차)	사전조건	테스트 데이터	예상결과	화면 ID	테스트 결과	비고
GMS_003_01	[조회] 추천된 100곡의 리스트를 메인 화면에 렌더링한다.	GMS_002 완료	DB 추천 테이블 데이터	곡명, 아티스트, 앨범아트가 정상 노출됨	result_view	Pass	GET /api/recommend/list
GMS_003_02	[매칭] 곡 선택 시 YouTube API를 호출하여 최적의 영상 ID를 매칭한다.	YouTube API Key	곡명 + 아티스트명	검색어와 일치하는 Video ID 및 썸네일 확보	youtube_api	Pass	GET /api/external/youtube/search
GMS_003_03	[재생] 확보된 ID를 통해 YouTube 임베디드 플레이어를 구동한다.	Video ID 확보	Youtube ID	모달창 내에서 해당 영상이 끊김 없이 재생됨	player_modal	Pass	(Client-side SDK / IFrame)

통합 테스트

테스트 케이스 ID	테스트 케이스 (절차)	사전조건	테스트 데이터	예상결과	화면 ID	테스트 결과	비고
GMS_004_01	[주입] 추천 곡의 오디오 피처를 LangChain 프롬프트 템플릿에 매핑한다.	LangChain 설정 완료	에너지, 밸런스 등 수치	수치 데이터가 자연어 템플릿에 결합됨	ai_bridge	Pass	POST /api/ai/prompt/mapping
GMS_004_02	[생성] Ollama(Gemma3) 모델이 해당 데이터를 분석하여 추천 사유를 생성한다.	Ollama 서버 구동	매핑된 프롬프트 문장	데이터 근거 기반의 개인화된 추천 문구 출력	ai_docent	Pass	GET /api/ai/monitor/langsmith-stats
GMS_004_03	[검증] LangSmith 로그를 통해 생성된 사유의 품질 및 지연시간을 체크한다.	LangSmith 연동	LLM 로그 데이터	답변 품질 확인 및 병목 구간 데이터 수집	langsmith_dash	Pass	POST /api/ai/generate/reason

통합 테스트

테스트 케이스 ID	테스트 케이스 (절차)	사전조건	테스트 데이터	예상결과	화면 ID	테스트 결과	비고
GMS_005_01	[피드백] 사용자가 추천 리스트에서 제외한 곡 정보를 수집한다.	추천 리스트 노출	삭제 요청 곡 ID	삭제 대상 10곡이 피드백 데이터셋으로 분리됨	feedback_log	Pass	POST /api/feedback/exclude
GMS_005_02	[가중치] 제외 곡에 50배의 페널티 가중치를 설정하여 재학습을 준비한다.	피드백 데이터 존재	sample_weight =50	제외 곡에 에러 가중치가 반영된 벡터 생성	training_set	Pass	PATCH /api/train/weights/update
GMS_005_03	[갱신] 가중치가 반영된 신규 모델 (v2)로 추천 엔진을 업데이트한다.	CatBoost 학습 로직	X_feedback, weights	유저가 싫어하는 스타일을 피하는 v2 모델 생성	model_v2	Pass	PUT /api/engine/model/deploy-v2

통합 테스트

테스트 케이스 ID	테스트 케이스 (절차)	사전조건	테스트 데이터	예상결과	화면 ID	테스트 결과	비고
GMS_006_01	[인증] 로그인 시 JWT Access/Refresh 토큰을 생성하고 전달한다.	Spring Security 설정	유저 계정 정보	토큰이 클라이언트 쿠키/로컬스 토리지에 저장됨	auth_server	Pass	POST /api/auth/login
GMS_006_02	[관리] Redis 저장소에 Refresh 토큰을 저장하여 세션을 유지한다.	Redis 서버 가동	Refresh Token	Redis Key-Value 구조로 토큰 정보가 기록됨	redis_session	Pass	POST /api/auth/refresh
GMS_006_03	[보안] 유효하지 않거나 만료된 토큰 요청 시 API 접근을 차단한다.	JWT 검증 필터	Expired Token	401 Unauthorized 에러 반환 및 접근 거부	api_gateway	Pass	-